

Angular

Angular is a powerful, open-source web application framework for building dynamic and scalable [single-page applications](#) (SPAs). Developed by Google, Angular provides a comprehensive solution for front-end development with tools for routing, form handling, HTTP services, and more.

- Designed for building scalable web applications
- Component-based architecture for efficient development
- Supports powerful features like [two-way data binding](#), routing, and [dependency injection](#)
- Ideal for single-page applications (SPAs) and progressive web apps (PWAs)

To begin developing with Angular, you first need to set up your development environment by installing Node.js and Angular CLI.

```
// Import Angular core modules
import { Component } from '@angular/core';

// Define the AppComponent
@Component({
  selector: 'app-root',
  template: '<h1>Welcome to the Angular Tutorial</h1>',
  styles: ['h1 { color: blue; }']
})
export class AppComponent {
  title = 'angular-tutorial';
}
```

In this example

- The AppComponent class is decorated with @Component, which defines its selector, template, and styles.
- The template uses Angular's interpolation syntax to display a simple "Welcome to the Angular Tutorial" message.
- The AppComponent class defines the component logic and properties.

Why use Angular?

- **Simplifies Front-End Development:** Angular provides all the tools needed for front-end development in a single framework, making it easier to build dynamic and scalable applications.
- **Component-Based Architecture:** Angular's component-based structure helps break down large applications into smaller, manageable parts, making development and maintenance easier.

- **Powerful Features:** Angular offers powerful features such as two-way data binding, routing, and form validation that speed up the development process.
- **Seamless Integration:** Angular integrates well with RESTful APIs, making it ideal for building full-stack web applications and SPAs.
- **Strong Community Support:** With strong backing from Google and an active community, Angular is continuously updated and improved, ensuring long-term stability.

Angular - MVC Architecture

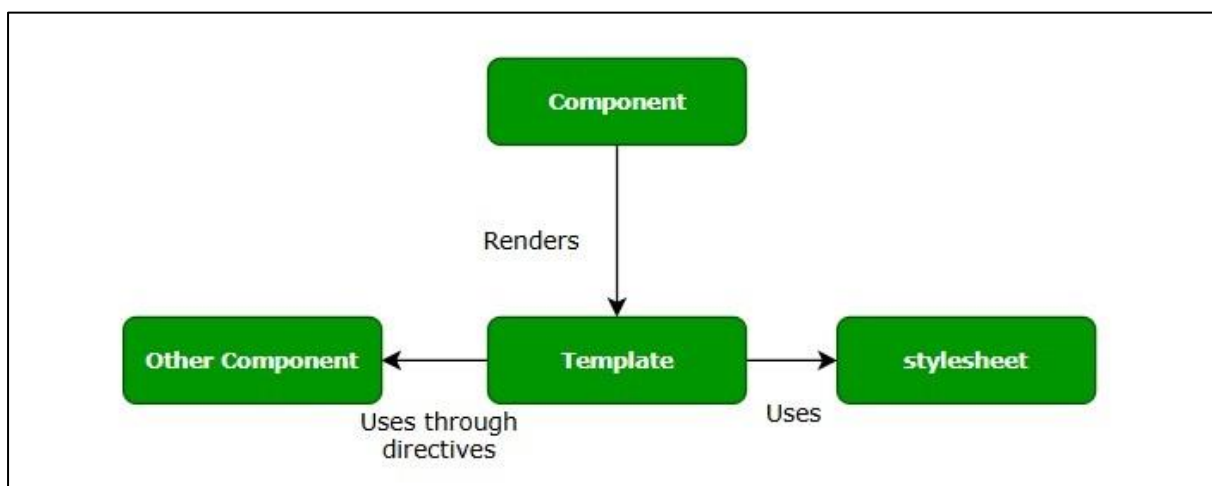
Angular framework is based on several core concepts and they are as follows –

- [Components](#)
- [Templates](#) with **Data binding** and **Directives**
- [Modules](#)
- [Services and dependency injection](#)
- [Metadata](#)

Component:

The core of the Angular framework architecture is **Angular Component**. Angular Component is the building block of every Angular application. Every angular application is made up of one more **Angular Component**. It is basically a plain JavaScript/Typescript class along with a HTML template and an associated name.

The HTML template can access the data from its corresponding JavaScript/Typescript class. Component's HTML template may include other component using its selectors value (name). The Angular Component may have an optional CSS Styles associated it and the HTML template may access the CSS Styles as well.



Let us analyse the **AppComponent** component in our **ExpenseManager** application.
The **AppComponent** code is as follows –

```
// src/app/app.component.ts

import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'expense-manager';
}
```

In the above code block,

- **@Component:** A decorator used to convert a normal Typescript class to **Angular Component**.
- **app-root:** It is the selector/name of the component and it is specified using **selector** meta data of the component's decorator.
- **app.component.html:** It is the HTML template document associated with the component. The component template is specified using **templateUrl** meta data of the **@Component** decorator.
- **AppComponent:** Its property (title) is used in the HTML template to set the title of the application.
- **app.component.css:** This is the CSS style document associated with the component. The component style is specified using **styleUrls** meta data of the **@Component** decorator.

To show the view of this component, the **app-root** selector is used by root document, i.e. **src/index.html** of the Angular application as shown below –

Template

Template is basically a super set of HTML. Template includes all the features of HTML and provides additional functionality to bind the component data into the HTML and to dynamically generate HTML DOM elements.

The core concept of the template can be categorised into two items and they are as follows –

Data binding

Used to bind the data from the component to the template.

```
{{ title }}
```

Here, **title** is a property in **AppComponent** and it is bind to template using **Interpolation**.

Directives

Used to include logic as well as enable creation of complex HTML DOM elements.

```
<p *ngIf="canShow">
```

This section will be shown only when the **canShow** property's value in the corresponding component is **true** </p>

```
<p [showToolTip]='tips' />
```

Here, **ngIf** and **showToolTip** (just an example) are directives. **ngIf** create the paragraph DOM element only when **canShow** is true. Similarly, **showToolTip** is **Attribute Directives**, which adds the tooltip functionality to the paragraph element.

When a user hover mouse over the paragraph, a tooltip will be shown. The content of the tooltip comes from tips property of its corresponding component.

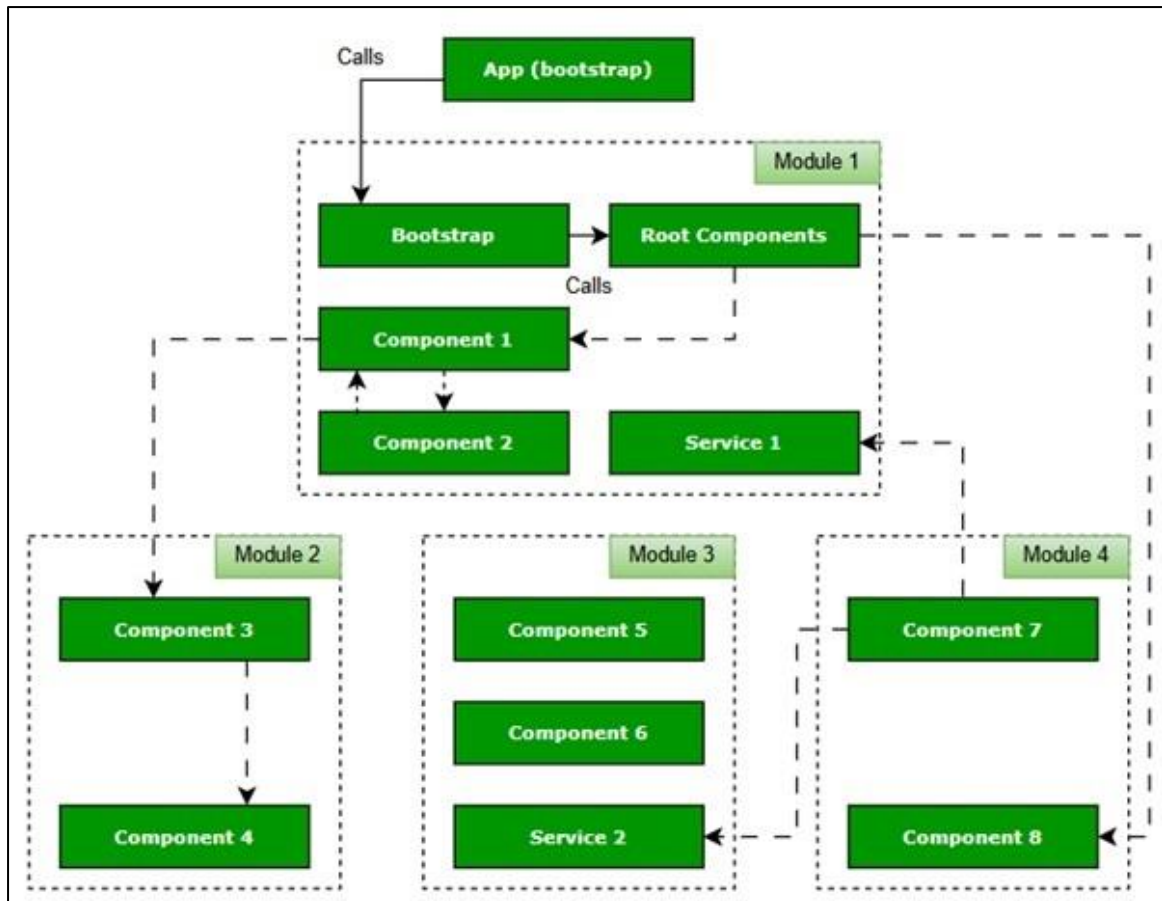
Modules

Angular Module is basically a collection of related features/functionality. It groups multiple components and services under a single context.

For example, animations related functionality can be grouped into single module and Angular already provides a module for the animation related functionality, **BrowserAnimationModule** module.

An Angular application can have any number of modules but only one module can be set as root module, which will bootstrap the application and then call other modules as and when necessary. A module can be configured to access functionality from other module as well. In short, components from any modules can access component and services from any other modules.

Following diagram depicts the interaction between modules and its components.



Let us check the root module of our **Expense Manager** application.

```

import { BrowserModule } from '@angular/platform-browser';
import { NgModule } from '@angular/core';
import { AppComponent } from './app.component'; @NgModule({
  declarations: [
    AppComponent
  ],
  imports: [
    BrowserModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }

```

Here,

- **NgModule** decorator is used to convert a plain Typescript/JavaScript class into **Angular module**.
- **declarations** option is used to include components into the **AppModule** module.
- **bootstrap** option is used to set the root component of the **AppModule** module.
- **providers** option is used to include the services for the **AppModule** module.
- **imports** option is used to import other modules into the **AppModule** module.

Services

Services are plain Typescript/JavaScript class providing a very specific functionality. They will do a single task and do it best. The main purpose of the service is to make a certain feature reusable. Instead of writing a functionality inside a component, separating it into a service will make it usable in other component as well.

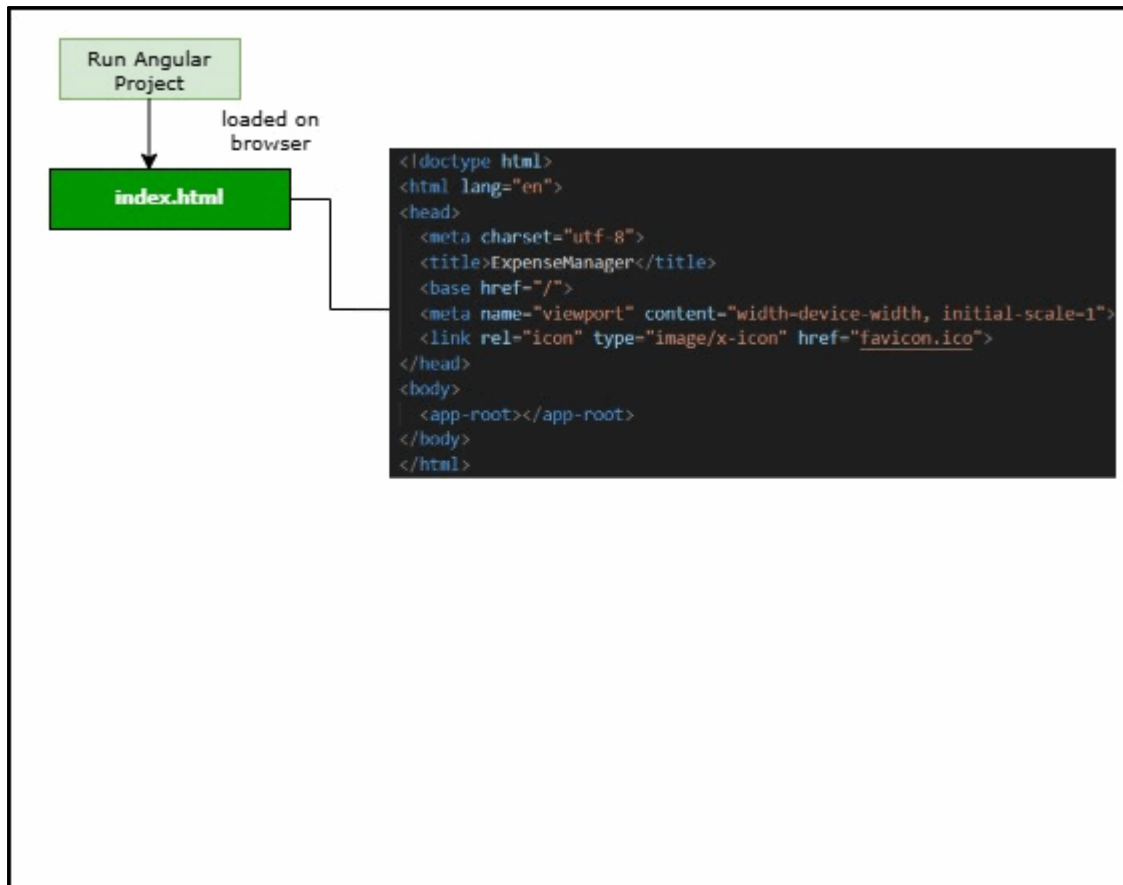
Also, **Services** enables the developer to organize the business logic of the application. Basically, component uses services to do its own job. **Dependency Injection** is used to properly initialize the service in the component so that the component can access the services as and when necessary without any setup.

Metadata

In Angular, metadata is used to provide additional information about a class, component, or service. This information helps Angular understand how to process and use these elements within the application. Metadata is defined using decorators, which are special functions that associate metadata to a class.

Workflow of Angular Application

We have learned the core concepts of Angular application. Let us see the complete flow of a typical Angular application.



When we run an Angular application, **index.html** is the first file that is loaded on the browser. Then, browser looks for the main TypeScript file, i.e. **src/main.ts** which is the entry point of Angular application.

Now, this file bootstraps the **AppComponent** (**src/app.component.ts**), the root component of every Angular application.

The **AppComponent** renders its template (**src/app.component.html**) and uses the corresponding styles (**src/app.component.css**). **AppComponent** name, i.e., **app-root** is used inside the **src/index.html** so that view of the angular application can be rendered.

```
<!doctype html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="utf-8">
```

```
<title>ExpenseManager</title>
```

```
<base href="/">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

```
<link rel="icon" type="image/x-icon" href="favicon.ico">
```

```
</head>
```

```
<body>
  <app-root></app-root>
</body>
</html>
```

A component can use another component through directive in its template using target component's selector name.

```
<component-selector-name></component-selector-name>
```

Also, all registered services are accessible to all Angular components through **Dependency Injection (DI)** framework.

NOTE: For the complete workflow of a non-standalone angular application.

Angular - Components

Components are the building blocks of an Angular application. The primary use of Angular Component is to generate a section of web page called **View**. By combining different views, a single application is created. Every component will have an associated template and it will be called in order to generate views.

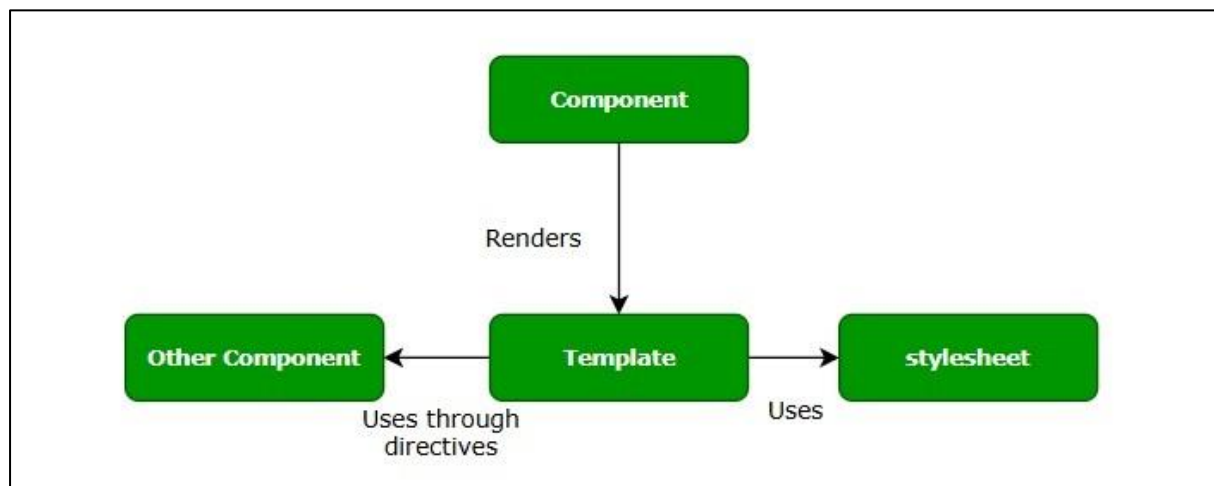
Let us learn the basic concept of components in this tutorial.

Structure of Angular Component

Each component of an Angular application has a few important parts which are as follows –

- **@component Decorator:** All the application related configurations are written inside this decorator.
- **HTML Template:** View of the Angular application.
- **Styles:** It controls the styles of a view.
- **TypeScript Class:** Code related to behavior of the Angular application goes into this class.

By default, all these parts are created by Angular CLI, you can update, add or even delete them if any of them is not required.



How to Create Components in Angular?

In Angular, a new Component is created using the **ng generate component** command as specified below –

```
ng generate component name-of-component
```

Example

Let's see an example where we create a new component in our **ExpenseManager** application. This component will contain our first expense entry. First, open the command prompt and navigate to **ExpenseManager** application.

```
cd expense-manager
```

Now, use the command given below to create a **expense-entry** component –

ng generate component expense-manager

Following files and folders will be created by Angular CLI on the above command –

CREATE src/app/expense-entry/expense-entry.component.html (28 bytes)

CREATE src/app/expense-entry/expense-entry.component.spec.ts (671 bytes)

CREATE src/app/expense-entry/expense-entry.component.ts (296 bytes)

CREATE src/app/expense-entry/expense-entry.component.css (0 bytes)

UPDATE src/app/app.module.ts (431 bytes)

Here,

- **ExpenseEntryComponent** is created under src/app/expense-entry folder.
- Component class, Template and stylesheet are created.

Next, we add a title property to **ExpenseEntryComponent**, i.e., (src/app/expense-entry/expense-entry.component.ts) component.

```
import { Component, OnInit } from '@angular/core';
```

```
@Component({
  selector: 'app-expense-entry',
  standalone: true,
  imports: [ExpenseEntryComponent],
  templateUrl: './expense-entry.component.html',
  styleUrls: ['./expense-entry.component.css']
})
export class ExpenseEntryComponent implements OnInit {
  title: any;
  constructor() {}
  ngOnInit(): void {
    this.title = "Expense Entry"
  }
}
```

Update template, **src/app/expense-entry/expense-entry.component.html** with below content.

```
<h3>{{title}}</h3>
```

```
<ul>
```

```
<li><b>Item:</b> Pizza</li>
<li><b>Amount:</b> 20</li>
<li><b>Category:</b> Food</li>
<li><b>Location:</b> Zomato</li>
<li><b>Spend On:</b> Dec 2024</li>
</ul>
```

Open **src/app/app.component.html** and include the newly created component.

```
<h1>Expense Management Application</h1>
<app-expense-entry></app-expense-entry>
<router-outlet />
```

Here,

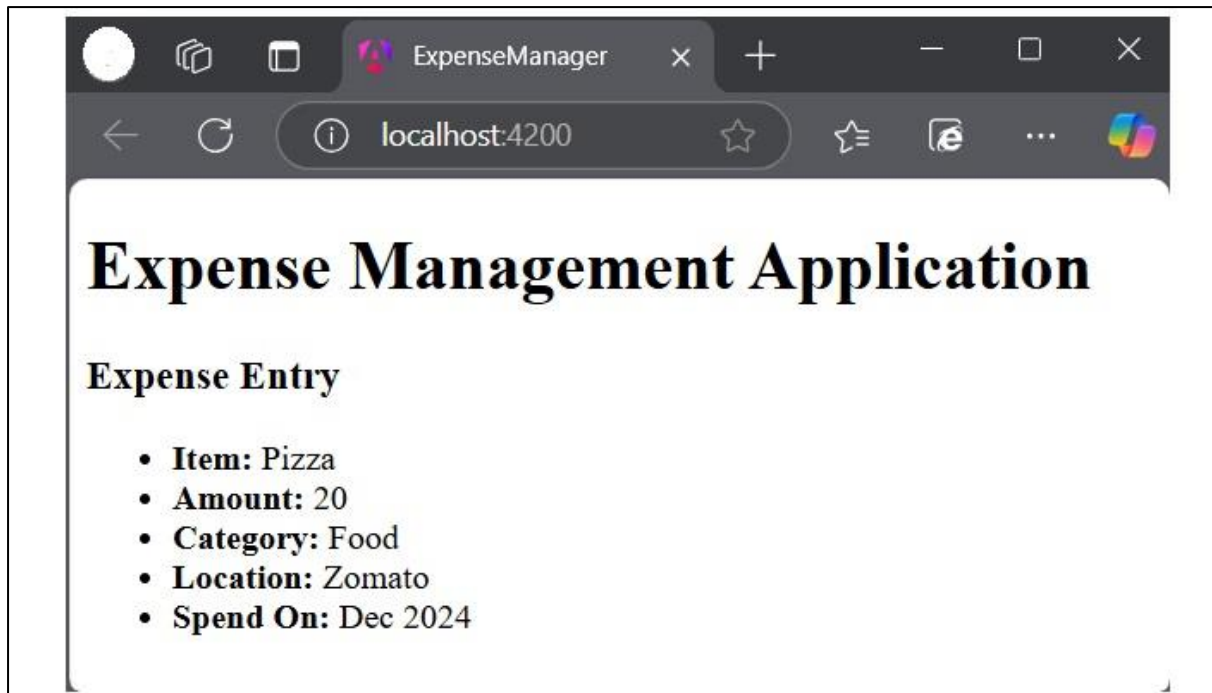
app-expense-entry is the selector value and it can be used as a regular HTML Tag.

At the end, import **ExpenseEntryComponent** to the **app.component.ts** file as shown below –

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { ExpenseEntryComponent } from './expense-entry/expense-entry.component';
```

```
@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, ExpenseEntryComponent],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'expense-manager';
}
```

Finally, the output of the application is –



Component Lifecycle Hook

Angular component goes through a series of stages/events during its existence. The different stages of the Angular [Components Lifecycle](#) are creation, change detection, rendering and destruction.

Each phase of the angular component is associated with a lifecycle hook interface which can be implemented to perform arbitrary action in that particular phase. The lifecycle hooks refer to the methods of lifecycle hook interfaces.

Component Interaction

[Component interaction](#) is one of the important and necessary features in the context of component based architecture. Angular provides multiple options to pass and receive data between components.

You can share data from parent component to child component as well as, child to parent component. Also, it is possible to share data between any other component within the Angular application.

In Angular, parent and child components interacts through the following ways –

- @Input decorator
- @Output decorator
- local variable
- @ViewChild decorator
- Services

Component Styling

[Component styling](#) is the process of designing and formatting the visual presentation of views or components. You can use the following ways for styling:

- Using "styles"
- Using "styleUrls"
- Styling through template
- Using global styles
- Using CSS preprocessor
- Customized styles
- Using custom selectors

Nested Components

[Nested components](#) are normal Angular Components with parent-child relations. The parent can access and share data with the child, either partially or fully. The component nested inside another component is called child component. The component containing the child component is called the parent component.

Dynamic Components

Angular allows the component to be dynamically created and loaded at run time at a specific location in the host (another) component.

You can create [dynamic components](#) in Angular using the following ways –

- Using NgComponentOutlet
- Using ViewContainerRef

Angular - Component Lifecycle

Angular [component](#) goes through a series of stages/events during its existence. Before moving further in the journey of learning Angular, it is necessary to understand how a component interacts with the framework and the DOM throughout its existence.

When an angular component is constructed, it first goes through the change detection phase, where it checks whether there are any changes in the input and then acts accordingly. Then, the initialization phase kicks on and continues to other phases and finally gets destroyed in the destruction phase.

Component Lifecycle Phases in Angular

The different stages of the Angular Components Lifecycle are as follows –

- **Creation:** It is the first phase where a component is instantiated.
- **Change Detection:** Then, Angular try to detect changes in the View and Content of the application.
- **Rendering:** After change detection, the new template is updated.
- **Destruction:** The Component is destroyed at the end.

Component Lifecycle Hook in Angular

Each phase of the angular component is associated with a lifecycle hook interface which can be implemented to perform arbitrary action in that particular phase. Generally, the lifecycle hooks refer to the methods of lifecycle hook interfaces. Let's see the phases, their sequence and the corresponding hooks.

Phases	Lifecycle Hooks	Description
Creation	Constructor	Constructor runs when Angular instantiates the component for the first time.
Change Detection	ngOnChanges()	Change detection is the first phase, where the angular component will check the inputs for changes and act accordingly. It has a corresponding lifecycle hook, ngOnChanges() . This hook runs before the ngOnInit() during the first initialization process.
	ngOnInit()	The ngOnInit() lifecycle hook runs exactly once after ngOnChanges() . It is raised to do the necessary initialization process based on the initial input.
	ngDoCheck()	Next, Angular tries to detect the changes in the component and act accordingly. The lifecycle hook used for checking is ngDoCheck() . This hook is invoked even if there is not change in the input bound properties. Avoid defining this hook as it might affect the page's performance.
	ngAfterContentInit()	This lifecycle hook is called only once after the initialization of all children nested inside the content of component.
	ngAfterContentchecked()	It is invoked during every change detection phase after the children nested inside the component's content have been checked for changes.
	ngAfterViewInit()	Next is the view initialization phase, where angular sets the various child views of the component template. The lifecycle hook used for view initialization phase is ngAfterViewInit() .
	ngAfterViewchecked()	Now, Angular tries to detect the changes in the view of the component/directive. The lifecycle hook for view checking phase is ngAfterViewchecked() .
Rendering	afterNextRender()	It runs only once when all components have been rendered to the DOM.
	afterRender()	Runs every time after all components have been rendered to the DOM.
Destruction	ngOnDestroy()	In the final phase, ngOnDestroy() hook is called to destroy the component/directive.

Execution Order of Component Lifecycle Hooks

Let us see the lifecycle sequence of an arbitrary component/directive through its hooks.

- `ngOnChanges`
- `ngOnInit`
- `ngDoCheck`
- `ngAfterContentInit`
- `ngAfterContentChecked`
- `ngAfterViewInit`
- `ngAfterViewChecked`
- `ngOnChanges`
- `ngDocheck`
- `ngAfterContentChecked`
- `ngAfterViewchecked`
- Repeation of Step 8 - 11 until destruction
- `ngOnDestroy`

Example

Let us create a new component, **MyLifecycleComponent**, wire up all hooks and check the sequence of the lifecylce using console output.

Step 1: Create a new component using angular CLI as shown below –

ng generate component my-lifecycle-sample

This will create a new component and its related template and styles as shown below.

\$ ng generate component my-lifecycle-sample

CREATE src/app/my-lifecycle-sample/my-lifecycle-sample.component.css (0 bytes)

CREATE src/app/my-lifecycle-sample/my-lifecycle-sample.component.html (34 bytes)

CREATE src/app/my-lifecycle-sample/my-lifecycle-sample.component.spec.ts (638 bytes)

CREATE src/app/my-lifecycle-sample/my-lifecycle-sample.component.ts (252 bytes)

Step 2: Add all lifecycle hooks into the component and log messages:

```
import { Component, OnChanges, OnInit, DoCheck, AfterContentInit, AfterContentChecked,
AfterViewInit, AfterViewChecked, OnDestroy } from '@angular/core';
```

```
@Component({
  selector: 'app-my-lifecycle-sample',
```

```
    templateUrl: './my-lifecycle-sample.component.html',
    styleUrls: ['./my-lifecycle-sample.component.css']
  })
export class MyLifecycleSampleComponent implements OnChanges, OnInit {

  ngOnChanges() {
    console.log("Change detection")
  }

  ngOnInit() {
    console.log("Initialization of component / directive")
  }

  ngDoCheck() {
    console.log("Custom change detection")
  }

  ngAfterContentInit() {
    console.log("Content initialization")
  }

  ngAfterContentChecked() {
    console.log("Checking changes in content")
  }

  ngAfterViewInit() {
    console.log("View initialization")
  }

  ngAfterViewChecked() {
    console.log("Checking changes in views")
  }
}
```



```
ngOnDestroy() {  
  console.log("Destruction of component / directive")  
}  
}
```

Step 3: Add the component in the app components template **app.component.html**.

```
<app-my-lifecycle-sample>  
</app-my-lifecycle-sample>  
<router-outlet></router-outlet>
```

Run the application using **ng serve** and test the console through developer tool in the browser. It will show all the lifecycle events executing in the above discussed order.

Initialization of component / directive

Custom change detection

Content initialization

Checking changes in content

View initialization

Checking changes in views

Angular is running in development mode.

Custom change detection

Checking changes in content

Checking changes in views

Custom change detection

Checking changes in content



ExpenseManager

localhost:4200

Expense Management Application

Expense Entry

- **Item:** Pizza
- **Amount:** 20
- **Category:** Food
- **Location:** Zomato
- **Spend On:** Dec 2024

Angular - Directives

Angular **directives** are classes that enhance the feature of a HTML element or component and add additional behaviour to the web application. They can transform a static HTML page into dynamic by manipulating DOM. They begin with **ng**, which stands for Angular.

The **@directive** decorator is used to mark a [TypeScript](#) class as an Angular Directive. This decorator contains some configuration metadata that determines how the directive should be processed, instantiated and used at runtime.

HTML DOM model is constructed as a tree of objects. It is a standard object model to access HTML elements.

Features and Uses of Angular Directives

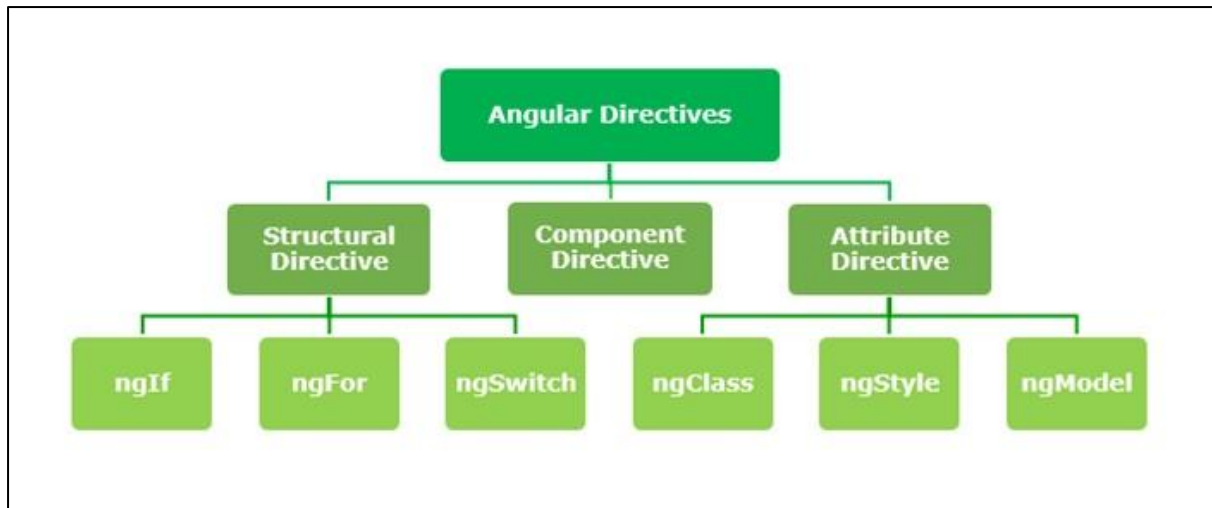
Following are the features and uses of Angular Directives –

- Using directives, you can create your custom HTML elements, attributes and classes.
- They allow developers to create the required features once and reuse them across different parts of the Angular application.
- It became easy to update and maintain the code as we didn't need to write code for the same feature again and again.
- Directives can control the DOM elements.

Types of Directives

Directives are categorized based on the type of feature it provides to the HTML element/component. The type of directive and its purpose are as follows:

- **[Components](#):** Component is basically a type of directive. As we know, they can generate a a piece of HTML document in memory (DOM structure), called **View**. The view will have both design and event based dynamic functionality.
- **[Attribute directives](#):** Attribute directives provides additional feature to a HTML element/component (host) with respect to appearance and behavior. For example, a menu component attached to a attribute directive can show next level of menu upon hovering the component.
- **[Structural directives](#):** Structural directives can change the entire layout of the host HTML element/component by adding or removing the component's DOM elements.



Structural Directives

[Structural directives](#) change the structure of **DOM** by adding or removing elements. It is denoted by an **asterisk (*)** symbol with three pre-defined directives **ngIf**, **ngFor** and **ngSwitch**. Let's understand one by one in brief.

The list of commonly used structural directives are:

- **ngIf** – This directive is used to display or hide data in your application. When the given condition becomes TRUE, it will display the data, otherwise not. We can add this to any tag in our template.
- **ngFor** – ngFor is used to repeat a portion of elements from the given list of items.
- **ngSwitch** – It checks multiple conditions.

Attribute Directives

[Attribute directives](#) change the appearance or behavior of DOM elements or components. It is used just like a normal HTML attribute. However, the directive should be enclosed within square brackets [] to bind it to the element.

The most commonly used attribute directives are as follows:

- **ngStyle** – It is used to add dynamic styles.
- **ngClass** – It adds or removes CSS classes in HTML elements.
- **ngModel** – This directive is used for two-way binding.

Component Directives

Each [component](#) of an Angular application is a directive itself. It is a special directive with Views. Also, it has **@Input** and **@Output** decorator to send and receive information between parent and child components.

Syntax

To create a component for your Angular application, use the command given below –

ng generate component component-name

Difference between Component and Directive

The table below shows how Components are different from Directives –

Components	Directives
Components in Angular are used to create UI elements and manage their state.	Directives in Angular are classes that can modify the behavior or appearance of existing DOM elements.
They create reusable UI elements.	They create reusable features and behaviors for certain elements.
The @Component decorator is used to declare a Component.	The @Directive decorator is used to declare a Directive.
There is only one component for each DOM element.	There can be one or more directives for each DOM element.

Custom Directives

A [custom directive](#) is a user-defined directive that allows developers to extend the functionality of HTML elements. The attribute and structural built-in directives (covered in previous two chapters) offers very basic and pre-defined functionalities. However, with custom directives, you can add specific behaviors to HTML elements based on project requirements, user interactions, or changes in data.

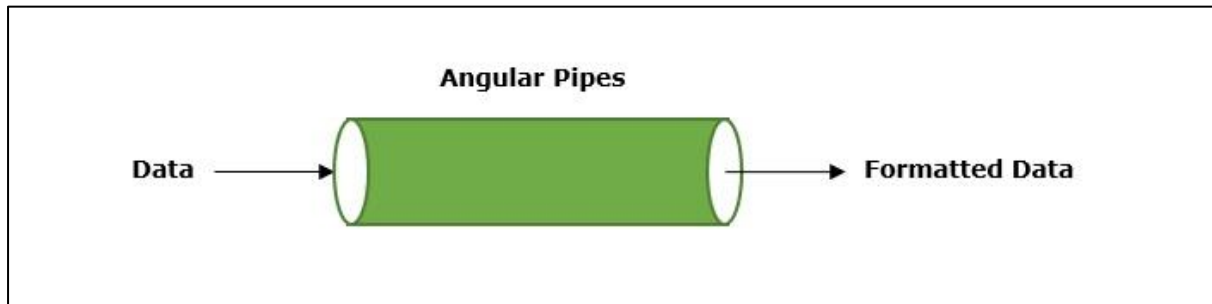
To create a custom directive, run the following command in Angular CLI –

ng generate directive <directive-name>

Angular - Pipes

What are Angular Pipes?

In [Angular](#), **pipes** are functions that format specified data before displaying it in the View. They help to transform and manage data within interpolation, denoted by `{{ | }}`. Therefore, pipes are sometimes referred to as filters. It accepts arrays, integers and strings as inputs which are separated by a vertical bar `"|"` symbol.



Features and Uses of Angular Pipes

Some of the features and uses of Angular pipes are listed below –

- Pipes can apply transformations directly within the HTML element.
- Once a custom pipe is created, it can be reused throughout the Angular application.
- Multiple pipes can be chained together. The output of one pipe can be passed as input to another pipe.
- You can use pipes to format and transform numbers, objects, strings, and dates in a way that is suitable for display in the UI.
- Pipes are used to filter data which means it can show only certain items from a list based on passed conditions.

How to Use Angular Pipes?

To use Angular pipes in your application, embed the pipe directly inside **template expression**. This is done using Angular's pipe operator which is denoted by a vertical bar character `"|"`.

The **pipe operator** is a binary operator. On the left of this operator, the input value is passed to the transformation function, and the right side operand is the pipe name.

All the built-in pipes in Angular are available in the **@angular/common** package. Hence, make sure to import the required pipe from this package by specifying the pipe name in the following command –

```
import { Pipe-Name } from '@angular/common';
```

Syntax

The syntax to use Angular pipe is as follows –

```
<html-tag-name>{{ input-value | pipe-name }}</html-tag-name>
```

Where,

html-tag-name can be replaced by any HTML tag, **input-value** is the input that will be passed into the pipe on the right side of pipe operator.

Chaining Angular Pipes

Chaining multiple pipes together is also possible. The output of one pipe can be passed as input to another pipe. And, the chained pipes run from left to right.

The syntax to chain Angular pipes is as follows –

```
<html-tag-name>{{ input-value | pipe1 | pipe2 }}</html-tag-name>
```

Passing Parameters to Angular Pipes

Some Angular pipes allow to configure the transformation by passing parameters. To specify a parameter, append the pipe name with a colon (:) followed by the parameter value.

The syntax to add parameters to Angular pipes is shown below –

```
<html-tag-name>{{ input-value | pipe : parameter }}</html-tag-name>
```

Working Example

The following example illustrates how to use the **DatePipe** in your Angular application to format dates.

Step 1: Create a **pipe-app** application using the below command:

```
ng new pipe-app
```

Step 2: Add the below content in **app.component.html** file.

```
<div>
  Today's date :- {{presentDate}}
</div>
```

Step 3: Import **@angular/common** package and create an Date object inside **app.component.ts** file –

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { DatePipe } from '@angular/common';
```

```
@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, DatePipe],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
```

```
})  
  
export class AppComponent {  
  title = 'pipe-app';  
  presentDate = new Date();  
}
```

Step 4: Start your application using the below command –

ng serve

It will display the following output on your browser –

Today's date :- Tue Jan 07 2025 11:52:26 GMT+0530 (India Standard Time)

Step 5: Let's add date pipe in the HTML file.

```
<div>  
  Today's date :- {{presentDate | date }}  
</div>
```

Now, you will see the below output on your screen –

Today's date :- Jan 7, 2025

Step 6: Pass **fullDate** parameter to the date pipe as shown below –

```
<div>  
  Today's date :- {{presentDate | date: 'fullDate' }}  
</div>
```

Step 7: Now, run your application and you can see the below response –

Today's date :- Tuesday, January 7, 2025

Angular Built-in Pipes

Angular supports a number of [built-in pipes](#), which are as follows –

SNo.	Pipes & Description
1.	DatePipe It is used to format a given date value.
2.	UpperCasePipe It converts individual letters of the specified texts into uppercase.
3.	LowerCasePipe It transforms individual letters of the specified texts into lowercase.

4.	CurrencyPipe This pipe is used to transform a given number into a currency string.
5.	DecimalPipe This pipe formats a number into a string of decimal point number.
6.	PercentPipe It formats specified numbers into a percentage string.
7.	TitleCasePipe It is used to convert the specified text to a title case.
8.	JsonPipe This built-in pipe is used to transform an object to its corresponding JSON string representation.
9.	KeyValuePipe Use this pipe to create an array of key-value pairs from a given object or map.
10.	SlicePipe It returns a new sub-string from the specified string.

Angular Custom Pipes

A [custom pipe](#) is a user-defined pipe that allows developers to perform specific transformations that Angular's built-in pipes can't achieve. The built-in pipes provide limited functionalities like formatting dates, numbers and strings. However, you can achieve more than that using custom pipes. For example, sorting and filtering.

To create a custom pipe, run the following command in Angular CLI –

ng generate pipe <pipe-name>

Angular - Forms

Forms are used to collect input data from users and enable users to interact with the application. A general form consists of various input fields such as text boxes, radio buttons, checkboxes, and dropdowns, along with a submit button that triggers the action of sending data to a server or performing some other operation within the application.

In this tutorial, we will learn what are Angular forms, how they work and their use cases.

Forms in Angular

Angular forms are a way to accept user input in an Angular application. They receive input events from users through the template view, validate the input given by the user, create a form model and data model to update, and also, provide a way to track changes. It is important to note that the Angular forms are structured using the [HTML form tag](#).

An Angular application that contains a form, keeps the view in sync with the component model and the component model in sync with the view. When users type or update values through the view, the new values are reflected in the data model. Similarly, if the program logic updates values in the data model, those values are also reflected in the view.

Features and Uses of Angular Forms

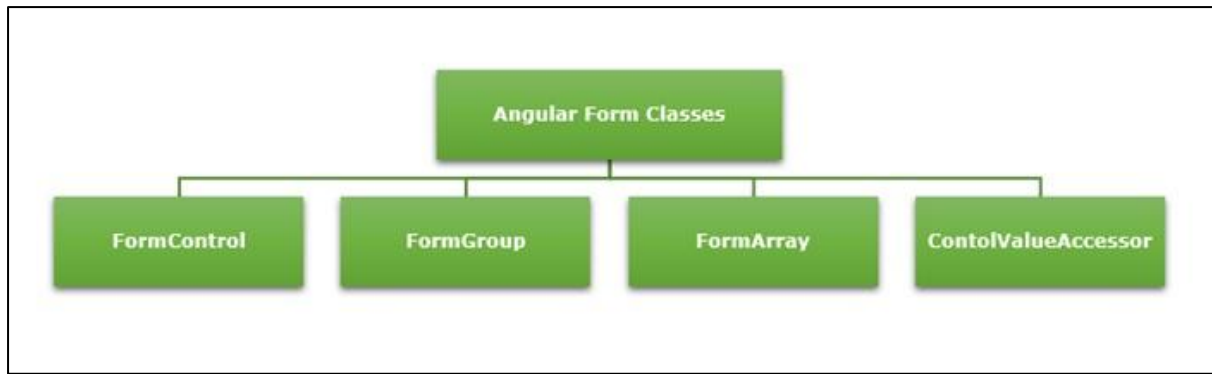
Following are the features and uses of Angular Forms –

- Angular forms support two-way data binding. It is a type of data binding that allows two-way interaction between component and view.
- You can find various built-in validators in Angular forms that help to check whether the user has entered valid data or not before submitting the form.
- Angular forms also allow developers to add, remove, and modify form controls dynamically as per the needs of the user interface.
- Developers can organize form controls into logical groups using **FormGroup**. It helps to manage and track the state of different sections of the form.

Angular Form Classes

The Angular forms are built on the following four foundation classes –

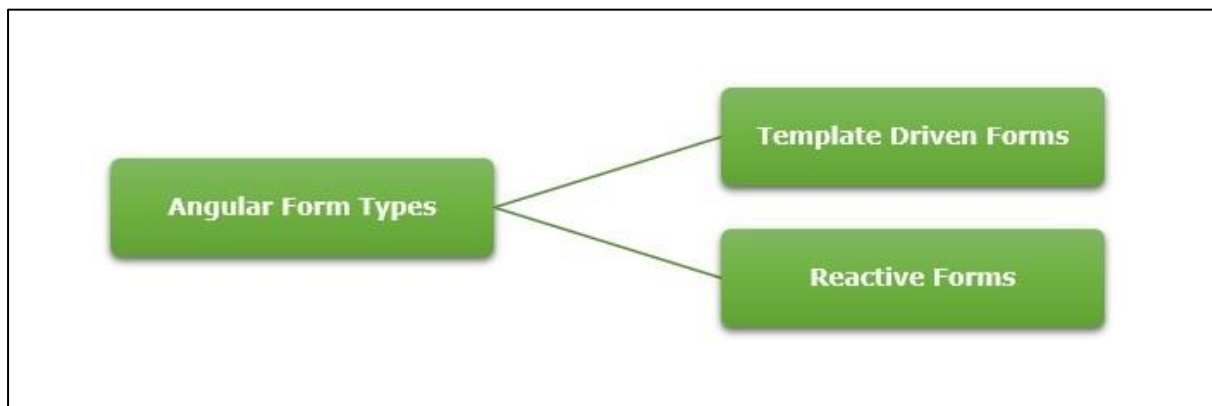
- **FormControl**: Represents a single form input field. It tracks the value, validation status, and user interactions for the individual input control.
- **FormGroup**: This foundation class represents a collection of FormControl instances or nested FormGroup instances. You can group related form controls together using this class. It's used to manage the state of a form section (e.g., user details, address information).
- **FormArray**: It is used to handle an array of form controls, such as a list of checkboxes or input fields.
- **ControlValueAccessor**: It connects Angular FormControl instances to built-in DOM elements.



Types of Form in Angular

Angular supports two types of forms. They are as follows –

- **Template-Driven Forms:** As the name suggests, the template-driven forms are closely associated with [Angular Templates](#). User input validation and behaviours are defined using directives within the template. For data flow, these forms use two-way data binding. You can build any kind of simple form within your Angular application, such as login forms and contact forms. Remember, if you have very basic form requirements and prefer to manage it solely using the template, then template-driven forms may be a suitable choice.
- **Reactive Forms:** The reactive forms follow a model-driven approach. Compared to template-driven forms, they are more robust, scalable and suitable for complex forms. Instead of a template, these forms are controlled from the [Angular Component](#) class. Use this type of form, if forms are a key part of your Angular application.



Difference between Reactive Forms and Template-Driven Forms

The table below shows how Reactive Forms are different from Template-Driven Forms –

Reactive Forms	Template-Driven Forms
They follow a model-driven approach.	They follow a view-driven approach
Forms are created and controlled from the component class.	Forms are created and validations are handled directly in the template.
The flow of data is Synchronous.	Asynchronous data flow.

It is flexible and scalable for complex forms with dynamic behavior.	More suitable for simple and static forms.
In reactive forms, the form control instances are accessed using FormControl and FormGroup.	Form controls are accessed using template reference variables.
Form validations are applied using the built-in validator functions.	Form validations are applied using the built-in directives.

Form Validation

[Form validation](#) is a process used to check whether the user input is in the correct format or not before submission. The validation process can be used to verify the format of email addresses and phone numbers as they have specific formats. Also, you can verify if the given input meets specific constraints like a minimum or maximum length. The form validation can prevent errors by catching invalid input before it is processed or sent to the server.

The **Validator** class in Angular provides a set of built-in validator functions that validate the form controls. A few example of validators are min, max, required, email, pattern and so on.

Dynamic Form?

A form generated or modified at the run time based on the application state or user interaction is called a **dynamic form**. It makes the forms adaptable to changes in data model. For example, if a user selects a country, the form could dynamically adjust to show additional fields like postal code, state, or country code.

Angular - Dependency Injection

What is Dependency Injection?

Dependency Injection (in short, DI) is a [design pattern](#) in which a class receives its dependencies from an external source instead of creating them itself. This approach helps applications achieve loose coupling and reduce tight coupling between different parts of the application. By injecting dependencies, applications become more flexible and adaptable to changes.

In [Angular](#), dependency injection is used to inject services and other dependencies into components and other classes. Before understanding dependency injection in Angular, let's discuss its features and uses.

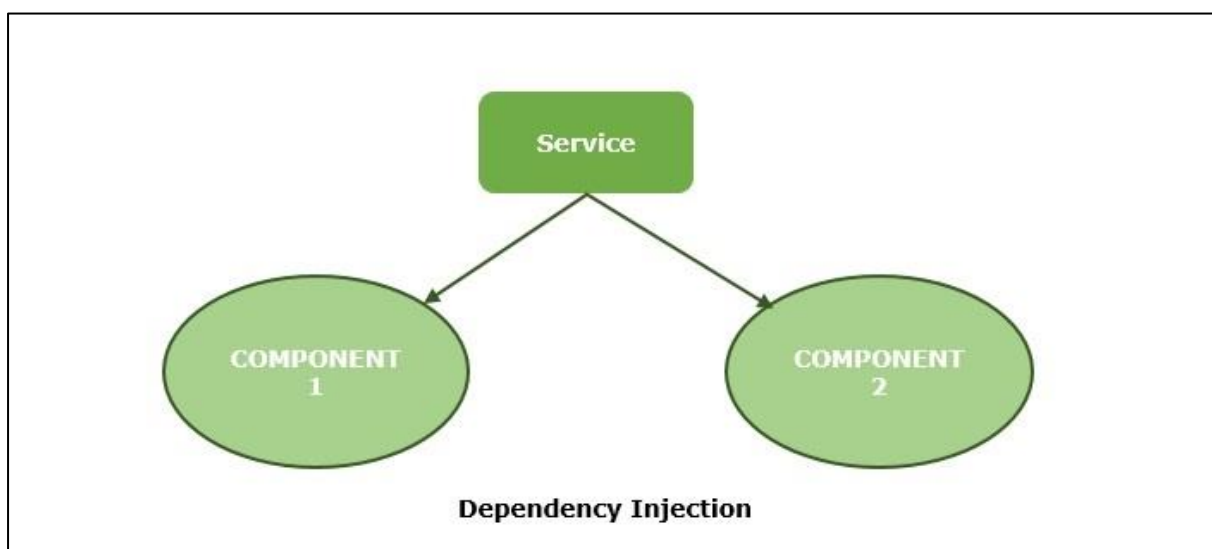
Features and Uses of Dependency Injection

Dependency injection is used because of the following reasons –

- DI decrease the tight coupling between classes and their dependencies.
- You create dependency once and reuse it according to the need in different parts of the application.
- DI helps in unit testing as configuration details are saved within configuration files and system can be reconfigured without recompiling.
- Using DI, lazy loading is implemented. Lazy loading is also a design pattern. In this design pattern, dependencies are created when they are required.
- With DI, the code become more testable, maintainable and reusable because the client classes do not need to know about the implementation process of dependencies.

Dependency Injection in Angular

In Angular, dependency injection is the feature of injecting services and values (like, strings and functions) into classes having [Angular decorators](#). Angular uses **providers** to define how dependencies should be created, and **injectors** to manage the lifecycle of these dependencies. Services, components, directives, and pipes can all use DI to receive their dependencies.



How Dependency Injection Works in Angular?

In Angular, when you declare a dependency in the Component's class constructor, Angular looks up the service in its dependency graph and injects it into the component automatically. Let's see how dependency injection works step by step –

- **Service Creation:** First, define a service using the **@Injectable** decorator. Inside this decorator use **providedIn** and give it a value **root** which will provide the service at the root level (means whole application).
- **Injecting the Service:** In the component's constructor, declare the service as a parameter. Angular will inject the required instance of the service automatically.
- **Usage:** Once injected, you can use the service in your component or other classes.
- **Scope of the Service:** By default, services provided in the root are singleton, which means only one instance is shared across the application. However, services can also be provided at different levels if you want different instances for different parts of the application.

Implementing Dependency Injection in Angular

In this section, we will see how to implement dependency injection in an Angular application with the help of an example.

Example

In the following example, we create an array of color names and then print it. The name of the colors will be printed using dependency injection.

Step 1: Create a service which is a class that holds the code to fetch data. This data will be shared across components within the application. Use the below command to create a service –

ng g s dataprovider

You can give any name of your choice.

Step 2: Open the service file and add the code given below –

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})

export class DataproviderService {

  constructor() { }

  getData() {

    return ['Red', 'Orange', 'Violet', 'Yellow', 'Saffron'];

  }

}
```

Step 3: Once you have created a service, inject it into any component that needs it by including it in the constructor as shown below. We are injecting it inside app component.

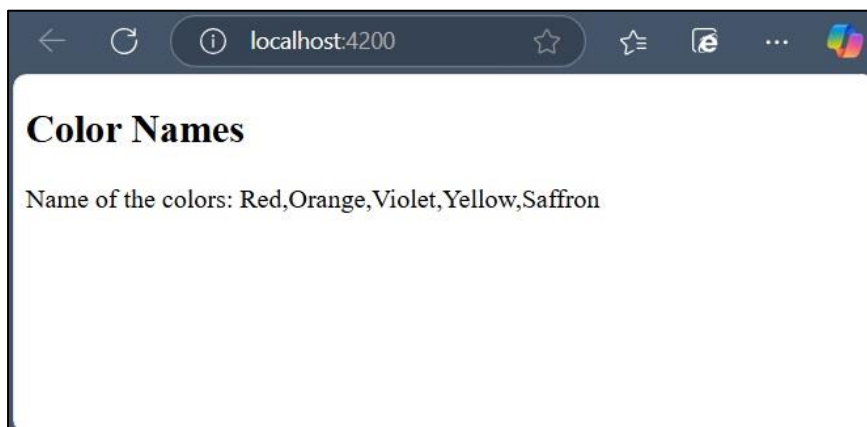
```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { DataproviderService } from './dataproducer.service';
```

```
@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  color: string[];
  constructor(private colorService: DataproviderService) {
    this.color = this.colorService.getData();
  }
}
```

Step 4: Now, add the following code inside template file.

```
<h2>Color Names</h2>
<p>Name of the colors: {{color}} </p>
<router-outlet />
```

Step 5: Run the application using **ng serve** command to get the output.



Angular - Routing

What is Routing?

In web development, **routing** is a technique for navigating between different views or pages in a an application. It allows the application to determine which page to display based on the URL requested by the user.

Routing in Angular

In Angular, **routing** is used to develop a single-page applications ([SPAs](#)). Although, an SPA does not support multipage navigation, it allows navigation from one view ([component](#)) to another without reloading the entire page. You can define the routes for different components to navigate when the URL changes.

Below is the snippet of code of how routes are defined for different **components**:

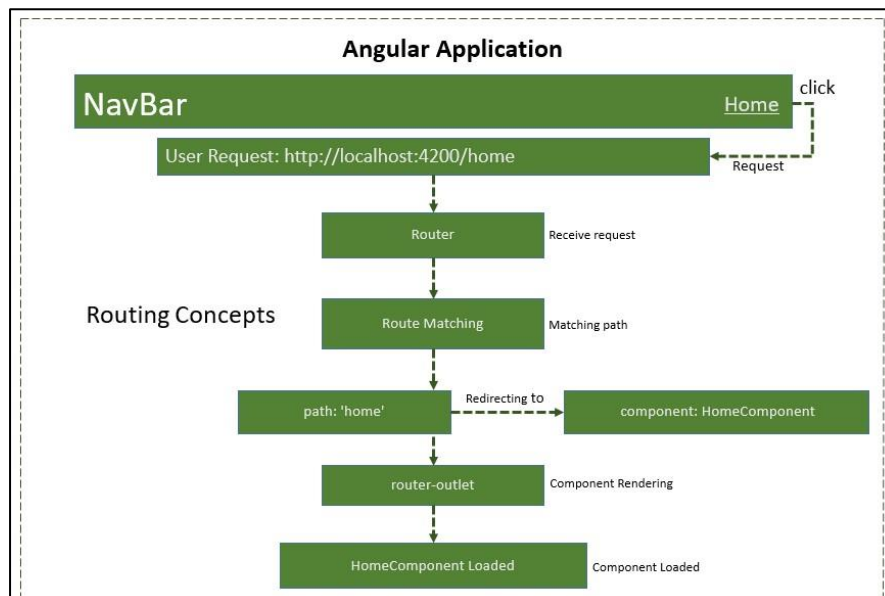
```
routes = [  
  {path: 'home', component: HomeComponent},  
  {path: 'about', component: AboutComponent}  
]
```

Here,

- **routes:** An array contains all the routes for different components.
- **path:** A path, which you can access on the URL to load the **target component**.
- **component:** A target component will be loaded when the **path** is active on the URL.

When you create a new application, an `app.routes.ts` file will automatically generated, where you can define these routes for navigation.

The diagram below will give you a clear understanding of how routing works in Angular when the user clicks on a link to navigate to a different component:



Types of routes in Angular

- **Static routes:** These are the simple routes that map a specific URL path to a component.
{path: 'path-name', component: component-name}
- **Dynamic routes:** A routes allow you to load different data or entities (such as products in an e-commerce application or videos in a streaming platform) based on the values in the URL. These routes are use **parameters** in the URL to dynamically load different entities of a components based on current paramter pased in URL:
{path: 'path-name/:parameter', component: component-name}
- **Wildcard routes:** These routes matches any URL path, which does not matched by other routes.
{path: '**', component: component-name}
- **Nested routes:** A routes defined within other routes allow for hierarchical navigation structures. Sometimes, we want a page (child route) to load only when its parent route is loaded first. For example, in the route **/settings/profile**, the "settings" are the "parent route", and the "profile" is the "child route". The profile page will only be accessible and displayed after the settings page has been loaded:

```
{
  path: 'parent-path', loadChildren:
  [
    {path: 'child-path1', component: component-name},
    {path: 'child-path2', component: component-name}
  ]
}
```

Configure Routing in Angular

To configure a static (basic) routing in our Angular application, the [Angular CLI](#) (command line interface) provides comprehensive support for setting up routing both during the application creation process and while working on an existing application.

Let's create a new application with routing enabled using the command below:

ng new routing-app --routing

Even if you do not provide the **--routing** flag, the above command will generate a new file named **app.routes.ts** by default:

```
import { Routes } from '@angular/router';

export const routes: Routes = [];
```

Here,

- Imports **Routes** from '@angular/router' package.
- **Routes** provides functionality to configure and execute routing in the application.
- **Routes** is the type used to setup the navigation rules.

- **Routes** is the local variable (of type Routes) used to configure the actual navigation rules of the application.

Defining Routes

To define routes in your Angular application that redirect to different components when the URL changes, follow the steps below:

Step 1: Add your routes to your application

Open the **app.routes.ts** file and place the below code:

```
import { Routes } from '@angular/router';

import { HomeComponent } from './home/home.component';

import { AboutComponent } from './about/about.component';
```

```
export const routes: Routes = [

  {path: 'home', component: HomeComponent},

  {path: 'about', component: AboutComponent}

];
```

Here,

- **home** and **about** are paths that will be used in the URL to navigate to the "Home" and "About" components.
- **HomeComponent** and **AboutComponent** are the target component or destination.
- When the URL changes **"/home"** and **"/about"**, the matched component will be called and rendered.

Adding routes in Configuration file

Make sure that the generated routes are added in the Configuration file (**app.config.ts**) otherwise, you might get an error:

```
import { ApplicationConfig } from '@angular/core';

import { provideRouter } from '@angular/router';

import { routes } from './app.routes';

import { provideClientHydration } from '@angular/platform-browser';

export const appConfig: ApplicationConfig = {

  providers: [provideRouter(routes), provideClientHydration()]

};
```

Adding 'router-outlet' in root Component

To work with routing, you need to include the `<router-outlet>` [directive](#) in your root component (recommended to add in `app.component.html`), where you want to display the loaded components.

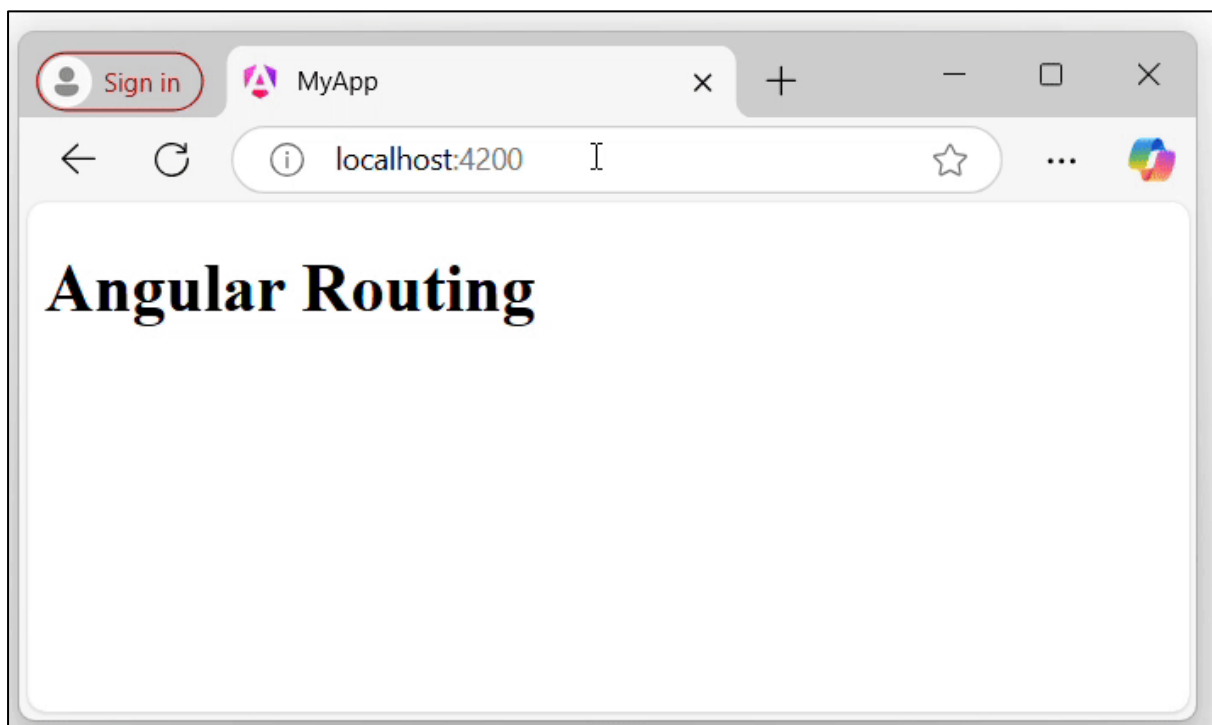
Including the "router-outlet" directive in `app.component.html`:

```
<h1>Angular Routing</h1>

<router-outlet></router-outlet>
```

Testing the Routing

To test the routing, run the application and try to navigate to `/home` and `/about` by changing the URL:



Using RouterLink for Navigation

Rather than changing the URL manually, you can use the RouterLink directive.

The **RouterLink** directive is used to set a path for navigating between views or components in an Angular application.

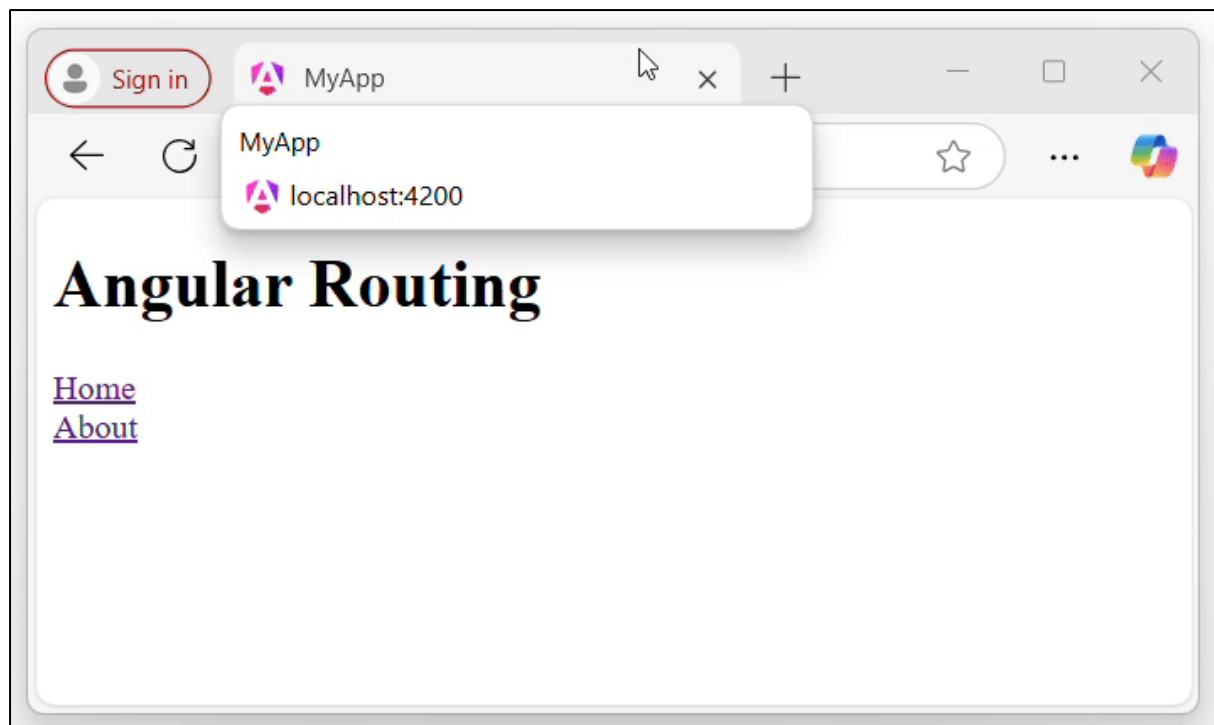
Binding a specified path in `app.routes.ts` with an anchor element (`<a>`) to handle navigation when links are clicked:

```
<h1>Angular Routing</h1>

<a routerLink="/home">Home</a><br>
<a routerLink="/about">About</a>

<router-outlet></router-outlet>
```

The output will look like:



Here,

- **routerLink** set the route to be called using the path.

Note: Make sure that all the necessary modules and dependencies such as **RouterOutlet**, **RouterLink**, and **CommonModule** should be added to your root component.

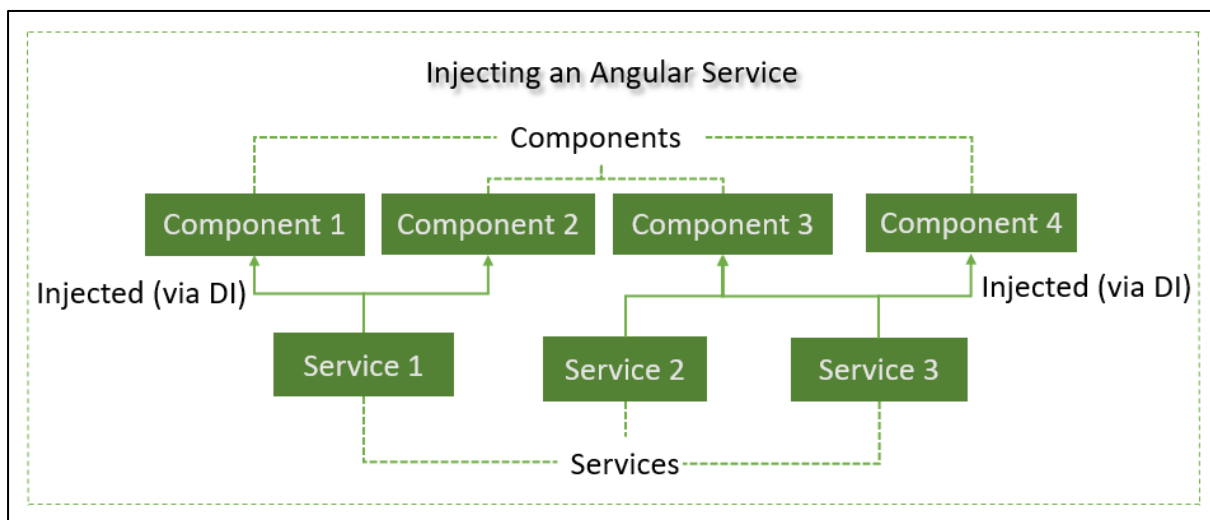
Angular - Services

What are Services in Angular?

In Angular, **Services** are **singleton** (having a single instance) classes that provide specific functionality or handle common logic in an Angular application, which can be used throughout the entire application.

In a single Angular application, one or more services can be used. Similarly, an Angular component may depend on one or more services.

Angular **services** may depend on other services to work properly. Dependency resolution is one of the complex and time-consuming activities in developing any application. To reduce this complexity, Angular provides **Dependency Injection** (a design pattern) as one of its core concepts.



Key Elements of a Service Class

As we define the component in angular, services also include the following:

- A TypeScript **decorator** that declares the class as an Angular service via **@Injectable** (i.e., is a decorator) and allows you to define what part of the application can access the service via the providedIn property (which is by default 'root') to allow the service to access throughout the entire application.
- A TypeScript **class (or say service)** that defines the desired code that will be accessible when the service is **injected** into another components, directive, etc.

Here is a basic example of the printName service:

```
import {Injectable} from '@angular/core';

@Injectable({providedIn: 'root'})

export class printName{

  display(name: string) {

    return name;

  }

}
```

```
}
```

Creating Angular Service

An Angular service is a plain **TypeScript** class with one or more methods (functionality) along with the **@Injectable** decorator. This decorator allows the normal TypeScript class to be used as a service in an Angular application.

To create a service in an Angular application, run the following command in your IDE's terminal –

```
ng generate service service-name
```

or

```
ng generate service Services/service-name
```

Here, the first command will create a service class directly in your Angular application without a separate folder, which can be a bit confusing. However, the second command will create a 'Services' folder and place the service class inside it.

After executing the second command, the following will be displayed in your IDE's terminal –

```
CREATE src/app/Services/print-name.service.spec.ts (389 bytes)
```

```
CREATE src/app/Services/print-name.service.ts (147 bytes)
```

The **printName** service is as follows –

```
import { Injectable } from '@angular/core';
```

```
@Injectable({  
  providedIn: 'root'
```

```
})
```

```
export class PrintNameService {
```

```
//define it after the service is created..
```

```
  display(name: string){
```

```
    return name;
```

```
  }
```

```
  constructor() { }
```

```
}
```

Here, **@Injectable** decorator converts a plain Typescript class into Angular service.

How to use a Service?

When you want to use a service in a component, you need to –

- Import the service in the component where you want to use it.
- Declare a class field where the service needs to be injected.

- Assign the class field to the instance of the service created by Angular's dependency injection.

Here is what it might look like in the **User** component –

```
import { Component, inject } from '@angular/core';
import { PrintNameService } from '../Services/print-name.service';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-user',
  standalone: true,
  imports: [PrintNameService, CommonModule],
  templateUrl: './user.component.html',
  styleUrls: ['./user.component.css']
})
export class UserComponent {
  private print_name = inject(PrintNameService);
  my_name = this.print_name.display("John");
}
```

In this example, the **printName** service is being used by calling the Angular function inject and passing in the service to it.

How Service Works?

Here are the key concepts about how services work in Angular –

Service Registration

- When you define a service with **@Injectable({ providedIn: 'root' })**, Angular registers it with the root injector.
- This means the service is available for injection throughout the entire application.

Service Injection

- When a component (or another service) requires the service, then the Angular **DI (dependency injection)** mechanism creates an instance of the service and injects it into the requesting component/service.

Singleton Pattern

- Services provided at the root level (**providedIn: 'root'**) are singletons by default. This means there is only one instance of the service throughout the entire application, which ensures a consistent state and shared data.

Register Angular service

To use **Dependency Injection**, every service needs to be registered into the system. Angular provides multiple option to register a service. They are as follows –

- **ModuleInjector @ root level**
- **ModuleInjector @ platform level**
- **ElementInjector using providers meta data**
- **ElementInjector using viewProviders meta data**
- **NullInjector**

ModuleInjector @ root

ModuleInjector enforces the service to be used only inside a specific module. **providedIn** meta data available in **@Injectable** has to be used to specify the module in which the service can be used.

The value should refer to the one of the registered Angular Module (decorated with **@NgModule**). **root** is a special option which refers the root module of the application. The sample code is as follows –

```
import { Injectable } from '@angular/core'; @Injectable({
  providedIn: 'root',
})
export class DebugService {
  constructor() {}
}
```

ModuleInjector @ platform

Platform Injector is one level higher than **ModuleInject** and it is only in advanced and rare situation. Every Angular application starts by executing **PreformBrowserDynamic().bootstrap** method (see **main.js**), which is responsible for bootstrapping root module of Angular application.

PreformBrowserDynamic() method creates an injector configured by **PlatformModule**. We can configure platform level services using **platformBrowser()** method provided by **PlatformModule**.

NullInjector

NullInjector is one level higher than platform level **ModuleInjector** and is in the top level of the hierarchy. We could not able to register any service in the **NullInjector**. It resolves when the required service is not found anywhere in the hierarchy and simply throws an error.

ElementInjector using providers

ElementInjector enforces the service to be used only inside some particular components. **providers** and **ViewProviders** meta data available in **@Component** decorator is used to specify the list of services to be visible for the particular component. The sample code to use providers is as follows –

ExpenseEntryListComponent


```
// import statement
import { DebugService } from '../debug.service';

// component decorator
@Component({
  selector: 'app-expense-entry-list',
  templateUrl: './expense-entry-list.component.html',
  styleUrls: ['./expense-entry-list.component.css'],
  providers: [DebugService] })
```

Here, **DebugService** will be available only inside the **ExpenseEntryListComponent** and its view. To make DebugService in other component, simply use **providers** decorator in necessary component.

ElementInjector using viewProviders

viewProviders is similar to **provider** except it does not allow the service to be used inside the component's content created using **ng-content** directive.

ExpenseEntryListComponent

```
// import statement
import { DebugService } from '../debug.service';

// component decorator
@Component({
  selector: 'app-expense-entry-list',
  templateUrl: './expense-entry-list.component.html',
  styleUrls: ['./expense-entry-list.component.css'],
  viewProviders: [DebugService]
})
```

Parent component can use a child component either through its view or content. Example of a parent component with child and content view is mentioned below –

Parent component view / template

```
<div>

  child template in view

  <child></child>

</div>

<ng-content></ng-content>
```

child component view / template

```
<div>
```

```
  child template in view
```

```
</div>
```

Parent component usage in a template (another component)

```
<parent>
```

```
  <!-- child template in content -->
```

```
  <child></child>
```

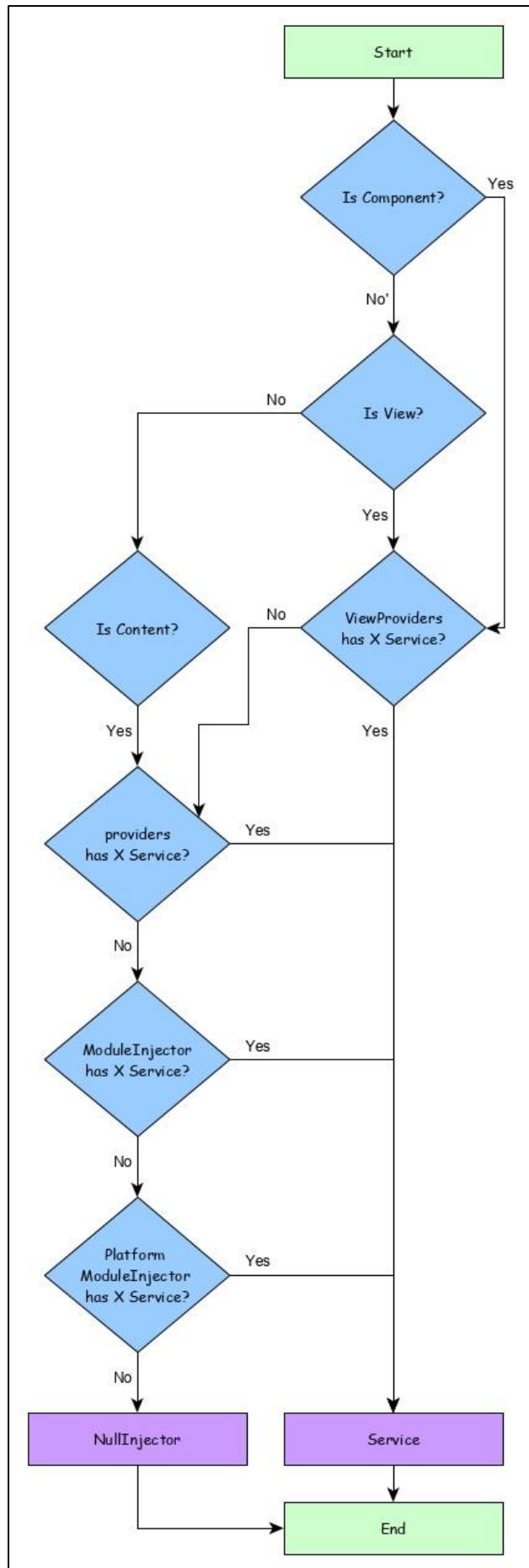
```
</parent>
```

Here,

- **child** component is used in two place. One inside the parent's view. Another inside parent content.
- Services will be available in child component, which is placed inside parent's view.
- Services will not be available in child component, which is placed inside parent's content.

Resolve Angular service

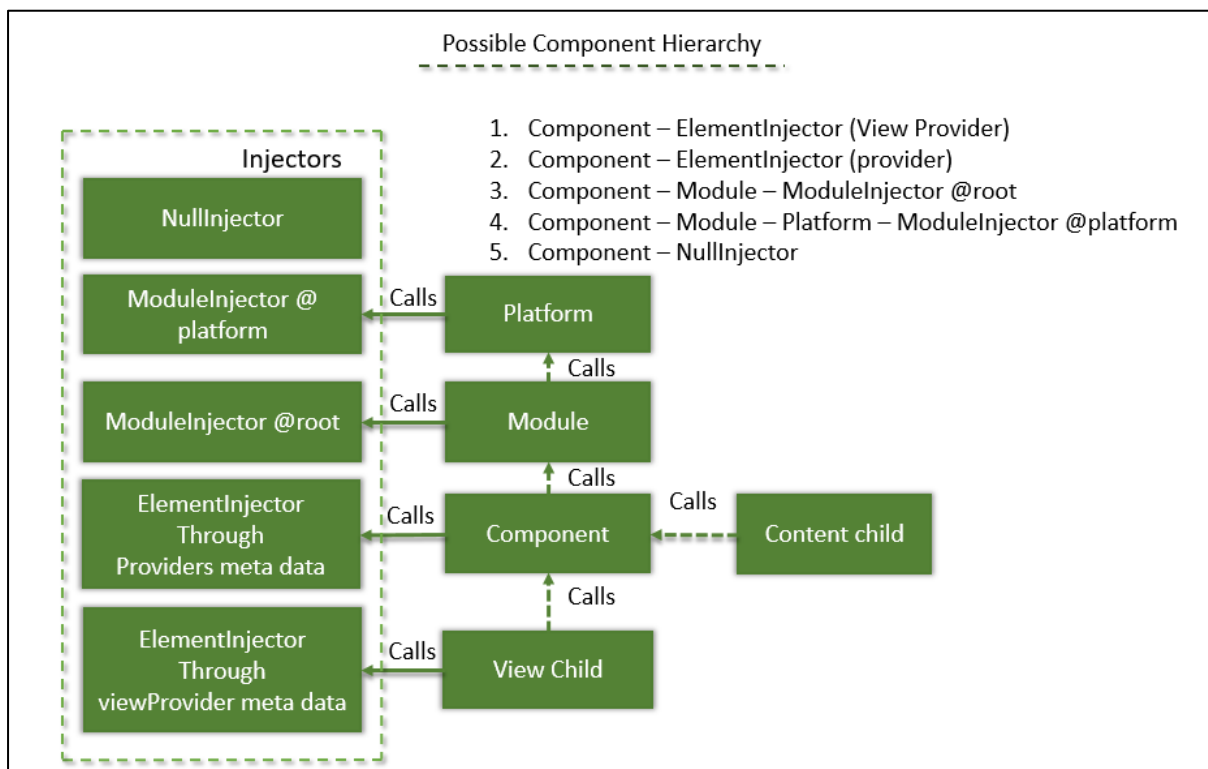
Let us see how a component can resolve a service using the below flow diagram.



Here,

- First, component tries to find the service registered using **viewProviders** meta data.
- If not found, component tries to find the service registered using **providers** meta data.
- If not found, Component tries to find the service registered using **ModuleInjector**
- If not found, component tries to find the service registered using **PlatformInjector**
- If not found, component tries to find the service registered using **NullInjector**, which always throws error.

The hierarchy of the Injector along with work flow of the resolving the service is as follows –



Angular - HTTP Client

HTTP is an application layer protocol on the Internet. It stands for **Hypertext Transfer Protocol** and is the foundation of any data exchange on the web. It transmits hypertext requests and information between a server and a browser (client).

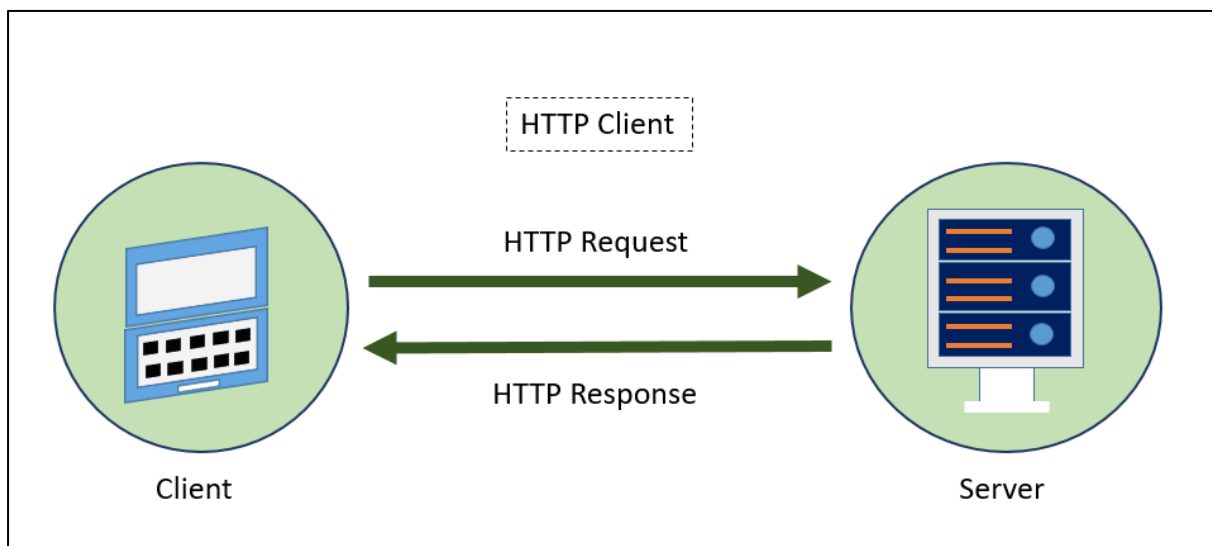
What is HTTP Client?

HTTP Client is a **client-side programming** that provides the HTTP server access to various resources and services. It allows the client (a browser or application) to send HTTP requests and receive responses from the server.

HTTP client programming is an important feature in every modern web application. Nowadays, many applications expose their functionality through **REST APIs**, which work over the HTTP protocol.

To support this, the Angular team offers extensive tools to enable HTTP communication with servers. Angular provides a module called **HttpClientModule** and a service called **HttpClient** to handle HTTP programming.

The following diagram provides a clear understanding of the HTTP Client, and the request and response mechanism –



Let us learn how to use the `HttpClient` service in this chapter. Developers should have a basic understanding of HTTP programming to understand the concepts discussed in this chapter.

Expense REST API

The prerequisite for HTTP programming is a basic understanding of the HTTP protocol and REST API techniques. HTTP programming involves two parts: the server and the client. Angular provides support for creating client-side applications, while Express (a popular web framework) provides support for creating server-side applications.

Let us create an **Expense Rest API** using the express framework and then access it from our `ExpenseManager` application using the **Angular HttpClient** service.

Open a command prompt and create a new folder, **express-rest-api**.

```
cd /go/to/workspace
```

```
mkdir express-rest-api
```

```
cd expense-rest-api
```

Initialize a new node application using the below command –

```
npm init
```

After running the **npm init command** will ask some basic questions like project name (express-rest-API), entry point (server.js), etc., as mentioned below –

This utility will walk you through creating a package.json file.

It only covers the most common items, and tries to guess sensible defaults.

See 'npm help json' for definitive documentation on these fields and exactly what they do.

Use 'npm install <pkg>' afterwards to install a package and save it as a dependency in the package.json file.

Press ^C at any time to quit.

```
package name: (expense-rest-api)
```

```
version: (1.0.0)
```

```
description: Rest api for Expense Application
```

```
entry point: (index.js) server.js
```

```
test command:
```

```
git repository:
```

```
keywords:
```

```
author:
```

```
license: (ISC)
```

About to write to \path\to\workspace\expense-rest-api\package.json: {

```
  "name": "expense-rest-api",
```

```
  "version": "1.0.0",
```

```
  "description": "Rest api for Expense Application",
```

```
  "main": "server.js",
```

```
  "scripts": {
```

```
    "test": "echo \"Error: no test specified\" && exit 1"
```

```
  },
```

```
  "author": "",
```

```
"license": "ISC"
}
```

Is this OK? (yes) yes

Install **express**, **SQLite**, and **cors** modules using the below command –

```
npm install express sqlite3 cors
```

Create a new file with the name **sqlitedb.js** and place the below code –

```
var sqlite3 = require('sqlite3').verbose()
```

```
const DBSOURCE = "expensedb.sqlite"
```

```
let db = new sqlite3.Database(DBSOURCE, (err) => {
```

```
  if (err) {
```

```
    console.error(err.message)
```

```
    throw err
```

```
  }else{
```

```
    console.log('Connected to the SQLite database.')
```

```
    db.run(`CREATE TABLE expense (
```

```
      id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
      item text,
```

```
      amount real,
```

```
      category text,
```

```
      location text,
```

```
      spendOn text,
```

```
      createdOn text
```

```
    `),
```

```
    (err) => {
```

```
      if (err) {
```

```
        console.log(err);
```

```
      }else{
```

```
        var insert = 'INSERT INTO expense (item, amount, category, location, spendOn, createdOn)
VALUES (?, ?, ?, ?, ?, ?)'
```

```

        db.run(insert, ['Pizza', 10, 'Food', 'KFC', '2020-05-26 10:10', '2020-05-26 10:10'])
        db.run(insert, ['Pizza', 9, 'Food', 'Mcdonald', '2020-05-28 11:10', '2020-05-28 11:10'])
        db.run(insert, ['Pizza', 12, 'Food', 'Mcdonald', '2020-05-29 09:22', '2020-05-29 09:22'])
        db.run(insert, ['Pizza', 15, 'Food', 'KFC', '2020-06-06 16:18', '2020-06-06 16:18'])
        db.run(insert, ['Pizza', 14, 'Food', 'Mcdonald', '2020-06-01 18:14', '2020-05-01 18:14'])
    }
}
);
}
});

```

```
module.exports = db
```

Here, we are creating a new SQLite database and loading some sample data.

Now, open **server.js** and place the below code (if you are not able to see the file in your application create it manually within the root directory) –

```

var express = require("express")
var cors = require('cors')
var db = require("./sqllitedb.js")

```

```

var app = express()
app.use(cors());

```

```

var bodyParser = require("body-parser");
app.use(bodyParser.urlencoded({ extended: false }));
app.use(bodyParser.json());

```

```

var HTTP_PORT = 8000
app.listen(HTTP_PORT, () => {
    console.log("Server running on port %PORT%".replace("%PORT%", HTTP_PORT))
});

```



```
app.get("/", (req, res, next) => {  
  res.json({"message": "Ok"})  
});
```

```
app.get("/api/expense", (req, res, next) => {  
  var sql = "select * from expense"  
  var params = []  
  db.all(sql, params, (err, rows) => {  
    if (err) {  
      res.status(400).json({"error": err.message});  
      return;  
    }  
    res.json(rows)  
  });  
});
```

```
app.get("/api/expense/:id", (req, res, next) => {  
  var sql = "select * from expense where id = ?"  
  var params = [req.params.id]  
  db.get(sql, params, (err, row) => {  
    if (err) {  
      res.status(400).json({"error": err.message});  
      return;  
    }  
    res.json(row)  
  });  
});
```

```
app.post("/api/expense/", (req, res, next) => {  
  var errors=[]
```

```

if (!req.body.item){
  errors.push("No item specified");
}
var data = {
  item : req.body.item,
  amount: req.body.amount,
  category: req.body.category,
  location : req.body.location,
  spendOn: req.body.spendOn,
  createdOn: req.body.createdOn,
}

var sql = 'INSERT INTO expense (item, amount, category, location, spendOn, createdOn) VALUES
(?,?,?,?,?,?)'

var params =[data.item, data.amount, data.category, data.location, data.spendOn, data.createdOn]
db.run(sql, params, function (err, result) {
  if (err){
    res.status(400).json({"error": err.message})
    return;
  }
  data.id = this.lastID;
  res.json(data);
});
})

```

```

app.put("/api/expense/:id", (req, res, next) => {
  var data = {
    item : req.body.item,
    amount: req.body.amount,
    category: req.body.category,
    location : req.body.location,
    spendOn: req.body.spendOn

```

```

    }

    db.run(
      `UPDATE expense SET
        item = ?,

        amount = ?,
        category = ?,
        location = ?,

        spendOn = ?
      WHERE id = ?`,
      [data.item, data.amount, data.category, data.location, data.spendOn, req.params.id],
      function (err, result) {
        if (err){
          console.log(err);
          res.status(400).json({"error": res.message})
          return;
        }
        res.json(data)
      });
  })

```

```

app.delete("/api/expense/:id", (req, res, next) => {
  db.run(
    'DELETE FROM expense WHERE id = ?',
    req.params.id,
    function (err, result) {
      if (err){
        res.status(400).json({"error": res.message})
        return;
      }
    }
  )
})

```

```
    res.json({"message":"deleted", changes: this.changes})
  });
}
```

```
app.use(function(req, res){
  res.status(404);
});
```

Here, we create a basic CRUD rest API to select, insert, update, and delete expense entries.

Run the application using the below command –

npm run start

Open a browser, enter **http://localhost:8000/** and press enter. You will see below response –

```
{
  "message": "Ok"
}
```

The above message confirms that our application is working fine.

Change the URL to **http://localhost:8000/api/expense**, and you will see all the expense entries in JSON format.

```
[
  {
    "id": 1,

    "item": "Pizza",
    "amount": 10,
    "category": "Food",
    "location": "KFC",
    "spendOn": "2020-05-26 10:10",
    "createdOn": "2020-05-26 10:10"
  },
  {
    "id": 2,
    "item": "Pizza",
```

```
"amount": 14,
"category": "Food",
"location": "Mcdonald",
"spendOn": "2020-06-01 18:14",
"createdOn": "2020-05-01 18:14"
},
{
  "id": 3,
  "item": "Pizza",
  "amount": 15,
  "category": "Food",
  "location": "KFC",
  "spendOn": "2020-06-06 16:18",
  "createdOn": "2020-06-06 16:18"
},
{
  "id": 4,
  "item": "Pizza",
  "amount": 9,
  "category": "Food",
  "location": "Mcdonald",
  "spendOn": "2020-05-28 11:10",
  "createdOn": "2020-05-28 11:10"
},
{
  "id": 5,
  "item": "Pizza",
  "amount": 12,
  "category": "Food",
  "location": "Mcdonald",
  "spendOn": "2020-05-29 09:22",
```

```
    "createdOn": "2020-05-29 09:22"
  }
]
```

Finally, we created a simple **CURD REST API** for expense entry, and we can access the REST API from our Angular application to learn the HttpClient module.

Configure Http client

Let's learn how to configure the **HttpClient** service. The HttpClient service is available inside the **HttpClientModule** module, which is available inside the **@angular/common/http** package.

To register **HttpClientModule** module. You need to import the HttpClientModule in **AppComponent**:

```
import { HttpClientModule } from '@angular/common/http';
```

Include HttpClientModule in imports meta data of AppComponent.

```
import { Component } from '@angular/core';
```

```
import { CommonModule } from '@angular/common';
```

```
import { RouterOutlet } from '@angular/router';
```

```
import { HttpClientModule } from '@angular/common/http';
```

```
@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, RouterOutlet, HttpClientModule],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'myApp';
}
```

Create expense service

Let us create a new servicenamed **ExpenseEntryService**in your **ExpenseManager**application to interact with**Expense REST API**. ExpenseEntryService will get the latest expense entries, insert new expense entries, modify existing expense entries, and delete the unwanted expense entries.

Open the command prompt and go to the project root folder:

```
cd /go/to/expense-manager
```

Start the application by running the below command:

ng serve

Run the below command to generate an Angular service with the name **ExpenseService**:

ng generate service ExpenseEntry

This will create two Typescript files (expense entry service & its testing file) as specified below:

CREATE src/app/expense-entry.service.spec.ts (364 bytes)

CREATE src/app/expense-entry.service.ts (141 bytes)

Now, open the **ExpenseEntryService** (src/app/expense-entry.service.ts) file

import **ExpenseEntry**, **throwError**, and **catchError** from the rxjs library, and

import **HttpClient**, **HttpHeaders**, and **HttpResponse** from **@angular/common/http** package:

```
import { Injectable } from '@angular/core';
```

```
import { ExpenseEntry } from './expense-entry';
```

```
import { throwError } from 'rxjs';
```

```
import { catchError } from 'rxjs/operators';
```

```
import { HttpClient, HttpHeaders, HttpResponse } from '@angular/common/http';
```

Inject the **HttpClient** service into our service:

```
constructor(private httpClient : HttpClient) { }
```

Create a variable, **expenseRestUrl** to specify the **Expense Rest API** endpoints:

```
private expenseRestUrl = 'http://localhost:8000/api/expense';
```

Create a variable, **httpOptions** to set the HTTP Header option. This will be used during the HTTP Rest API call by the Angular **HttpClient** service:

```
private httpOptions = {
```

```
  headers: new HttpHeaders( { 'Content-Type': 'application/json' })
```

```
};
```

The complete code is as follows:

```
import { Injectable } from '@angular/core';
```

```
import { ExpenseEntry } from './expense-entry';
```

```
import { Observable, throwError } from 'rxjs';
```

```
import { catchError, retry } from 'rxjs/operators';
```

```
import { HttpClient, HttpHeaders, HttpResponse } from '@angular/common/http';
```

```
@Injectable({
```

```

    providedIn: 'root'
  })
  export class ExpenseEntryService {
    private expenseRestUrl = 'api/expense';
    private httpOptions = {
      headers: new HttpHeaders({'Content-Type': 'application/json'})
    };

    constructor(private httpClient : HttpClient) { }
  }

```

HTTP GET

The HttpClient provides a **get()** method to fetch data from a web page. The main argument is the target web URL. Another optional argument is the option object with the below format –

```

{
  headers?: HttpHeaders | {[header: string]: string | string[]},
  observe?: 'body' | 'events' | 'response',

  params?: HttpParams | {[param: string]: string | string[]},
  reportProgress?: boolean,
  responseType?: 'arraybuffer' | 'blob' | 'json' | 'text',
  withCredentials?: boolean,
}

```

Here,

- **headers:** HTTP Headers of the request, either as string, an array of string, or an array of HttpHeaders.
- **observe:** Process the response and return the specific content of the response. Possible values are body, response, and events. The default option of the observer is the body.
- **params:** HTTP parameters of the request, either as string, array of string, or array of **HttpParams**.
- **reportProgress:** Whether to report the progress of the process or not (true or false).
- **responseType:** Refers to format of the response. Possible values are **arraybuffer**, **blob**, **JSON**, and **text**.
- **withCredentials:** Whether the request has credentials or not (true or false).

Note: All options are optional, you can choose to use them or not.

The **get()** method returns the response of the request as **Observable**. The returned Observable emits the data when the response is received from the server.

The sample code to use **get()** method is as follows:

```
httpClient.get(url, options) .subscribe( (data) => console.log(data) );
```

Typed Response

The **get()** method has an option to return observables, which emits a typed response as well. The sample code to get a typed response (ExpenseEntry) is as follows:

```
httpClient.get<T>(url, options) .subscribe( (data: T) => console.log(data) );
```

Handling errors

Error handling is one of the important aspects of HTTP programming. Encountering errors is one of the common scenarios in HTTP programming.

Errors in HTTP Programming can be categorized into two groups –

- Client-side issues can occur due to network failure, misconfiguration, etc. If a client-side error happens, then the **get()** method throws an **ErrorEvent** object.
- Server-side issues can occur due to wrong URL, server unavailability, server programming errors, etc.

Let us write a simple error handling for our **ExpenseEntryService** service.

```
private errorHandler (error: HttpResponse) {  
  if (error.error instanceof ErrorEvent) {  
    console.error("A client side error occurs. The error message is "  
      + error.message);  
  } else {  
    console.error(  
      "An error happened in server. The HTTP status code is "  
        + error.status + " and the error returned is " + error.message);  
  }  
  return throwError("Error occurred. Please try again");  
}
```

The error function can be called in **get()** as specified below –

```
httpClient.get(url, options)  
  .pipe(catchError(this.errorHandler))  
  .subscribe( (data) => console.log(data) )
```

Handle failed request

As we mentioned earlier, errors can happen, and one way is to handle them by implementing the error handling. Another option is to try for a certain number of times. If the request fails due to a network issue or the HTTP server is temporarily offline, the next request may succeed.

We can use the **rxjslibrary** "retry" operator in this scenario as specified below :

```
httpClient.get(url, options)

.pipe(
  retry(5),
  catchError(this.httpErrorHandler))
.subscribe( (data) => console.log(data) )
```

Fetch expense entries

Let us do the actual coding to fetch the expenses from **Expense Rest API** in our ExpenseManager application.

Open the command prompt and go to the project root folder:

```
cd /go/to/expense-manager
```

Start the application by running the following command:

```
ng serve
```

Now, add the **getExpenseEntries()** and **httpErrorHandler()** methods in **ExpenseEntryService** (src/app/expense-entry.service.ts) service as follows:

```
getExpenseEntries() : Observable<ExpenseEntry[]> {
  return this.httpClient.get<ExpenseEntry[]>(this.expenseRestUrl, this.httpOptions)
    .pipe(retry(3),catchError(this.httpErrorHandler));
}
```

```
getExpenseEntry(id: number) : Observable<ExpenseEntry> {
  return this.httpClient.get<ExpenseEntry>(this.expenseRestUrl + "/" + id, this.httpOptions)
    .pipe(
      retry(3),
      catchError(this.httpErrorHandler)
    );
}
```

```

private errorHandler (error: HttpResponse) {
  if (error.error instanceof ErrorEvent) {
    console.error("A client side error occurs. The error message is " + error.message);
  } else {
    console.error(
      "An error happened in server. The HTTP status code is " +
        error.status + " and the error returned is " + error.message);
    }
  }

  return throwError("Error occurred. Pleas try again");
}

```

Here,

- **getExpenseEntries()** calls the **get()** method using expense end point and also configures the error handler. Also, it configures **httpClient** to try for maximum of 3 times in case of failure. Finally, it returns the response from server as typed **(ExpenseEntry[])** Observable object.
- **getExpenseEntry** is similar to getExpenseEntries() except it passes the id of the ExpenseEntry object and gets ExpenseEntry Observable object.

The complete coding of **ExpenseEntryService** is as follows:

```

import { Injectable } from '@angular/core';
import { ExpenseEntry } from './expense-entry';
import { Observable, throwError } from 'rxjs';
import { catchError, retry } from 'rxjs/operators';
import { HttpClient, HttpHeaders, HttpResponse } from '@angular/common/http';
@Injectable({
  providedIn: 'root'
})
export class ExpenseEntryService {
  private expenseRestUrl = 'http://localhost:8000/api/expense';
  private httpOptions = {
    headers: new HttpHeaders( { 'Content-Type': 'application/json' })
  };

```

```

constructor(private httpClient : HttpClient) { }

getExpenseEntries() : Observable<ExpenseEntry[]> {
    return this.httpClient.get<ExpenseEntry[]>(this.expenseRestUrl, this.httpOptions)
        .pipe(
            retry(3),
            catchError(this.httpErrorHandler)
        );
}

getExpenseEntry(id: number) : Observable<ExpenseEntry> {
    return this.httpClient.get<ExpenseEntry> (this.expenseRestUrl + "/" + id, this.httpOptions)
        .pipe(
            retry(3),
            catchError(this.httpErrorHandler)
        );
}

```

```

private httpErrorHandler (error: HttpResponse) {
    if (error.error instanceof ErrorEvent) {
        console.error("A client side error occurs. The error message is " + error.message);
    } else {
        console.error(
            "An error happened in server. The HTTP status code is "
                + error.status + " and the error returned is " + error.message);
    }
    return throwError("Error occurred. Pleas try again");
}

```

Open the **ExpenseEntryListComponent** and go to the "src-entry-list-entry-list.component.ts" file and injectExpenseEntryService through the constructor as specified below:

```

constructor(private debugService: DebugService, private restService :
ExpenseEntryService ) { }

```

Change the **getExpenseEntries()** method, and call the "getExpenseEntries()" method from **ExpenseEntryService** instead of returning the mock items:

```
getExpenseItems() {  
    this.restService.getExpenseEntries()  
        .subscribe( data => this.expenseEntries = data );  
}
```

The complete **ExpenseEntryListComponent** codes are as follows –

```
import { Component, OnInit } from '@angular/core';  
import { ExpenseEntry } from '../expense-entry';  
import { DebugService } from '../debug.service';  
import { ExpenseEntryService } from '../expense-entry.service';
```

```
@Component({  
    selector: 'app-expense-entry-list',  
    templateUrl: './expense-entry-list.component.html',  
    styleUrls: ['./expense-entry-list.component.css'],  
    providers: [DebugService]  
})  
export class ExpenseEntryListComponent implements OnInit {  
    title: string;  
    expenseEntries: ExpenseEntry[];  
    constructor(  
        private debugService: DebugService,  
        private restService : ExpenseEntryService  
    ) { }
```

```
    ngOnInit() {  
        this.debugService.info("Expense Entry List component initialized");  
        this.title = "Expense Entry List";  
  
        this.getExpenseItems();  
    }
```

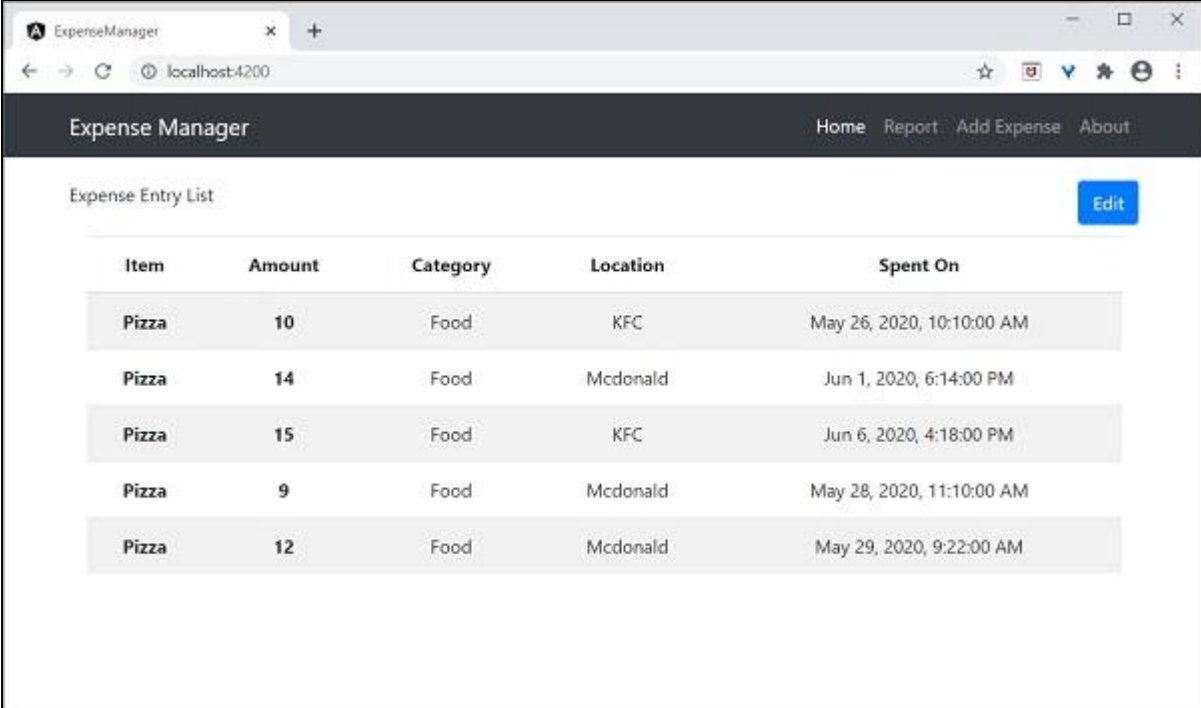
```

    }

    getExpenseItems() {
        this.restService.getExpenseEntries()
            .subscribe( data => this.expenseEntries = data );
    }
}

```

Finally, check the application and you will see the below response:



Item	Amount	Category	Location	Spent On
Pizza	10	Food	KFC	May 26, 2020, 10:10:00 AM
Pizza	14	Food	Mcdonald	Jun 1, 2020, 6:14:00 PM
Pizza	15	Food	KFC	Jun 6, 2020, 4:18:00 PM
Pizza	9	Food	Mcdonald	May 28, 2020, 11:10:00 AM
Pizza	12	Food	Mcdonald	May 29, 2020, 9:22:00 AM

HTTP POST

The **HTTP POST** is similar to **HTTP GET** except that the post request will send the necessary data as posted content along with the request. HTTP POST is used to insert new records into the system. **HttpClient** provides the **post()** method, which is similar to **get()**, except it supports an extra argument to send the data to the server.

Let us add a new method, **addExpenseEntry()** in our **ExpenseEntryService** to add new expense entry as mentioned below:

```

addExpenseEntry(expenseEntry: ExpenseEntry): Observable<ExpenseEntry> {
    return this.httpClient.post<ExpenseEntry>(this.expenseRestUrl, expenseEntry, this.httpOptions)
        .pipe(
            retry(3),
            catchError(this.httpErrorHandler)
        )
}

```

```
);  
}
```

HTTP PUT

The **HTTP PUT** is similar to **HTTP POST** requests. This is used to update existing records in the system. The `HttpClient` provides a **put()** method, which is similar to **post()**.

Update expense entry

Let us add a new method, **updateExpenseEntry()** in our **ExpenseEntryService** to update existing expense entry as mentioned below:

```
updateExpenseEntry(expenseEntry: ExpenseEntry): Observable<ExpenseEntry> {  
    return this.httpClient.put<ExpenseEntry>(this.expenseRestUrl + "/" + expenseEntry.id,  
        expenseEntry, this.httpOptions)  
        .pipe(  
            retry(3),  
            catchError(this.httpErrorHandler)  
        );  
}
```

HTTP DELETE

The **HTTP DELETE** is similar to the **HTTP GET** request. It is used to delete entries in the system. The `HttpClient` provides a **delete()** method, which is similar to **get()**.

Delete expense entry

Let us add a new method, **deleteExpenseEntry()** in our **ExpenseEntryService** to delete existing expense entries as mentioned below:

```
deleteExpenseEntry(expenseEntry: ExpenseEntry | number) :  
Observable<ExpenseEntry> {  
    const id = typeof expenseEntry == 'number' ? expenseEntry :  
        expenseEntry.id  
    const url = `${this.expenseRestUrl}/${id}`;  
    return this.httpClient.delete<ExpenseEntry>(url, this.httpOptions)  
        .pipe(  
            retry(3),  
            catchError(this.httpErrorHandler)  
        );  
}
```

}

Angular - CRUD Operations Using HTTP

What are CRUD Operations?

CRUD is an acronym (a shortened name) that stands for **Create**, **Read**, **Update**, and **Delete**. These are the four basic **operations** performed on data in most databases and web applications such as Employee Management System, LMS, etc. Each operation has its own functionality as follows:

- **Create**: This operation is used to **add** (create) a new record to the database.
- **Read**: This operation is used to **retrieve** (view) existing data from the database.
- **Update**: This operation is used to **modify** (edit) existing data in the database.
- **Delete**: This operation is used to **remove** (delete) existing data from the database.

Now that we have a basic understanding of CRUD operations. Let's learn about implementing CRUD operations in Angular and how to perform them using **HTTP** (HyperText Transfer Protocol).

CRUD Operations in Angular

In Angular, the **CRUD** (Create, Read, Update, and Delete) operations play an important role in managing data and performing specific tasks. These operations are handled using the Angular [HttpClient](#) service class to interact with HTTP **RESTful** APIs.

These operations allows you as follows:

- **Create** new records (**POST** requests).
- **Read** existing data (**GET** requests).
- **Update** existing data (**PUT/PATCH** requests).
- **Delete** records (**DELETE** requests).

Let's discuss these angular operations one by one:

Create Operation

In Angular, the **Create** operation involves sending a **POST** request to **add (create)** a new record to the server. This is done using the **HttpClient** service class, which provides an easy way to communicate with a **RESTful** API's.

Signature

Following is the signature (different from syntax) of the **Create** operation (which is **POST** a method) –
post<T>(url: string, body: any | null, options?: Object): Observable<T>

Here,

- **URL** – The URL to which the Create (POST) request is sent.
- **body** – It represents the data to be sent (updated) to the server. Normally, it will be in JSON format.

- **options** – It represents the options to be sent along with the request, such as headers, query parameters, etc.
- **Observable<T>** – The return type, where 'T' represents the expected response type.

Read Operation

In Angular, the **Read** operation involves sending a [GET](#) request to **retrieve (get)** an existing record from the server. This is done using the **HttpClient** service class, which provides an easy way to communicate with **RESTful** API's.

Signature

Following is the signature (different from syntax) of the **Read** operation (which is **GET** a method) –

`get<T>(url: string, options?: Object): Observable<T>`

Here,

- **url** – The URL to which the Read (GET) request is sent.
- **options** – represents the options to be send along with the resource URL.
- **Observable<T>** – The return type, where 'T' represents the expected response type.

Update Operation

In Angular, the **Update** operation involves sending a [PUT/PATCH](#) request to **modify (update)** an existing record on the server. This is done using the **HttpClient** service class, which provides an easy way to communicate with **RESTful** API's.

Signature

Following is the signature (different from syntax) of the **Update** operation (which is **PUT/PATCH** a method) –

`put<T>(url: string, body: any, options?: Object): Observable<T>`

Here,

- **URL** – The URL to which the Update (PUT) request is sent.
- **body** – It represents the data to be sent (updated) to the server. Normally, it will be in JSON format.
- **options** – It represents the options to be sent along with the request, such as headers, query parameters, etc.
- **Observable<T>** – The return type, where 'T' represents the expected response type.

Delete Operation

In Angular, the **Delete** operation involves sending a [DELETE](#) request to **remove (delete)** an existing record on the server. This is done using the **HttpClient** service class, which provides an easy way to communicate with **RESTful** API's.

Signature

Following is the signature (different from syntax) of the **Delete** operation (which is **DELETE** a method)

–

`delete<T>(url: string, options?: Object): Observable<T>`

Here,

- **URL** – The URL to which the Delete (DELETE) request is sent.
- **options** – An object containing HTTP options such as headers, query parameters, etc.
- **Observable<T>** – The return type, where 'T' represents the expected response type.

Sample Example

We will create an Angular application named **EMS** (Employee Management System), where we will implement all four CRUD operations to 'create', 'read', 'update', and 'delete' employee data.

Now, let's see the step-by-step guides for setting up a mini Angular project and implementing all four CRUD operations:

Project Setup

Step 1: Open your friendly IDE's terminal (e.g, **vs code**) or [node.js](#) command, and go to your favorite workspace as follows:

```
cd /go/to/your/favorite/workspace
```

Step 2: To manage your Angular application via commands, install the Angular [CLI](#) using the following command:

```
npm install @angular/cli
```

```
//for latest version
```

```
npm install @angular/cli@latest
```

Step 3: Create a new Angular application using the following command ([see more](#)):

ng new my-crud-app

Once you run the above command, it will ask you a few questions. You can reply with the default answers as shown below:

? Which stylesheet format would you like to use? (Use arrow keys)

> CSS

SCSS [https://sass-lang.com/documentation/syntax#scss]

Sass [https://sass-lang.com/documentation/syntax#the-indented-syntax]

Less [http://lesscss.org]

....

? Which stylesheet format would you like to use? CSS

? Do you want to enable Server-Side Rendering (SSR) and Static Site

Generation (SSG/Prerendering)? Yes

Step 4: Navigate to your "application directory" and run the following commands one by one to create the four components:

```
cd my-crud-app
```

```
-----
```

```
ng generate component dashboard
```

```
ng generate component Create
```

```
ng generate component Update
```

```
ng generate component View
```

Step 5: Now, open the **app.component.html** file, remove everything, and leave only the following:

```
<router-outlet></router-outlet>
```

Step 6: Finally, run the application using the following command:

```
ng serve
```

Setting up Database

As we set up the application, we are ready to implement the CRUD operations in our Angular my-crud-app application. Let's set up the database too.

Step 7: Go to your assets folder (cd src/assets), create a **.json** file (e.g., db.json), and place the dummy data below:

```
{
  "employees": [
    {
      "id": "1",
      "name": "Employee1",
      "email": "employee1@gmail.com",
      "mobile": 9034123334,
      "department": "XYZ"
    }
  ]
}
```

The above data will be treated as the application **database** (endpoint URL), where we will send all the requests to manipulate data.

Step 8: Now, install the **JSON-Server** to access the above database as follows:

```
npm install json-server
```

Step 9: Run the following command to start the JSON-Server:

```
cd src -> assets
```

```
npx json-server --watch db.json
```

Open your browser and navigate to **http://localhost:3000** (default port) to see the JSON-Server data. To see the employee's data, navigate to **http://localhost:3000/employees**.

Setting up Service

Step 10: Install the service to communicate over the HTTP protocol for performing **CRUD** operations:

ng generate service services/auth

Once the above command is executed successfully, you will be able to see a folder named **services** containing two files: **auth.service.ts** and **auth.service.spec.ts**. We will implement all the CRUD operations in our "auth.service.ts" file.

To work with the HTTP protocol, make sure the **HttpClientModule** is added and imported into the root component as follows:

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterOutlet } from '@angular/router';
//here....
import { HttpClientModule } from '@angular/common/http';
```

```
@Component({
  selector: 'app-root',
  standalone: true,
  //here....
  imports: [CommonModule, RouterOutlet, HttpClientModule],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'my-crud-app';
```

```
}
```

Note: Make sure the **HttpClientModule** is also added in providers within the **app.config.ts** file as follows:

```
import { ApplicationConfig, importProvidersFrom } from '@angular/core';  
import { provideRouter } from '@angular/router';  
import { routes } from './app.routes';  
import { provideClientHydration } from '@angular/platform-browser';  
import { HttpClientModule } from '@angular/common/http';
```

```
export const appConfig: ApplicationConfig = {  
  providers: [provideRouter(routes), provideClientHydration(),  
    importProvidersFrom(HttpClientModule)],  
};
```

Step 11: Open your **auth.service.ts** file and place the below code:

```
import { HttpClient } from '@angular/common/http';  
import { Injectable } from '@angular/core';
```

```
@Injectable({  
  providedIn: 'root'  
})
```

```
export class AuthService {
```

```
  private baseUrl: string = "http://localhost:3000/employees";  
  constructor(private http: HttpClient) { }
```

```
  //read employees data ( Read operation)
```

```
  getAllEmployees(){  
    return this.http.get(this.baseUrl);  
  }
```

```
  //get employee by id (read operation)
```

```

getEmployeeById(id: any){
  return this.http.get(this.baseUrl + "/" + id);
}

//create or add new employee (Post operation)
createEmployee(obj: any){
  return this.http.post(this.baseUrl + "/", obj);
}

//update employee data (Put operation)
updateEmployee(id: any, obj: any) {
  return this.http.put(this.baseUrl + '/' + id , obj);
}

//delete employee (Delete operation)
deleteEmployee(id: any){
  return this.http.delete(this.baseUrl + "/" + id);
}
}

```

The above service class code will implement all four operations to "read", "update", "delete", and "create" employee data.

Setting up Routing

We need to set up [routing](#) that defines the paths for each component so that when the links or buttons are clicked, we can navigate through the different components.

Step 12: Open the **app.routes.ts** file, and place the below code to navigate to the respective component:

```

import { Routes } from '@angular/router';

import { DashboardComponent } from './dashboard/dashboard.component';

import { ViewComponent } from './view/view.component';

import { UpdateComponent } from './update/update.component';

import { CreateComponent } from './create/create.component';

```

```
export const routes: Routes = [
  {path: '', redirectTo: 'dashboard', pathMatch: 'full'},
  {path: 'dashboard', component: DashboardComponent},
  {path: 'create', component: CreateComponent},
  {path: 'view/:id', component: ViewComponent},
  {path: 'update/:id', component: UpdateComponent},
];
```

Reading Employees Data

To retrieve (read) all the employee data from the JSON-server, we will use the **HTTP GET** operation.

Step 13: Open the respective files of the dashboard component in your angular application i.e **dashboard.component.html**, **dashboard.component.ts**, and **dashboard.component.css** files, and place the respective code below:

dashboard.component.html file,

```
<div class="dashboard">
  <div class="header">
    <h2>Welcome to EMS</h2>
  </div>
  <div class="add">
    <a routerLink="/create">Add Employee</a>
  </div>
  <table>
    <tbody>
      <tr>
        <th>ID</th>
        <th>Name</th>
        <th>Email</th>
        <th>Actions</th>
      </tr>
      <tr *ngFor="let employee of allEmployees;">
        <td>{{employee.id}}</td>
        <td>{{employee.name}}</td>
        <td>{{employee.email}}</td>
```

```

        <td>

            <a [routerLink]="['/view', employee.id]" id="view">View</a>

            <a [routerLink]="['/update', employee.id]" id="update">Update</a>

            <a [routerLink]="['/delete', employee.id]" id="delete"
(click)="delete(employee.id)">Delete</a>

        </td>

    </tr>

</tbody>

</table>

<div class="center">{{message}}</div>

</div>

```

dashboard.component.ts file,

```

import { Component, OnInit } from '@angular/core';
import { AuthService } from '../services/auth.service';
import { CommonModule } from '@angular/common';
import { ActivatedRoute, RouterModule } from '@angular/router';

@Component({
  selector: 'app-dashboard',
  standalone: true,
  imports: [CommonModule, RouterModule],
  templateUrl: './dashboard.component.html',
  styleUrls: ['./dashboard.component.css']
})
export class DashboardComponent implements OnInit{
  constructor(private auth: AuthService, private router: ActivatedRoute){}
  allEmployees: any;
  ngOnInit(): void {
    this.auth.getAllEmployees().subscribe(res=>{
      this.allEmployees = res;
      console.log(this.allEmployees);
    });
  }
}

```



```
});  
}  
}
```

dashboard.component.css file,

```
*{  
    font-family: sans-serif;  
}  
  
.dashboard{  
    width: 70%;  
    margin: 20px auto;  
    box-shadow: 1px 3px 4px;  
    padding: 10px;  
}  
  
.header{  
    width: 100%;  
    text-align: center;  
    background-color: gainsboro;  
    position: relative;  
    padding: 8px 0px;  
}  
  
.add{  
    float: right;  
    position: relative;  
    top: 20px;  
    right: 55px;  
}  
  
.add a{  
    text-decoration: none;  
    background-color: rgb(133, 142, 142);  
    padding: 10px;  
    color: white;
```

```
border-radius: 5px;
cursor: pointer;
}
table{
    position: relative;
    width: 90%;
    border-collapse: collapse;
    margin: 50px auto;
    border-collapse: collapse;
    padding: 10px;
}
table th, td{
    border: 1px solid black;
    text-align: center;
    padding: 10px;
}
table th{
    padding: 10px;
}
table a{
    text-decoration: none;
    margin: 0px 10px;
    padding: 8px;
    border-radius: 5px;
    color: white;
}
#view{
    background-color: rgb(63, 147, 196);
}
#update{
    background-color: rgb(54, 101, 68);
```

```

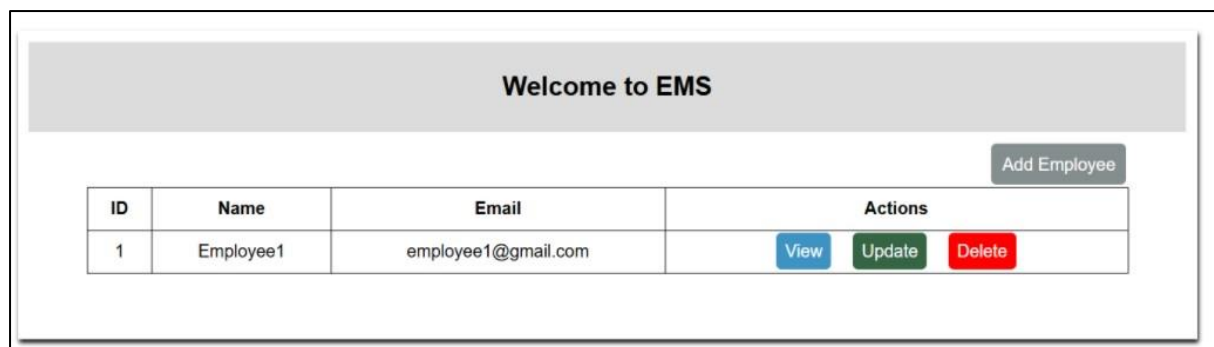
}

#delete{
    background-color: red;
}

.center{
    text-align: center;
}

```

The dashboard (landing) page will look like this:



Adding New Employee

To create (add) a new employee record in our JSON-server (i.e., database), we will use the **HTTP POST** operation.

Step 14: Now open **create.component.html**, **create.component.ts**, and **create.component.css** files and place the code below:

create.component.html file,

```

<div class="myForm">
    <div class="header">
        <h2>Create Employee Form</h2>
    </div>
    <form [formGroup]="createForm">
        <input type="text" placeholder="Name" formControlName="name" required>
        <input type="text" placeholder="Email" formControlName="email" required>
        <input type="text" placeholder="Mobile" formControlName="mobile" required>
        <input type="text" placeholder="Department" formControlName="department" required>
        <button type="button" (click)="createEmp()">Create</button>
        <a routerLink = "/dashboard">Go to Dashboard</a>
    </form>
</div>

```

```
        <p>{{message}}</p>
    </form>
</div>
```

create.component.ts file,

```
import { Component, OnInit } from '@angular/core';
import { RouterModule, Router } from '@angular/router';
import { AuthService } from '../services/auth.service';
import { FormBuilder, FormControl, FormGroup, FormsModule, ReactiveFormsModule } from '@angular/forms';
```

```
@Component({
  selector: 'app-create',
  standalone: true,
  imports: [RouterModule, FormsModule, ReactiveFormsModule],
  templateUrl: './create.component.html',
  styleUrls: ['./create.component.css']
})
```

```
export class CreateComponent implements OnInit {
```

```
  createForm: FormGroup = new FormGroup({});
```

```
  message: string = '';
```

```
  constructor(private auth: AuthService, private fb: FormBuilder, private router: Router) {}
```

```
  ngOnInit(): void {
```

```
    this.createForm = this.fb.group({
```

```
      name: new FormControl(''),
```

```
      email: new FormControl(''),
```

```
      mobile: new FormControl(''),
```

```
      department: new FormControl('')
```

```
    });
```

```
}
```

```

createEmp(): void {
  const name = this.createForm.get('name')?.value;
  if (name.trim().length == 0) {
    alert("Name can't be empty");
  } else {
    this.auth.createEmployee(this.createForm.value).subscribe(res => {
      this.message = "Employee created successfully...!";
      setTimeout(()=>{
        this.router.navigate(['/dashboard']);
      }, 1000);
    });
  }
}

```

create.component.css file,

```

*{
  font-family: sans-serif;
}

.myForm{
  width: 70%;
  margin: 20px auto;
  box-shadow: 1px 3px 4px;
  padding: 10px;
}

.header{
  width: 100%;
  text-align: center;
  background-color: gainsboro;
  position: relative;
  display: block;

```

```
padding: 8px 0px;
}
.myForm form{
width: 90%;
margin: 30px auto;
}
.myForm form input{
width: 90%;
display: block;
padding: 14px;
margin: 20px 0px;
border-radius: 5px;
}
.myForm form button{
padding: 10px 30px;
cursor: pointer;
background-color: rgb(56, 82, 130);
font-size: 18px;
border: none;
color: white;
border-radius: 5px;
}
a{
margin: 0px 10px;
}
```

The employee create page will look like:

Create Employee Form

Create
[Go to Dashboard](#)

Employee created successfully...!

Once the new employee is created, the dashboard will look like this:

Welcome to EMS

Add Employee

ID	Name	Email	Actions
1	Employee1	employee1@gmail.com	View Update Delete
0ce1	Aman Gupta	aman12@gmail.com	View Update Delete
59df	Revathi Satya	revathi13@gmail.com	View Update Delete

Updating Employee Data

To update the employee record (data), we will use the **HTTP PUT** operation.

Step 15: Open the **update.component.html**, **update.component.ts**, and **update.component.css** files and place the respective code below:

update.component.html file,

```
<div class="myForm" *ngIf="isLoading">
  <div class="header">
    <h2>Create Employee Form</h2>
  </div>
  <form [formGroup]="updateForm">
    <input type="text" placeholder="Name" formControlName="name" required>
    <input type="text" placeholder="Email" formControlName="email" required>
    <input type="text" placeholder="Mobile" formControlName="mobile" required>
```

```

    <input type="text" placeholder="Department" formControlName="department" required>
    <button type="button" (click)="UpdateEmp()">Update</button>
    <a routerLink = "/dashboard">Go to Dashboard</a>
    <p>{{message}}</p>
  </form>
</div>

```

update.component.ts file,

```

import { CommonModule } from '@angular/common';
import { Component, OnInit } from '@angular/core';
import { FormBuilder, FormControl, FormGroup, FormsModule, ReactiveFormsModule } from '@angular/forms';
import { ActivatedRoute, RouterModule, Router } from '@angular/router';
import { AuthService } from '../services/auth.service';

@Component({
  selector: 'app-update',
  standalone: true,
  imports: [FormsModule, ReactiveFormsModule, CommonModule, RouterModule],
  templateUrl: './update.component.html',
  styleUrls: ['./update.component.css']
})
export class UpdateComponent implements OnInit {
  constructor(private route: ActivatedRoute, private fb: FormBuilder, private auth: AuthService, private router: Router){}

  empId: any;
  empDetails: any;
  isLoading: boolean = false;
  updateForm: FormGroup = new FormGroup({});
  ngOnInit(): void {
    this.route.params.subscribe(res=>{
      this.empId = res['id'];
      console.log(this.empId);
    });
  }
}

```



```

});
if(this.empId !== ""){
  this.auth.getEmployeeById(this.empId)
    .toPromise()
    .then(res=>{
      this.empDetails = res;
      this.updateForm = this.fb.group({
        'name': new FormControl(this.empDetails.name),
        'email': new FormControl(this.empDetails.email),
        'mobile': new FormControl(this.empDetails.mobile),
        'department': new FormControl(this.empDetails.department)
      });
      this.isLoading = true;
    });
}
}
message: any;
UpdateEmp(){
  this.auth.updateEmployee(this.empId, this.updateForm.value).subscribe(res=>{
    this.message = "Employee updated successfully..!";
    setTimeout(()=>{
      this.router.navigate(['/view/' + this.empId]);
    }, 1000);
  });
}
}

```

update.Component.css file,

```

*{
  font-family: sans-serif;
}
.myForm{

```

```
width: 70%;  
margin: 20px auto;  
box-shadow: 1px 3px 4px;  
padding: 10px;  
}  
.header{  
width: 100%;  
text-align: center;  
background-color: gainsboro;  
position: relative;  
display: block;  
padding: 8px 0px;  
}  
.myForm form{  
width: 90%;  
margin: 30px auto;  
}  
.myForm form input{  
width: 90%;  
display: block;  
padding: 14px;  
margin: 20px 0px;  
border-radius: 5px;  
}  
.myForm form button{  
padding: 10px 30px;  
cursor: pointer;  
background-color: rgb(56, 82, 130);  
font-size: 18px;  
border: none;  
color: white;
```

```

border-radius: 5px;
}
a{
margin: 0px 10px;
}

```

The employee update page looks like:

Deleting Employee

To delete an employee, we will use the **HTTP DELETE** operation.

Step 16: Simply add the function below to your **dashboard.component.ts** file:

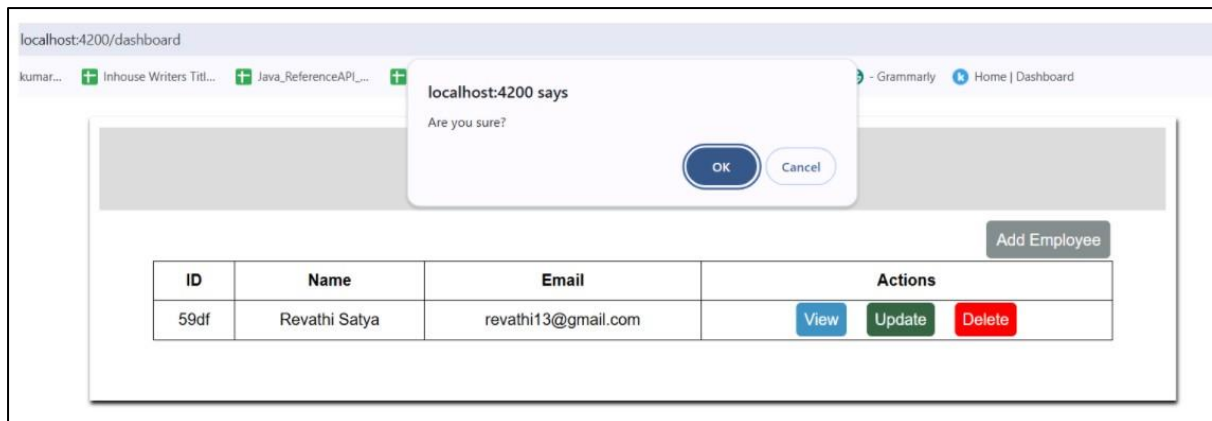
```

message = "";
delete(id: any){
  if(confirm("Are you sure?") == false){
    this.message = "Canceled.....!";
    setTimeout(()=>{
      this.message = "";
    }, 2000);
    return;
  }
  this.auth.deleteEmployee(id).subscribe(res=>{
    this.message = "Deleted.....!";
    this.allEmployees = this.allEmployees.filter((emp) => emp.id !== id);
  });
}

```

```
setTimeout(()=>{  
    this.message = "";  
}, 2000);  
})  
}
```

After deleting an employee, the dashboard will look like this:



Angular - Modules

Angular **modules** are **core concepts** in Angular application, which help to organize the application structure by grouping the related components, services, etc.

In this chapter, we will learn about the angular module, its importance in the application, how to create a module, and how to use it in our application in detail.

What are Angular Modules?

In Angular, a **module** refers to a place or container where you can group the [components](#), [directives](#), [pipes](#), and [services](#), which are related to the application. This helps organize the application, making it easier to understand and manage dependencies efficiently.

For example, if you are developing a website or an application, the **header**, **footer**, **left**, **center**, and **right** sections become part of a module. However, they are not modules themselves, they are individual components that belong to that module.

Types of Angular Modules

Following is a list of commonly used **Angular modules** –

- [Root Module](#)
- [Feature Module](#)
- [Shared Module](#)
- [Routing Module](#)

Importance of Angular Modules

In Angular, a **module** plays an important role in "structuring an Angular application", making it easier for others to understand by simply viewing the application's hierarchy. An application is considered well-structured if each concern (say component or section) is separated into its own module.

For example, **login** and **signup** components can belong to an **Authentication** module.

How to Create an Angular Module?

To create an Angular module, you must have a basic understanding of the [first Angular application](#) creation and about CLI, etc. Here are the step-by-step guides to creating an Angular module in an application:

Step 1: In your code editor (such as VS Code) open any existing angular application or create new one.

Step 2: Open the "terminal" in your code editor and go to your application directory as follows:

```
cd application_directory
```

e.g....

```
cd myapp
```

Step 3: Now create a **new module** using the following command:

ng generate module auth

Here, **auth** is your "module name".

Once the above command is executed successfully, you may see:

CREATE src/app/auth/auth.module.ts (202 bytes)

Step 4: Toggle the **Auth** folder you may able to see the **auth.module.ts** file with some default code as follows:

```
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
```

```
@NgModule({
  declarations: [],
  imports: [
    CommonModule
  ]
})
export class AuthModule { }
```

How to use Angular Module?

As we have already created a new module named "**auth**" in our application. To use it properly in our angular application follow the steps below:

Step 1: Import Module

Import the newly created module within your root **module (older version)** or in the **component (recent version)** of your application as follows:

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { AuthModule } from '../auth/auth.module';
```

```
@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, AuthModule],
  templateUrl: './app.component.html',
```

```

    styleUrls: ['./app.component.css']
  })
  export class AppComponent {

  }

```

Step 2: Create Components

Now create "two" different components named **login** and **signup** as follows:

```

ng g c auth/login --no-standalone
ng g c auth/signup --no-standalone

```

Note: Make sure that the **component** within the **@NgModule** should be **"no standalone"**, otherwise, you may get an "error".

Step 3: Import and Export the components

Open the **auth.module.ts** file and import both the components within the **@NgModule** as follows:

```

import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { SignupComponent } from './signup/signup.component';
import { LoginComponent } from './login/login.component';

```

```

@NgModule({
  declarations: [SignupComponent, LoginComponent],
  imports: [
    CommonModule
  ],
  exports: [LoginComponent, SignupComponent]
})
export class AuthModule { }

```

Step 4: Use Components in your Templates

You can now use the **LoginComponent** and **SignupComponent** in your templates. For example, in the **app.component.html** file, you can include the following:

```

<app-login></app-login>

<app-signup></app-signup>

```

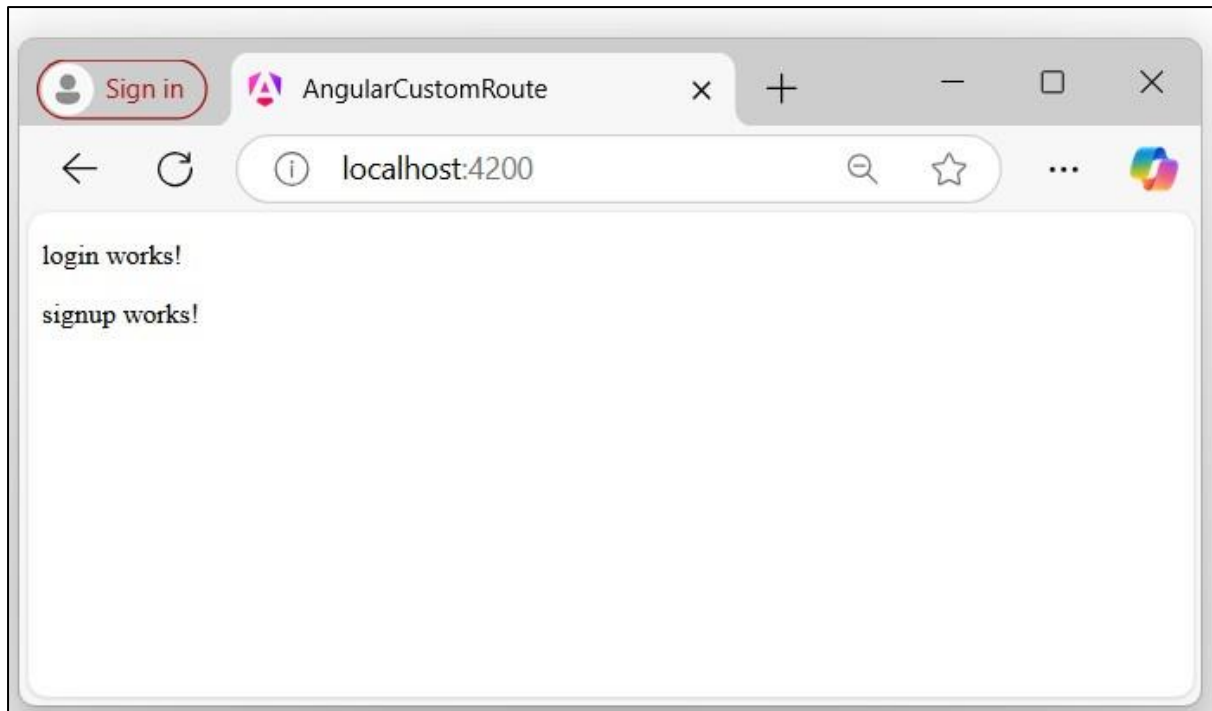
Step 5: Run Application

Finally, run the application to see the output:

ng serve

Step 6: Navigate to localhost:4200

Open your friendly browser (e.g **chrome**) and navigate through the **localhost:4200**.



As you can see in the above picture the the login and signup components belongs to AuthModule.

Conclusion

In conclusion, the **Angular modules** are important for building "scalable", "maintainable", and structured applications. Modules in Angular help organize and structure your code, making it easier to manage and understand.

Angular - Service Workers & PWA

Progressive web apps (PWA) are normal web application with few enhancements and behaves like a native application. PWA apps does not depends on network to work. PWA caches the application and renders it from local cache. It regularly checks the live version of the application and then caches the latest version in the background.

PWA can be installed in the system like native application and shortcut can be shown in the desktop. Clicking the shortcut will open the application in browser with local cache even without any network available in the system.

Angular application can be converted into PWA application. To convert an Angular application, we need to use service worker API. Service worker is actually a proxy server, which sits in between the browser, application and the network.

Service workers is separate from web pages. It does not able to access DOM objects. Instead, Service Workers interact with web pages through **PostMessage** interface.

PWA application has two prerequisites. They are as follows,

- **Browser support** – Even though lot of browser supports the PWA app, IE, Opera mini and few other does not provides the PWA support.
- **HTTPS delivery** – The application needs to be delivered through HTTPS protocol. One exception of the https support is **localhost** for development purpose.

Let us create a new application and convert it into PWA application.

Create a new Angular application using below command –

```
cd /go/to/workspace
```

```
ng new pwa-sample
```

Add PWA support using below command –

```
cd pwa-sample
```

```
ng add @angular/pwa --project pwa-sample
```

Build the production version of the application,

```
ng build --prod
```

PWA application does not run under Angular development server. Install, a simple web server using below command –

```
npm install -g http-server
```

Run the web server and set our production build of the application as root folder.

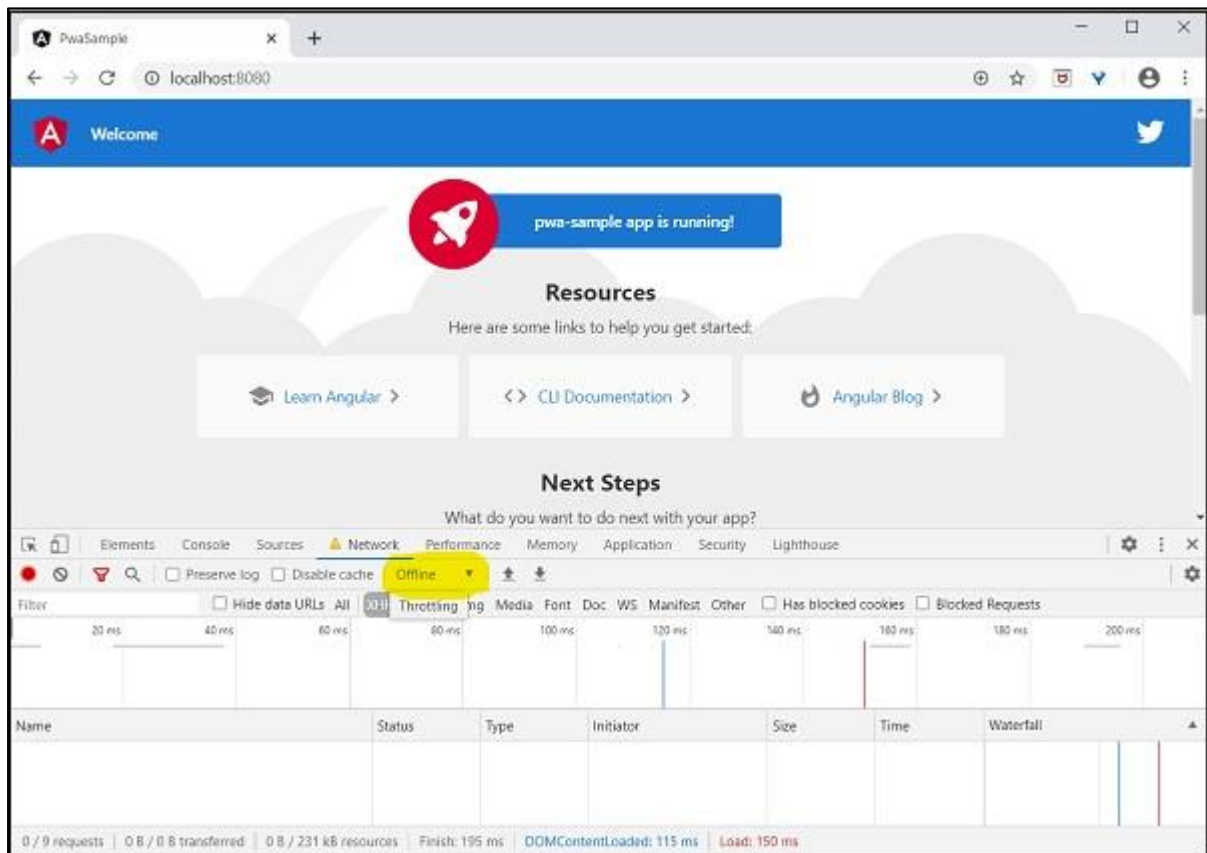
f the application as root folder.

```
http-server -p 8080 -c-1 dist/pwa-sample
```

Open browser and enter **http://localhost:8080**.

Now, go to **Developer tools -> Network** and select **Offline** option.

Normal application stops working if network is set to Offline but, PWA application works fine as shown below –



Angular - Signals

This chapter will discuss **Angular Signals**, a new way to build Angular applications. In this, we will discuss what exactly Signals are, why they are really useful in Angular, and how they can improve your application by providing a simpler and more efficient approach.

The chapter will also highlight the advantages of using Signals in Angular development.

What are Signals in Angular?

A **Signal** is a wrapper around a value that notifies interested consumers when that value changes. A signal can contain any value, from primitives to complex data structures.

In Angular, **Signals** are a system (or reactive primitives) that **granularly (closely) tracks and updates** how and where your state is used throughout the application, allowing the framework to optimize rendering updates.

When a signal value changes, it automatically triggers updates to any dependent components or services that are observing it. For example:

```
// Declaring a regular variable
```

```
count1: number = 0;
```

```
// Declaring a variable using signals
```

```
count2 = signal(0);
```

In this case, **count1** will not automatically reflect the latest value when it changes. However, **count2**, which is a signal, will automatically update and reflect the latest value whenever it changes.

Important! In Angular, the signals were introduced in version **16.0** and became stable in version **17.0**.

Note: To read a signal, you need to call its **getter** function, which allows angular to track where the signal is used. The signals in angular may be either **writable** or **read-only**.

Reactive Primitives in Angular Signals

Here is a list of the commonly used **Signals** or **reactive primitives (constructs)** in Angular:

- [Writable Signals](#)
- [Computed Signals](#)
- [Effects](#)

Writable Signals

In Angular, the **writable signals** are specific types of signals that allow you to "**modify** the value directly". In addition, they are used to represent the value that can be changed.

These signals provide an API for "updating their values directly". You can create writable signals by calling the `signal` function with the signal's initial value.

Syntax

Following is the syntax of the Angular **Writable Signal** –

```
count = signal(initial_value = 0);
```

Here,

- The **count** is a variable whose "initial signal value" is **0** (can be any value).
- **Signal()** is a **constructor** that "defines the writable signal".

Here are "two" commonly used methods to **set** or **update** the writable signal value:

- `set()`
- `update()`

The `set()` method

To "change" the value of the **writable signal**, you can use the **set()** method to set it directly.

Syntax

Below is the syntax of the **set()** method –

```
//Initializing a signal with an initial value
```

```
count.set(initial_value);
```

Here, **count** is a variable for which we 'set' the **initial_value**.

The `update()` method

You can use the **update()** method or operation to compute a "new value" from the "previous one".

Syntax

Below is the syntax of the **update()** method –

```
count.update(value => value + new_value);
```

Here, the **value** is the previous "value", which will be updated by adding a **new_value** in it.

Note! The writable signals have the type **WritableSignal**.

Example of Writable Signal in Angular

We create a **Signal** with an initial value of **0** using the **signal(0) constructor**. We will use the **set()** and **update()** methods in different functions to "increase" and "decrease" the **count** value on each click of the given button –

TypeScript code (component.ts)

```
import { Component, OnInit, Signal, signal } from '@angular/core';
```

```
import { CommonModule } from '@angular/common';
```

```
import { RouterOutlet } from '@angular/router';
```

```
@Component({
```

```

selector: 'app-root',
standalone: true,
imports: [CommonModule, RouterOutlet],
templateUrl: './app.component.html',
styleUrl: './app.component.css'
})
export class AppComponent{
  //declaring a signal
  count = signal(0);
  increase(){
    //using .set() method to increase count value
    this.count.set(this.count()+1);
  }
  decrease(){
    //using .update() method to decrease count value
    this.count.update(value => this.count()-1);
  }
}

```

HTML Code (component.html)

```

<div>
  <div class="count">
    <h3>{{count()}}</h3>
  </div>
  <div class="btn">
    <button (click)="increase()">Increase count</button>
    <button (click)="decrease()">Decrease count</button>
  </div>
</div>

```

CSS Code (component.css)

```

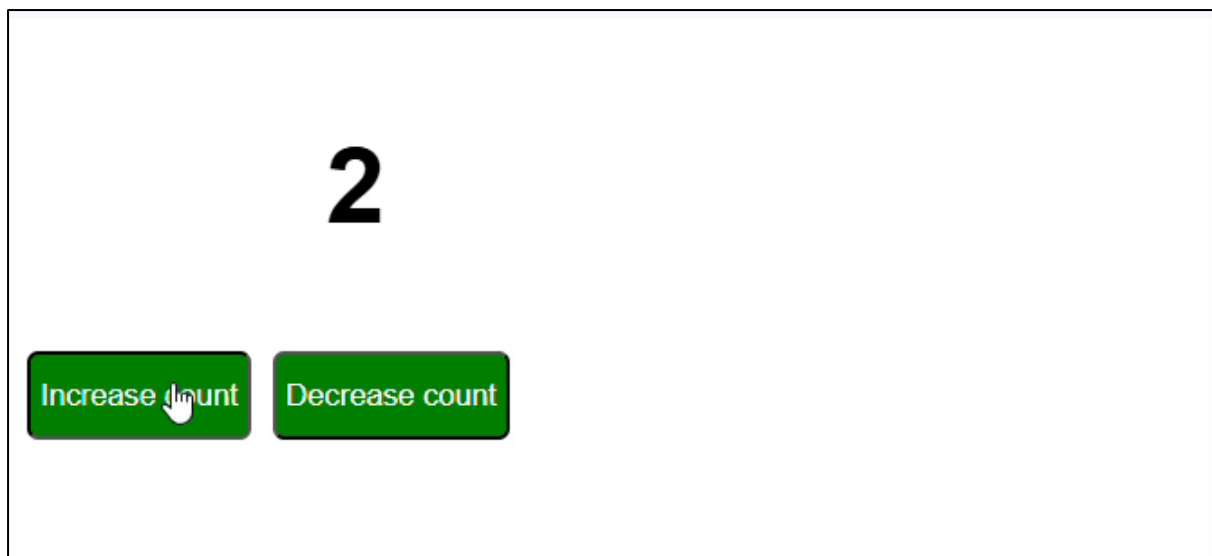
.main{
  width: 300px;
}

```

```
}  
.count{  
  font-size: 40px;  
  font-weight: bold;  
  font-family: sans-serif;  
  text-align: center;  
}  
button{  
  padding: 10px 4px;  
  margin: 0px 5px auto;  
  background-color: green;  
  color: white;  
  border-radius: 5px;  
  cursor: pointer;  
}
```

Output

Following is the output of the above code:



Computed Signals

In Angular, the **Computed Signals** are **read-only** signals, which means they "can't be edited" by the user. The computed signals drive their values from other signals (e.g., writable signals).

You can define the **Computed signal** using the **computed()** function by specifying the derivation.

Syntax

Following is the syntax of the Angular **Computed Signal** –

```
// Initializing a signal with an initial value  
count = signal(initial_value);  
  
// Creating a computed signal  
new_count: Signal<number> = computed(() => count() + new_value);
```

Here,

In the above snippet of code, the **new_count signal** depends on the count signal, so whenever the **count signal** updates, angular knows that the new_count also needs to be updated.

Important Points of the Computed Signals

1. Computed signals are both lazily evaluated and memorized

In angular, the **Computed** (read-only) signals are both lazily evaluated and memorized, so the **new_count** derivation function does not run to calculate its value until the first time you read the "new_count".

2. Computed signals are not writable signals

The **Computed** signals are "not writable signals", because you can not directly assign values to them as:

```
new_count.set(10);
```

The above snippet of code in the application will produce a compilation error because **new_count** is "not" a **Writable Signal**.

Example of Computed Signal in Angular

We will create two writable signals named **length** and **breadth** with initial values of **20** and **40**. Then we will create a **Computed** signal named **area**, which will be the "product" of both writable signal –

TypeScript code (component.ts)

```
import { Component, computed, signal, WritableSignal } from '@angular/core';  
import { CommonModule } from '@angular/common';  
import { RouterOutlet } from '@angular/router';
```

```
@Component({  
  selector: 'app-root',  
  standalone: true,  
  imports: [CommonModule, RouterOutlet],  
  templateUrl: './app.component.html',
```

```

    styleUrls: ['./app.component.css']
  })
  export class AppComponent{
    length: WritableSignal<number> = signal(20);
    breadth: WritableSignal<number> = signal(40);
    area:any = 0;
    calculate(){
      this.area = computed(()=> this.length() * this.breadth());
    }
  }
}

```

HTML code (component.html)

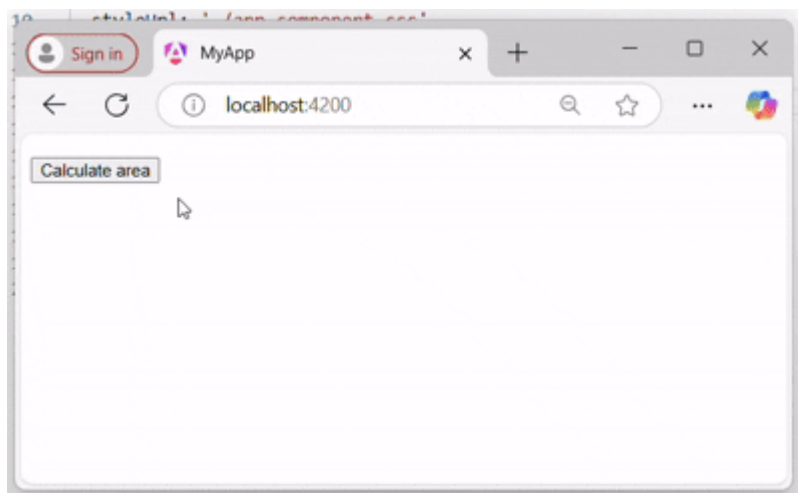
```

<div class="main">
  <div class="count">
    <h3>{{area()}}</h3>
  </div>
  <div class="btn">
    <button (click)="calculate()">Calculate area</button>
  </div>
</div>

```

Output

The output of the above code –



Note: When the **Writable** signals (length and breadth) values will change the **Computed** signal (area) value will change automatically.

Effects

In Angular, the **effects** is an "operation" that "runs" when "one or more signal values changed" at a time. The effect always runs at least once.

Similar to the **Angular signals**, **effects** keep track of their dependencies dynamically.

***Hint:** The good place to create **effects** is within the **constructor** because the effect function requests an "injection context".*

Syntax

Following is the syntax of the Angular **effects** –

```
effect(() => {  
  //statement or expressions....  
})
```

effect accepts a "function", within which will perform all the tasks.

Example of Effect in Angular Signal

We will create "two effects" for **count** and **color signal** within the constructor, which will run when one or more signal values change or at least once when the application runs –

TypeScript code (component.ts)

```
import { Component, effect, signal } from '@angular/core';  
import { CommonModule } from '@angular/common';  
import { RouterOutlet } from '@angular/router';  
  
@Component({  
  selector: 'app-root',  
  standalone: true,  
  imports: [CommonModule, RouterOutlet],  
  templateUrl: './app.component.html',  
  styleUrls: ['./app.component.css']  
})  
  
export class AppComponent{  
  constructor( ){  
    //count signal  
    effect(()=>{  
      console.log("Effect due to count signal is trigger: " + this.count());  
    })  
  }  
}
```

```
});

//color signal
effect(()=>{
  console.log("Effect due to color signal is trigger: " + this.color());
});
}
```

```
count = signal(0);
color = signal(["red", "green"]);
```

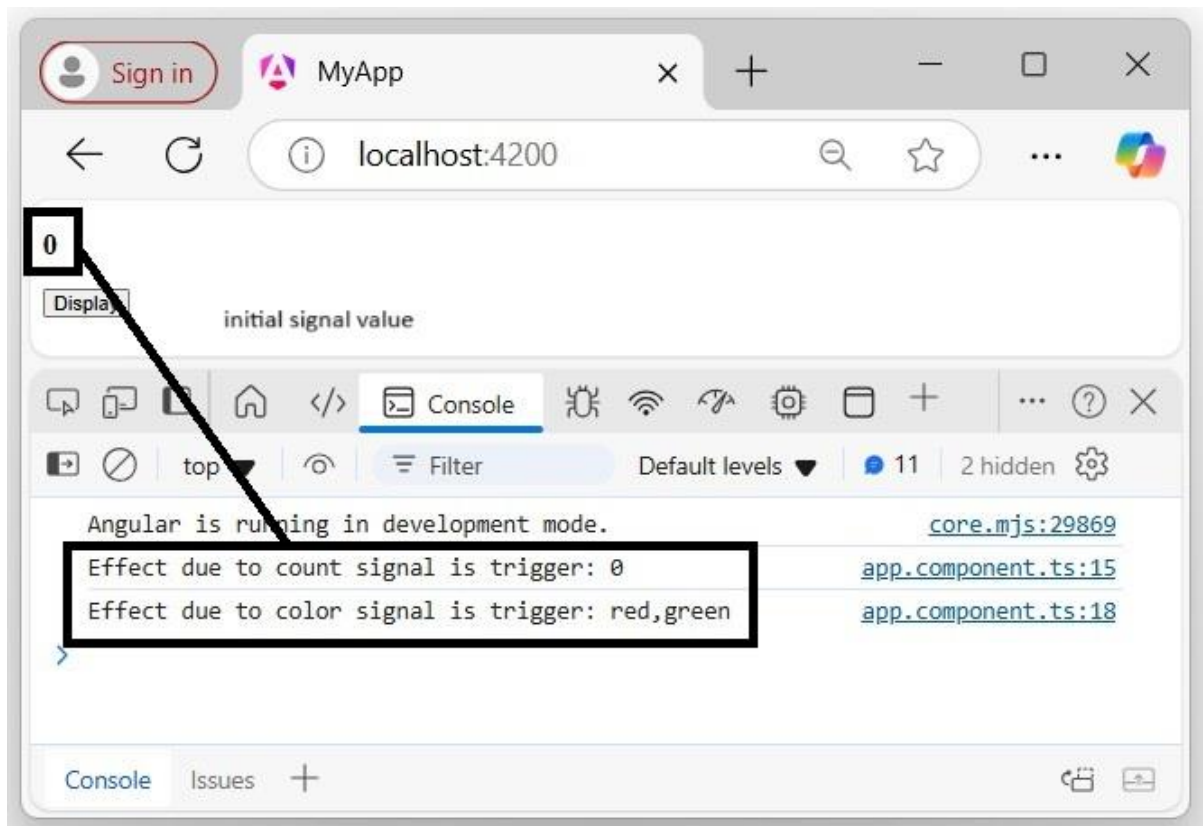
```
display(){
  this.count.set(30);
  this.color.update(value => [...value, "Yellow"]);
}
}
```

HTML code (component.html)

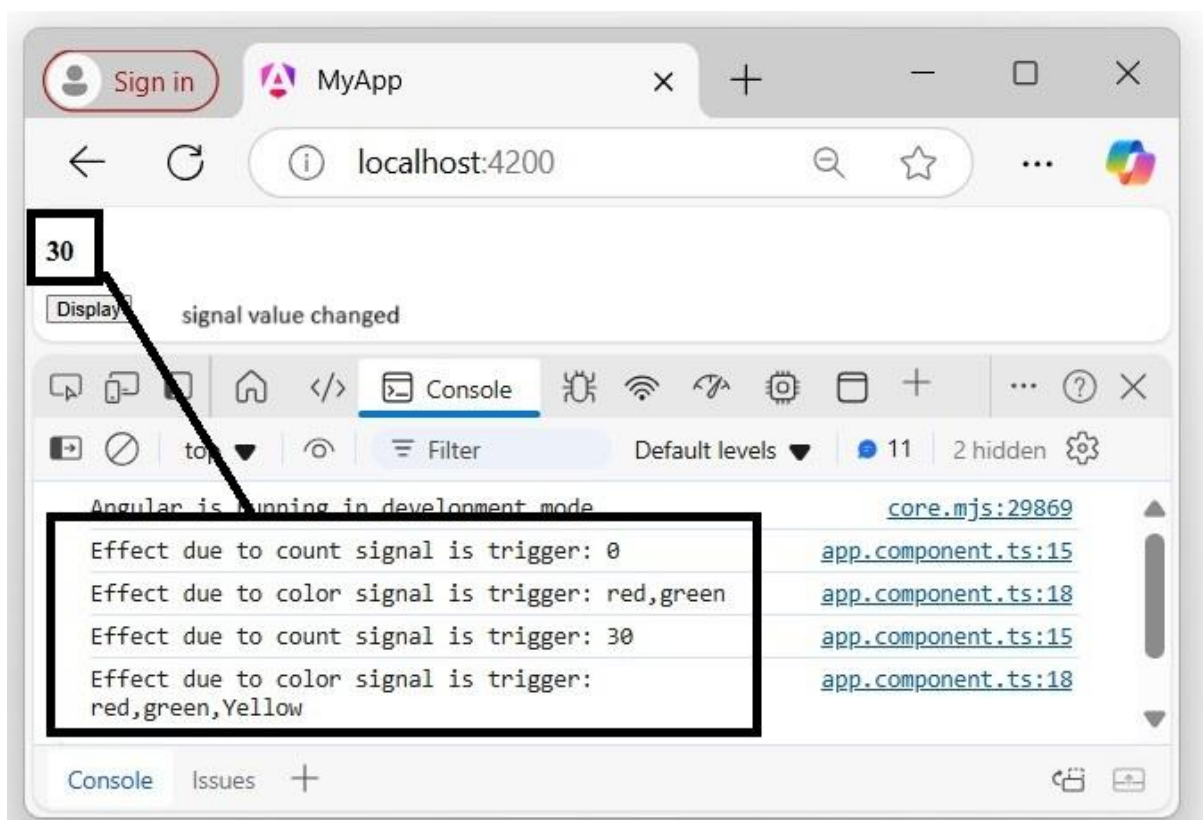
```
<div class="main">
  <div class="count">
    <h3>{{count()}}</h3>
  </div>
  <div class="btn">
    <button (click)="display()">Display</button>
  </div>
</div>
```

Output

When the signal "values are not changed", the **effect** runs at least once:



When signal "values changed":



Why to use Signals in Angular?

Here are some points that tell about why to use the **Signals** in Angular project:

- **Automatic updates:** Signals are automatically update any dependent computed signals or view when their values changes.
- **Efficient Change Detection:** Signals are optimized for minimal change detection cycles, which enhance the application performance.
- **Composable Logic:** Signals and computed signals can be used together to build complex reactive logic in a flexible and reusable manner.

Advantages of Signals in Angular

Below is a list of the advantages of the Angular **Signals** –

- **Signals** make Angular lighter and point the way to a future without Zone.js (used for change detection). They enable Angular to find out about components that need to be updated directly.
- We will be notified when the signal value changes, and then do something in response to the new signal value.
- It is avoiding unnecessary checks of components whose data didn't change.

Angular - Authentication and Authorization

Authentication is the process matching the visitor of a web application with the pre-defined set of user identity in the system. In other word, it is the process of recognizing the users identity. Authentication is very important process in the system with respect to security.

Authorization is the process of giving permission to the user to access certain resource in the system. Only the authenticated user can be authorized to access a resource.

Let us learn how to do Authentication and Authorization in Angular application in this tutorial.

Guards in Routing

In a web application, a resource is referred by url. Every user in the system will be allowed access a set of urls. For example, an administrator may be assigned all the url coming under administration section.

As we know already, URLs are handled by [Routing](#). Angular routing enables the urls to be guarded and restricted based on programming logic. So, a url may be denied for a normal user and allowed for an administrator.

[Angular](#) provides a concept called **Router Guards** which can be used to prevent unauthorized access to certain part of the application through routing. Angular provides multiple guards and they are as follows:

- **CanActivate:** Used to stop the access to a route.
- **CanActivateChild:** Used to stop the access to a child route.

- **CanDeactivate:** Used to stop ongoing process getting feedback from user. For example, delete process can be stop if the user replies in negative.
- **Resolve:** Used to pre-fetch the data before navigating to the route.
- **CanLoad:** Used to load assets.

Working Example

In this example, we are going to add login and logout functionality to an angular application and secure it using **CanActivate** guard. Follow the given steps:

Step 1: Create an angular application using the following command –

```
ng new new-app
```

Step 2: Now, navigate to project root folder.

```
cd new-app
```

Step 3: Next, we include the Bootstrap into our **new-app** application using **styles** option and change the default template to use Bootstrap components. Use the below command to install **Bootstrap** and **JQuery** in the project –

```
npm install --save bootstrap jquery
```

Now, open **angular.json** file and set bootstrap and jquery library path –

```
{
  "projects": {
    "expense-manager": {
      "architect": {
        "build": {
          "builder": "@angular-devkit/build-angular:browser", "options": {
            "outputPath": "dist/expense-manager",
            "index": "src/index.html",
            "main": "src/main.ts",
            "polyfills": "src/polyfills.ts",
            "tsConfig": "tsconfig.app.json",
            "aot": false,
            "assets": [
              "src/favicon.ico",
              "src/assets"
            ],
          },
        },
      },
    },
  },
}
```

```

    "styles": [
      "./node_modules/bootstrap/dist/css/bootstrap.css", "src/styles.css"
    ],
    "scripts": [
      "./node_modules/jquery/dist/jquery.js",
      "./node_modules/bootstrap/dist/js/bootstrap.js"
    ]
  },
},
}
}},
"defaultProject": "expense-manager"
}

```

Here,

scripts option is used to include JavaScript library.

Step 4: Create a new service named **AuthService** to authenticate the user. The command given below will create a service with the name **auth** inside **Services** folder.

ng generate service Services/auth

On successful creation of service, you may see the below output on Angular CLI –

CREATE src/app/Services/auth.service.spec.ts (363 bytes)

CREATE src/app/Services/auth.service.ts (142 bytes)

Step 5: Open **AuthService** and include below code:

```

import { Injectable } from '@angular/core';
import { Observable, of, BehaviorSubject } from 'rxjs';
import { tap, delay } from 'rxjs/operators';

@Injectable({
  providedIn: 'root'
})
export class AuthService {

  // Track login state with BehaviorSubject
  private isUserLoggedInSubject = new BehaviorSubject<boolean>(false);

```

```

constructor() {
    // Only initialize sessionStorage on the client-side (browser)
    if (typeof window !== 'undefined' && window.sessionStorage) {
        const storedLoginState = sessionStorage.getItem('isUserLoggedIn') === 'true';
        this.isUserLoggedInSubject.next(storedLoginState);
    }
}

```

```

login(userName: string, password: string): Observable<boolean> {
    const isLoggedIn = userName === 'admin' && password === 'admin';

    if (typeof window !== 'undefined' && window.sessionStorage) {
        sessionStorage.setItem('isUserLoggedIn', isLoggedIn ? 'true' : 'false');
    }

    // Update the BehaviorSubject with new login state
    this.isUserLoggedInSubject.next(isLoggedIn);

    return of(isLoggedIn).pipe(
        delay(1000),
        tap(val => console.log("Is User Authentication successful: " + val))
    );
}

```

```

logout(): void {
    if (typeof window !== 'undefined' && window.sessionStorage) {
        sessionStorage.removeItem('isUserLoggedIn');
    }

    // Update the BehaviorSubject to false when logged out
    this.isUserLoggedInSubject.next(false);
}

```

```
// Expose the login status as an observable
get isUserLoggedIn$(): Observable<boolean> {
    return this.isUserLoggedInSubject.asObservable();
}
}
```

Here,

- We have written two methods, **login** and **logout**.
- The purpose of the **login** method is to validate the user and if the user successfully validated, it stores the information in **localStorage** and then returns true.
- Authentication validation is that the user name and password should be **admin**.
- We have not used any backend. Instead, we have simulated a delay of 1s using Observables.
- The purpose of the **logout** method is to invalidate the user and removes the information stored in **localStorage**.

Step 6: Create a **login** component using below command –

ng generate component login

Following output will be produced on running the above code –

CREATE src/app/login/login.component.html (21 bytes)

CREATE src/app/login/login.component.spec.ts (608 bytes)

CREATE src/app/login/login.component.ts (242 bytes)

CREATE src/app/login/login.component.css (0 bytes)

Step 7: Open **LoginComponent** and update the existing code with the below code –

```
import { Component, OnInit } from '@angular/core';
import { FormGroup, FormControl, ReactiveFormsModule } from '@angular/forms';
import { AuthService } from '../Services/auth.service';
import { Router } from '@angular/router';
```

```
@Component({
  selector: 'app-login',
  standalone: true,
  imports: [ReactiveFormsModule],
  templateUrl: './login.component.html',
  styleUrls: ['./login.component.css']
})
```



```

})

export class LoginComponent implements OnInit {

  userName: string = "";
  password: string = "";
  formData!: FormGroup;

  constructor(private authService : AuthService, private router : Router) { }

  ngOnInit() {
    this.formData = new FormGroup({
      userName: new FormControl("admin"),
      password: new FormControl("admin"),
    });
  }

  onClickSubmit(data: any) {
    this.userName = data.userName;
    this.password = data.password;

    console.log("Login page: " + this.userName);
    console.log("Login page: " + this.password);

    this.authService.login(this.userName, this.password)
      .subscribe( data => {
        console.log("Is Login Success: " + data);
        if(data) this.router.navigate(['/expenses']);
      });
  }
}

```

Here,

- Used [reactive forms](#).

- Imported AuthService and Router and configured it in constructor.
- Created an instance of FormGroup and included two instance of FormControl, one for user name and another for password.
- Created a onClickSubmit to validate the user using authService and if successful, navigate to expense list.

Step 8: Open **LoginComponent** template and include below template code.

```
<div class="container">

  <div class="row">

    <div class="col-lg-12 text-center" style="padding-top: 20px;">

      <div class="container box" style="margin-top: 10px; padding-left: 0px; padding-right: 0px;">

        <div class="row">

          <div class="col-12" style="text-align: center;">

            <form [formGroup]="formData" (ngSubmit)="onClickSubmit(formData.value)"
class="form-signin">

              <h2 class="form-signin-heading">Please sign in</h2>

              <label for="inputEmail" class="sr-only">Email address</label>

              <input type="text" id="username" class="form-control" formControlName="userName"
placeholder="Username" required autofocus>

              <label for="inputPassword" class="sr-only">Password</label>

              <input type="password" id="inputPassword" class="form-control"
formControlName="password" placeholder="Password" required>

              <button class="btn btn-lg btn-primary btn-block" type="submit">Sign in</button>

            </form>

          </div>

        </div>

      </div>

    </div>

  </div>

</div>
```

Here, we created a reactive form and designed a login form. And, attached the **onClickSubmit** method to the form submit action.

Step 9: Open **LoginComponent** CSS file and include below CSS Code.

```
.form-signin {
```

```
max-width: 330px;

padding: 15px;
margin: 0 auto;
}
```

```
input {
  margin-bottom: 20px;
}
```

Here, some styles are added to design the login form.

Step 10: Create a logout component using below command –
ng generate component logout

You may see the below output –

```
CREATE src/app/logout/logout.component.html (22 bytes)
CREATE src/app/logout/logout.component.spec.ts (615 bytes)
CREATE src/app/logout/logout.component.ts (246 bytes)
CREATE src/app/logout/logout.component.css (0 bytes)
```

Step 11: Open **LogoutComponent** and include below code.

```
import { Component, OnInit } from '@angular/core';
import { AuthService } from '../Services/auth.service';
import { Router } from '@angular/router';
```

```
@Component({
  selector: 'app-logout',
  standalone: true,
  imports: [],
  templateUrl: './logout.component.html',
  styleUrls: ['./logout.component.css']
})
```

```
export class LogoutComponent implements OnInit {
```

```
constructor(private authService : AuthService, private router: Router) { }
```

```
ngOnInit() {  
  this.authService.logout();  
  this.router.navigate(['/']);  
}
```

```
}
```

Here,

- Used logout method of AuthService.
- Once the user is logged out, the page will redirect to home page (/).

Step 12: Create a guard using below command –

ng generate guard authenticate

On running the above command, Angular CLI will ask "Which type of guard would you like to create?" Choose **canActivate**.

CREATE src/app/authenticate.guard.spec.ts (510 bytes)

CREATE src/app/authenticate.guard.ts (141 bytes)

Step 13: Open authenticateGuard and include below code –

```
import { Injectable } from '@angular/core';
```

```
import { CanActivate, ActivatedRouteSnapshot, RouterStateSnapshot, Router, UrlTree } from  
'@angular/router';
```

```
import { AuthService } from './Services/auth.service';
```

```
@Injectable({  
  providedIn: 'root'  
})
```

```
export class authenticateGuard implements CanActivate {
```

```
  constructor(private authService: AuthService, private router: Router) {}
```

```
  canActivate(  

```

```

    next: ActivatedRouteSnapshot,
    state: RouterStateSnapshot): boolean | UrlTree {
    let url: string = state.url;
    return this.checkLogin(url);
  }

  checkLogin(url: string): true | UrlTree {
    console.log("Url: " + url);

    // Check if sessionStorage is available (only in the browser)
    if (typeof window !== 'undefined' && window.sessionStorage) {
      let val = sessionStorage.getItem('isUserLoggedIn');

      // Check if the value is 'true'
      if (val === "true" && val !== null) {
        // If the user is already logged in and trying to access the login page, redirect to /expenses
        if (url === "/login") {
          return this.router.parseUrl('/expenses');
        } else {
          return true; // User is allowed to proceed
        }
      } else {
        // If the user is not logged in, redirect to /login
        return this.router.parseUrl('/login');
      }
    }

    // In case sessionStorage isn't available (for SSR)
    return this.router.parseUrl('/login');
  }
}

```

Here,

- **checkLogin** will check whether the localStorage has the user information and if it is available, then it returns true.
- If the user is logged in and goes to login page, it will redirect the user to expenses page
- If the user is not logged in, then the user will be redirected to login page.

Step 14: Let's add a new component in our application. User will be redirected to this page on successful login.

ng generate component ExpenseEntryList

The output is as follows –

CREATE src/app/expense-entry-list/expense-entry-list.component.html (34 bytes)

CREATE src/app/expense-entry-list/expense-entry-list.component.spec.ts (687 bytes)

CREATE src/app/expense-entry-list/expense-entry-list.component.ts (292 bytes)

CREATE src/app/expense-entry-list/expense-entry-list.component.css (0 bytes)

Here, the command creates the **ExpenseEntryList** Component and add the necessary code by default.

Step 15: Create a **ExpenseEntry** interface within **src/app/expense-entry-list.component.ts** file. Then, add a method named **getExpenseEntries()** to return list of expense entry (mock items) in **ExpenseEntryListComponent**. Also, declare a local variable, **expenseEntries** and load the mock list of expense entries.

```
import { CommonModule } from '@angular/common';
```

```
import { Component } from '@angular/core';
```

```
export interface ExpenseEntry {
```

```
  id: number;
```

```
  item: string;
```

```
  amount: number;
```

```
  category: string;
```

```
  location: string;
```

```
  spendOn: Date;
```

```
  createdOn: Date;
```

```
}
```

```
@Component({
```

```

selector: 'app-expense-entry-list',

standalone: true,

imports: [CommonModule],

templateUrl: './expense-entry-list.component.html',

styleUrl: './expense-entry-list.component.css'
})

export class ExpenseEntryListComponent {

  getExpenseEntries() : ExpenseEntry[] {

    let mockExpenseEntries : ExpenseEntry[] = [

      { id: 1,

        item: "Pizza",

        amount: Math.floor((Math.random() * 10) + 1),

        category: "Food",

        location: "Mcdonald",

        spendOn: new Date(2020, 4, Math.floor((Math.random() * 30) + 1), 10, 10, 10),

        createdOn: new Date(2020, 4, Math.floor((Math.random() * 30) + 1), 10, 10, 10) },

      { id: 1,

        item: "Pizza",

        amount: Math.floor((Math.random() * 10) + 1),

        category: "Food",

        location: "KFC",

        spendOn: new Date(2020, 4, Math.floor((Math.random() * 30) + 1), 10, 10, 10),

        createdOn: new Date(2020, 4, Math.floor((Math.random() * 30) + 1), 10, 10, 10) },

      { id: 1,

        item: "Pizza",

        amount: Math.floor((Math.random() * 10) + 1),

        category: "Food",

        location: "Mcdonald",

        spendOn: new Date(2020, 4, Math.floor((Math.random() * 30) + 1), 10, 10, 10),

        createdOn: new Date(2020, 4, Math.floor((Math.random() * 30) + 1), 10, 10, 10) },

      { id: 1,

```

```

    item: "Pizza",
    amount: Math.floor((Math.random() * 10) + 1),
    category: "Food",
    location: "KFC",
    spendOn: new Date(2020, 4, Math.floor((Math.random() * 30) + 1), 10, 10, 10),
    createdOn: new Date(2020, 4, Math.floor((Math.random() * 30) + 1), 10, 10, 10) },
    { id: 1,
      item: "Pizza",
      amount: Math.floor((Math.random() * 10) + 1),
      category: "Food",
      location: "KFC",
      spendOn: new Date(2020, 4, Math.floor((Math.random() * 30) + 1), 10, 10, 10),
      createdOn: new Date(2020, 4, Math.floor((Math.random() * 30) + 1), 10, 10, 10)
    },
  ];
  return mockExpenseEntries;
}

title: string = "";
expenseEntries!: ExpenseEntry[];

constructor() {}

ngOnInit() {
  this.title = "Expense Entry List";
  this.expenseEntries = this.getExpenseEntries();
}
}

```

Step 16: Open the template file, **src/app/expense-entry-list/expense-entry-list.component.html** and show the mock entries in a table.

```

<!-- Page Content -->

<div class="container">

  <div class="row">

    <div class="col-lg-12 text-center" style="padding-top: 20px;">

```



```
<div class="container" style="padding-left: 0px; padding-right: 0px;">
```

```
  <div class="row">
```

```
    <div class="col-sm" style="text-align: left;">
```

```
      {{ title }}
```

```
    </div>
```

```
    <div class="col-sm" style="text-align: right;">
```

```
      <button type="button" class="btn btn-primary">Edit</button>
```

```
    </div>
```

```
  </div>
```

```
</div>
```

```
<div class="container box" style="margin-top: 10px;">
```

```
  <table class="table table-striped">
```

```
    <thead>
```

```
      <tr>
```

```
        <th>Item</th>
```

```
        <th>Amount</th>
```

```
        <th>Category</th>
```

```
        <th>Location</th>
```

```
        <th>Spent On</th>
```

```
      </tr>
```

```
    </thead>
```

```
    <tbody>
```

```
      <tr *ngFor="let entry of expenseEntries">
```

```
        <th scope="row">{{ entry.item }}</th>
```

```
        <th>{{ entry.amount }}</th>
```

```
        <td>{{ entry.category }}</td>
```

```
        <td>{{ entry.location }}</td>
```

```
        <td>{{ entry.spendOn | date: 'short' }}</td>
```

```
      </tr>
```

```
    </tbody>
```

```
</table>
```

```

        </div>
    </div>
</div>
</div>

```

Here,

- Used bootstrap table. **table** and **table-striped** will style the table according to Bootstrap style standard.
- Used **ngFor** to loop over the **expenseEntries** and generate table rows.

Step 17: Open **src/app/app.routes.ts** and update below code –

```

import { Routes } from '@angular/router';

import { LoginComponent } from './login/login.component';
import { LogoutComponent } from './logout/logout.component';
import { authenticateGuard } from './authenticate.guard';
import { ExpenseEntryListComponent } from './expense-entry-list/expense-entry-list.component';

export const routes: Routes = [

    { path: 'login', component: LoginComponent },
    { path: 'logout', component: LogoutComponent },
    { path: 'expenses', component: ExpenseEntryListComponent, canActivate: [authenticateGuard]},
    {path: ' ', redirectTo: '/login', pathMatch: 'full'}

];

```

Here,

- Imported LoginComponent and LogoutComponent.
- Imported authenticateGuard.
- Created routes, login and logout to access LoginComponent and LogoutComponent.
- Add new option canActivate for ExpenseEntryListComponent.

Step 18: Open **AppComponent** template and add two login and logout link.

```

<nav class="navbar navbar-expand-lg navbar-dark bg-dark static-top">
  <div class="container">
    <a class="navbar-brand" href="#">{{ title }}</a>

```

```
<button class="navbar-toggler" type="button" data-toggle="collapse" data-
target="#navbarResponsive" aria-controls="navbarResponsive" aria-expanded="false" aria-
label="Toggle navigation">
```

```
<span class="navbar-toggler-icon"></span>
```

```
</button>
```

```
<div class="collapse navbar-collapse" id="navbarResponsive">
```

```
<ul class="navbar-nav ml-auto">
```

```
<li class="nav-item active">
```

```
<a class="nav-link" href="#">Home <span class="sr-only">(current)</span></a>
```

```
</li>
```

```
<li class="nav-item">
```

```
<a class="nav-link" href="#">Report</a>
```

```
</li>
```

```
<li class="nav-item">
```

```
<a class="nav-link" href="#">Add Expense</a>
```

```
</li>
```

```
<li class="nav-item">
```

```
<a class="nav-link" href="#">About</a>
```

```
</li>
```

```
<li class="nav-item">
```

```
<div *ngIf="isUserLoggedIn; else isLogOut">
```

```
<a class="nav-link" routerLink="/logout">Logout</a>
```

```
</div>
```

```
<ng-template #isLogOut>
```

```
<a class="nav-link" routerLink="/login">Login</a>
```

```
</ng-template>
```

```
</li>
```

```
</ul>
```

```
</div>
```

```
</div>
```

```
</nav>
```

```
<router-outlet></router-outlet>
```

Step 19: Open **AppComponent** and update below code –

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { ExpenseEntryListComponent } from './expense-entry-list/expense-entry-list.component';
import { LoginComponent } from './login/login.component';
import { LogoutComponent } from './logout/logout.component';
import { AuthService } from './Services/auth.service';
import { CommonModule } from '@angular/common';
```

```
@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, ExpenseEntryListComponent, LoginComponent, LogoutComponent,
CommonModule],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'Angular Authentication';
  isUserLoggedIn: any = false;
  router: any;

  constructor(private authService: AuthService) {}

  ngOnInit() {
    // Subscribe to the isUserLoggedIn observable from AuthService
    this.authService.isUserLoggedIn$.subscribe(status => {
      this.isUserLoggedIn = status;
      console.log("Is User Logged In: ", this.isUserLoggedIn);
    });
  }
}
```

```

logout(): void {
    // Trigger logout in AuthService
    this.authService.logout();

    // Redirect to the homepage after logout
    this.router.navigate(['/']);
}
}

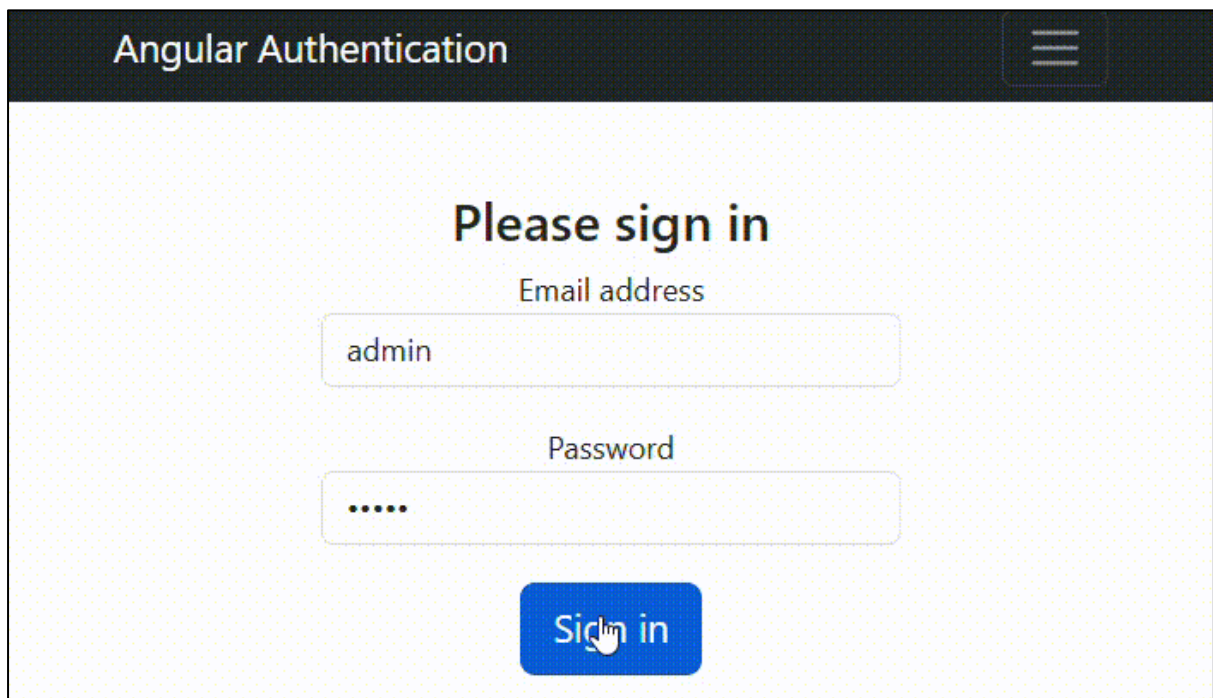
```

Here, we have added the logic to identify the user status so that we can show login/logout functionality.

Step 20: Start the application using following command –

ng serve

Enter admin as username and password and then, click submit. The application process the login and redirects the user to expense list page as shown below –

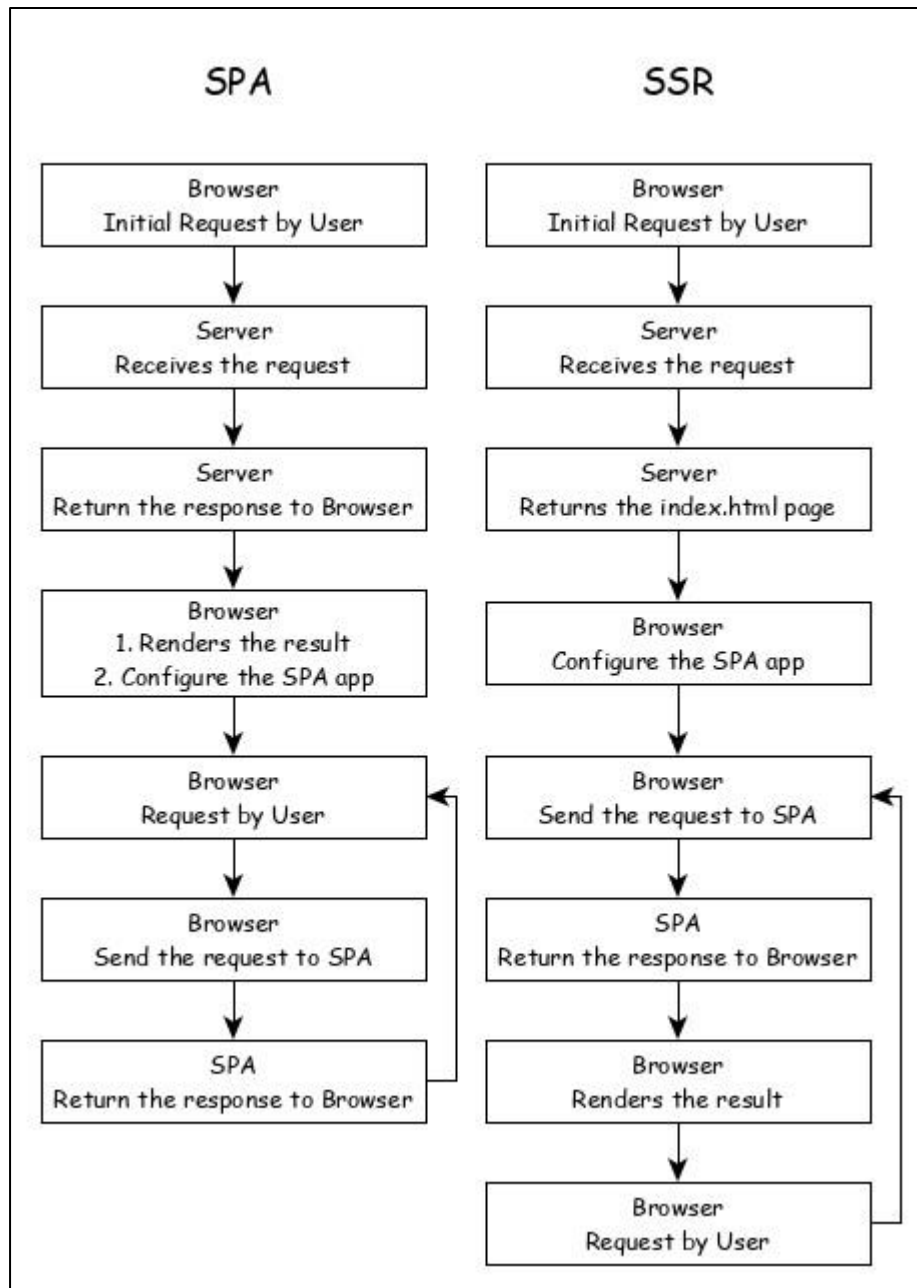


Finally, you can click logout and exit the application.

Angular - Server Side Rendering

Server side Rendering (SSR) is a modern technique to convert a Single Page Application (SPA) running in the browser into a server based application. Usually, in SPA, the server returns a simple index.html file with the reference to the JavaScript based SPA app. The SPA app take over from there, configure the entire application, process the request and then send the final response.

But in SSR supported application, the server as well do all the necessary configuration and then send the final response to the browser. The browser renders the response and start the SPA app. SPA app takeover from there and further request are diverted to SPA app. The flow of SPA and SSR is as shown in below diagram.



Converting a SPA application to SSR provides certain advantages and they are as follows –

- **Speed:** First request is relatively fast. One of the main drawback of SPA is slow initial rendering. Once the application is rendered, SPA app is quite fast. SSR fixes the initial rendering issue.
- **SEO Friendly:** Enables the site to be SEO friendly. Another main disadvantage of SPA is not able to crawled by web crawler for the purpose of SEO. SSR fixes the issue.

Angular Universal

To enable SSR in Angular, Angular should be able to rendered in the server. To make it happen, Angular provides a special technology called Angular Universal. It is quite new technology and it is continuously evolving. Angular Universal knows how to render Angular application in the server. We can upgrade our application to Angular Universal to support SSR.

Angular - Paginator

What is Paginator?

A **paginator** is a user interface component that allows users to navigate through a large set of data by dividing it into multiple pages. It provides controls for moving between pages, such as next, previous, first, and last buttons, along with an indication of the current page number.

You can use paginator in your Angular application to show data tables, search results, and list views. It shows the data by breaking it into smaller chunks. It also reduces loading times and improves performance. Paginator makes your application user-friendly as it provides an organized way for users to navigate through data.

The paginator appears at the bottom of a data table, search result or list. It has buttons to move to the next or previous page, as well as buttons to jump to the first or last page. Additionally, it displays the current page number and the total number of pages.

Creating Paginator in Angular

To create a paginator in an Angular application, use its Material UI component library. It is developed by Google and follows its Material Design principles. It has a rich set of UI components, such as buttons, forms, dialogs, and more. With minimal code, you can add these ready-to-use components to your application.

The paginator can be added by following the two steps given below –

- First, import the **MatPaginatorModule**.
- Once imported, add the **<mat-paginator>** element to your template and configure it with properties like **length**, **pageSize**, and **pageSizeOptions**.

Example

The following example illustrates how to add a paginator to your Angular application.

Step 1: Create a new Angular project. Give name of your choice, we are naming it **paginator-app**.

```
ng new paginator-app
```

Step 2: Now, navigate to the newly created app using the below command –

```
cd paginator-app
```

Step 3: Use the command given below to add Angular Material to your application –

```
ng add @angular/material
```

While adding the Angular Material, the CLI will ask for different configuration. Choose the default options for each configuration.

Step 4: Open **app.component.html** and add the code given below –

```
<mat-paginator  
  [length]="totalItems"  
  [pageSize]="pageSize"
```



```

[pageSizeOptions]="[5, 10]"
(page)="onPageChange($event)">
</mat-paginator>

<table mat-table [dataSource]="data">
  <ng-container matColumnDef="item">
    <th mat-header-cell *matHeaderCellDef> Item </th>
    <td mat-cell *matCellDef="let element"> {{element}} </td>
  </ng-container>

  <tr mat-header-row *matHeaderRowDef="['item']"></tr>
  <tr mat-row *matRowDef="let row; columns: ['item']"></tr>
</table>

<router-outlet />

```

Step 5: Open **app.component.ts** and add the following code –

```

import { CommonModule } from '@angular/common';
import { Component, OnInit } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { MatPaginatorModule } from '@angular/material/paginator';
import { MatTableModule } from '@angular/material/table';
import { PageEvent } from '@angular/material/paginator';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, CommonModule, MatPaginatorModule, MatTableModule],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})

export class AppComponent implements OnInit {
  totalItems = 10; // Total number of items
  pageSize = 5; // Number of items per page

```

```
data: any[] = [];  
currentPage = 0;
```

```
ngOnInit() {  
  this.loadData();  
}
```

```
loadData() {  
  const start = this.currentPage * this.pageSize;  
  this.data = Array.from({ length: this.pageSize }, (_, i) => `Item ${start + i + 1}`);  
}
```

```
onPageChange(event: PageEvent) {  
  this.pageSize = event.pageSize;  
  this.currentPage = event.pageIndex;  
  this.loadData();  
}  
}
```

Step 6: Open **app.component.css** and give margin to the paginator so that it looks good –

```
mat-paginator {  
  margin-top: 20px;  
}
```

Step 7: Finally, run the application using the below command –

ng serve

On running, the paginator will be displayed as shown below –

Items per page: 5 1 - 5 of 10 < >

Item
Item 1
Item 2
Item 3
Item 4
Item 5

Angular - Datepicker

What is Datepicker?

A **datepicker** is a user interface component that allows users to select a date from a calendar-like visual representation. It has an input field clicking on which displays a calendar where users can navigate and choose a specific date. This UI component helps you to get accurate date without manual typing.

In booking and event scheduling forms datepickers are used for date input. In angular, you can configure additional functionalities in datepickers, such as date ranges and specific date formats.

A datepicker looks similar to an input field with a calendar icon. When the user clicks on the input field or the calendar icon, a calendar overlay appears. Users can then navigate through months and years to select a desired date. Once a date is chosen, it populates the input field, and the calendar overlay closes.

Creating Datepicker in Angular

For creating a datepicker, we are going to use Angular Material. It is a UI component library developed by Google that follows the principles of Material Design. It provides a rich set of reusable UI components such as buttons, forms, and dialog boxes.

Angular Material has a built-in responsive design feature so that the website created using this library will adjust its size as per the device size. This library is specially built for Angular applications.

To use the datepicker component in an Angular application, the steps are as follows –

- Import the **MatDatepickerModule** and **MatNativeDateModule**.
- Now, add the **<mat-datepicker>** element to your template and bind it to an input field.

Example

In the following example, we are showing how to use datepicker in an Angular application.

Step 1: Create a new Angular project. Give name of your choice, we are naming it **datepicker-app**.

```
ng new datepicker-app
```

Step 2: Now, navigate to the newly created app using the below command –

```
cd datepicker-app
```

Step 3: Use the command given below to add Angular Material to your **datepicker-app** application –

```
ng add @angular/material
```

While adding the Angular Material, the CLI will show different configurations. Choose the default options for each configuration.

Step 4: Open **app.component.html** and add the code given below –

```
<div>

  <h3>{{ title }}</h3>

  <mat-form-field appearance="fill">

    <mat-label>Choose a date</mat-label>

    <input matInput [matDatepicker]="picker" [(ngModel)]="selectedDate" />

    <mat-hint>MM/DD/YYYY</mat-hint>

    <mat-datepicker-toggle matIconSuffix [for]="picker"></mat-datepicker-toggle>

    <mat-datepicker #picker></mat-datepicker>

  </mat-form-field>

  <p *ngIf="selectedDate">Selected Date: {{ selectedDate | date }}</p>

</div>

<router-outlet />
```

Where,

- **mat-form-field** defines the appearance of the form field.
- **matInput** creates an input field with Material Design styling.
- **[matDatepicker]="picker"** binds input field with the mat-datepicker component.

Step 5: Open **app.component.ts** and add the following code –

```
import { Component, ChangeDetectionStrategy } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { MatDatepickerModule } from '@angular/material/datepicker';
import { MatFormFieldModule } from '@angular/material/form-field';
import { MatInputModule } from '@angular/material/input';
```

```

import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { provideNativeDateAdapter } from '@angular/material/core';

@Component({
  selector: 'app-root',
  standalone: true,
  providers: [provideNativeDateAdapter()],
  imports: [RouterOutlet, MatDatepickerModule, MatFormFieldModule,
    MatInputModule, CommonModule, FormsModule],
  changeDetection: ChangeDetectionStrategy.OnPush,
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'Datepicker Application';
  selectedDate: Date | null = null;
}

```

Step 6: Open **app.component.css** and add the following CSS –

```

div {
  margin-top: 20px;
  display: flex;
  align-items: center;
  justify-content: center;
  flex-direction: column;
}

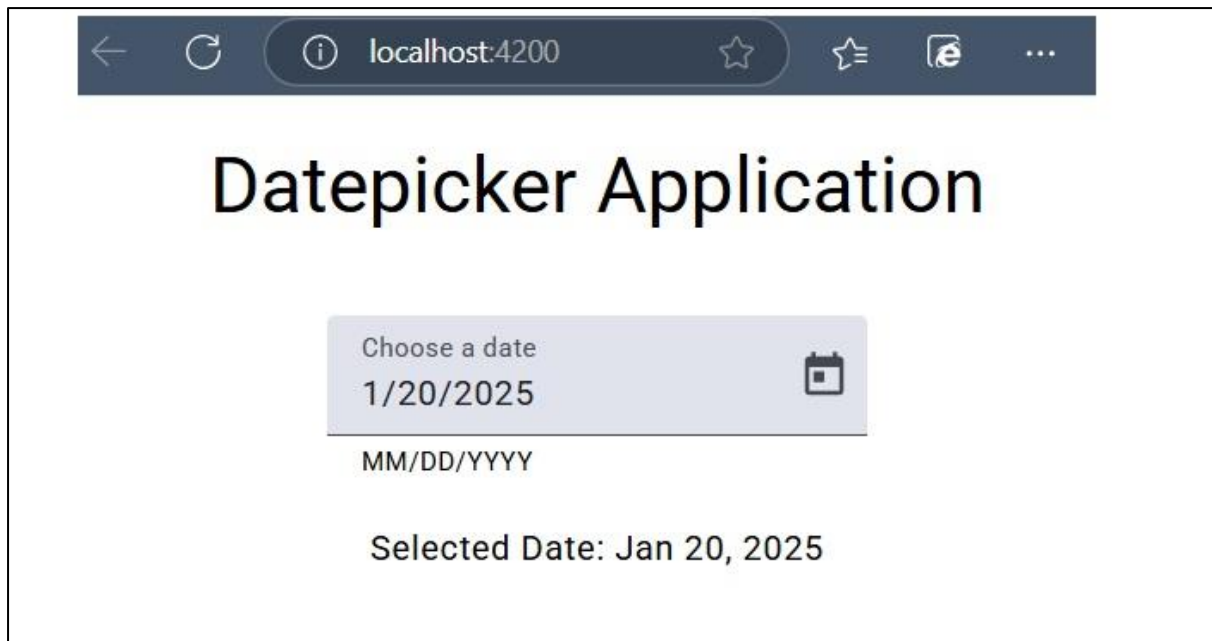
mat-form-field {
  width: 50%;
  margin-top: 20px;
  margin-bottom: 20px;
}

```

Step 7: Now, run the application using the command given below –

ng serve

On running, following output will be displayed on the screen –



Angular - Select Dropdown

What is Select Dropdown?

A **select dropdown** (also called as a dropdown menu) is a user interface element that allows users to choose one option from a list of predefined options. Its usage can be seen in forms where users need to make a selection from a given set of choices.

In web applications built using frameworks like Angular, select dropdowns are widely used to select a single option from a long list of options, such as choosing a country or selecting a category. They hide the options until they are needed. They also prevent users from entering invalid data as users can make choices only from the predefined options.

When you use a select dropdown in your Angular application, it will appear as a compact box with default value. If the user clicks on it, it expands to show all available options. Each option is listed in a vertical stack, and the user can click on an option to select it. Once an option is selected, the dropdown collapses back to its compact form and displays the chosen option.

Creating Select Dropdown in Angular

For creating a select dropdown, we are going to use Angular Material. It is a UI component library developed by Google that follows the principles of Material Design. It provides a rich set of reusable UI components such as buttons, forms, and dialog boxes. With minimal code, you can add these ready-to-use components to your application.

To use the select dropdown component in an Angular application, follow the steps given below –

- First, import the **MatSelectModule** and **MatFormFieldModule**.

- Then, add the **<mat-form-field>** and **<mat-select>** elements to the template. This **mat-select** component provides a dropdown menu with a list of options. Use the two-way data binding to bind the selected value to a property in the component class.

Example

In this example, we will show how to use select dropdown in an Angular application.

Step 1: Create a new Angular project. Give name of your choice, we are naming it **dropdown-app**.

ng new dropdown-app

Step 2: Now, navigate to the newly created app using the below command –

cd dropdown-app

Step 3: Use the command given below to add Angular Material to your **dropdown-app** application –

ng add @angular/material

While adding the Angular Material, the CLI will different configuration. Choose the default options for each configuration.

Step 4: Open **app.component.html** and add the code given below –

```
<div>
  <h3> {{title}} </h3>
  <mat-form-field appearance="fill">
    <mat-label>Choose Color Name</mat-label>
    <mat-select [(value)]="selectedValue">
      <mat-option *ngFor="let option of options" [value]="option.value">
        {{ option.viewValue }}
      </mat-option>
    </mat-select>
  </mat-form-field>
</div>
<router-outlet />
```

Step 5: Open **app.component.ts** and add the following code –

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { FormsModule } from '@angular/forms';
import { MatInputModule } from '@angular/material/input';
import { MatSelectModule } from '@angular/material/select';
```

```
import {MatFormFieldModule} from '@angular/material/form-field';
```

```
import { CommonModule } from '@angular/common';
```

```
@Component({
```

```
  selector: 'app-root',
```

```
  standalone: true,
```

```
  imports: [RouterOutlet, MatFormFieldModule, MatSelectModule, MatInputModule, FormsModule,
    CommonModule],
```

```
  templateUrl: './app.component.html',
```

```
  styleUrls: ['./app.component.css']
```

```
})
```

```
export class AppComponent {
```

```
  title = 'Dropdown Example';
```

```
  selectedValue: string = "";
```

```
  options = [
```

```
    { value: 'option1', viewValue: 'Red' },
```

```
    { value: 'option2', viewValue: 'Saffron' },
```

```
    { value: 'option3', viewValue: 'Violet' },
```

```
    { value: 'option4', viewValue: 'Gray' }
```

```
  ];
```

```
}
```

Step 6: Open **app.component.css** and add some margin to the component so that it looks good –

```
div{
```

```
  margin-top: 25px;
```

```
  margin-left: 25px;
```

```
}
```

Step 7: Finally, run the application using the below command –

ng serve

On running, following output will be displayed on the screen –

Dropdown Example

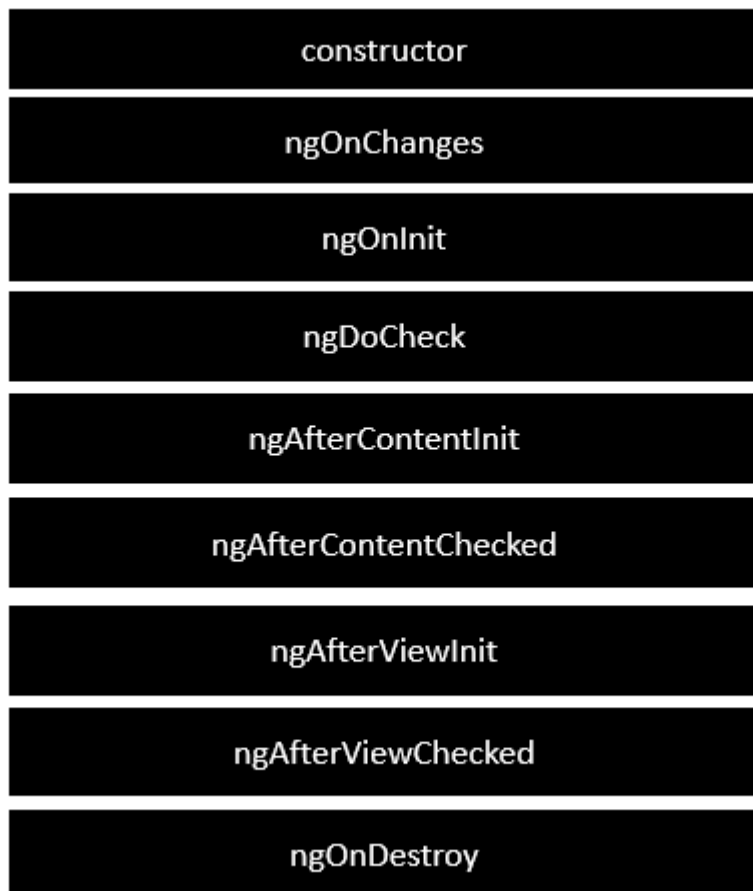
Choose Color Name ▼



Angular - Lifecycle Hooks

Angular application goes through an entire set of processes or has a lifecycle right from its initiation to the end of the application.

The following diagram shows the entire processes in the lifecycle of the Angular 2 application.



Following is a description of each lifecycle hook.

- **ngOnChanges** – When the value of a data bound property changes, then this method is called.
- **ngOnInit** – This is called whenever the initialization of the directive/component after Angular first displays the data-bound properties happens.
- **ngDoCheck** – This is for the detection and to act on changes that Angular can't or won't detect on its own.
- **ngAfterContentInit** – This is called in response after Angular projects external content into the component's view.
- **ngAfterContentChecked** – This is called in response after Angular checks the content projected into the component.
- **ngAfterViewInit** – This is called in response after Angular initializes the component's views and child views.
- **ngAfterViewChecked** – This is called in response after Angular checks the component's views and child views.
- **ngOnDestroy** – This is the cleanup phase just before Angular destroys the directive/component.

Following is an example of implementing one lifecycle hook. In the **app.component.ts** file, place the following code.

```
import {  
  Component  
} from '@angular/core';  
  
@Component ({  
  selector: 'my-app',  
  template: '<div> {{values}} </div> '  
})  
  
export class AppComponent {  
  values = '';  
  ngOnInit() {  
    this.values = "Hello";  
  }  
}
```

In the above program, we are calling the **ngOnInit** lifecycle hook to specifically mention that the value of the **this.values** parameter should be set to Hello.

Once you save all the code changes and refresh the browser, you will get the following output.



Angular - Configuration

This chapter will discuss the **configuration** of an Angular application, including important files, their specific role and usage in the Angular project, advantages, etc.

Angular Configuration

The Angular configuration is a set of instructions that defines how the Angular application is built and developed. It includes various information about the Angular application such as **project files**, **tools**, **environments**, and some other dependencies.

Important Configuration files in Angular

Here is a list of important **configuration files** that are automatically created when an Angular application is built using the [Angular CLI](#). These files are important for running, deploying, and testing the application.

- [angular.json](#)
- [package.json](#)
- [main.ts](#)
- [tsconfig.json](#)

The angular.json Configuration File

In an Angular application, the **angular.json** file is a key configuration file that defines and manages all the settings for the Angular CLI (Command Line Interface). This file is necessary for customizing various aspects of your Angular project.

The **angular.json** file will look like:

```
"projects": {  
  "my-app": {  
    "architect": {  
      "build": {
```

```

"options": {
  "outputPath": "dist/my-app",
  "index": "src/index.html",
  "main": "src/main.ts",
  "polyfills": "src/polyfills.ts",
  "tsConfig": "tsconfig.app.json",
  "assets": [
    "src/favicon.ico",
    "src/assets"
  ],
  "styles": [
    "src/styles.css"
  ],
  "scripts": []
}
}
}
}
}
}
}

```

This file contains key aspects of your Angular project, such as the "application name", "index.html", "main.ts", "assets", "styles", "scripts", "environment configurations", "build" and "test" options, and other settings important for your Angular application's build and deployment process.

Important Points of the angular.json File

Here are a few important aspects of the angular.json file that you should know:

- **my-app:** This is the **name** of your Angular project or application.
- **index.html:** The **main HTML** file that serves as the **entry point** for your application. It usually contains the "root" HTML element where the Angular app will be **bootstrapped**.
- **main.ts:** The main TypeScript file that acts as the entry point for the Angular application. It is responsible for bootstrapping the **root module or component** of the application.
- **assets:** A directory that contains static assets such as images, fonts, and other files that are needed by the application.
- **styles:** An array of global stylesheets that are included in the application. These stylesheets can be CSS or SCSS files and are applied to the entire application.

- **scripts:** An array of global scripts that are included in the application. These can be JavaScript files that need to be loaded before the Angular app starts.

The package.json Configuration File

In an Angular application, the **package.json** file serves as a fundamental part of managing the project's **dependencies** and **scripts**. It contains "metadata" about the project, including the project "name", "version", "scripts for starting", "building", "serving", and "watching the application", as well as the "dependencies" and "devDependencies".

The **package.json** file data will look like this:

```
{
  "name": "my-crud-app",
  "version": "0.0.0",
  "scripts": {
    "ng": "ng",
    "start": "ng serve",
    "build": "ng build",
    "watch": "ng build --watch --configuration development",
    "test": "ng test",
    "serve:ssr:my-crud-app": "node dist/my-crud-app/server/server.mjs"
  },
  "private": true,
  "dependencies": {
    "@angular/animations": "^17.0.0",
    "@angular/common": "^17.0.0",
    "@angular/compiler": "^17.0.0",
    "@angular/core": "^17.0.0",
    "@angular/forms": "^17.0.0",
    "@angular/platform-browser": "^17.0.0",
    "@angular/platform-browser-dynamic": "^17.0.0",
    "@angular/platform-server": "^17.0.0",
    "@angular/router": "^17.0.0",
    "@angular/ssr": "^17.0.8",
    "express": "^4.18.2",
```

```

"json-server": "^1.0.0-beta.3",
"rxjs": "~7.8.0",
"tslib": "^2.3.0",
"zone.js": "~0.14.2"
},
"devDependencies": {
"@angular-devkit/build-angular": "^17.0.8",
"@angular/cli": "^17.0.8",
"@angular/compiler-cli": "^17.0.0",
"@types/express": "^4.17.17",
"@types/jasmine": "~5.1.0",
"@types/node": "^18.18.0",
"jasmine-core": "~5.1.0",
" karma": "~6.4.0",
" karma-chrome-launcher": "~3.2.0",
" karma-coverage": "~2.2.0",
" karma-jasmine": "~5.1.0",
" karma-jasmine-html-reporter": "~2.1.0",
"typescript": "~5.2.2"
}
}

```

Important Points of the package.json File

Here are a few important aspects of the package.json file that you should know:

- **name & version:** The **name** and the **version** of your application, which is useful for identifying, and tracking updates and dependencies.
- **scripts:** It contains **commands** for common tasks such as "building", "serving", and "testing" the Angular project. These commands are executed using **npm run <command>**
- **dependencies:** It is a list of Angular **packages** and other **libraries** your project depends on. These are important for the application to run and are installed when you run the **npm install** command.
- **devDependencies:** It is a list of **packages** that are needed only for "development purposes", such as "testing tools" and "build tools".

The main.ts Configuration File

In an Angular application, the **main.ts** file serves as the **entry point**. It is responsible for "bootstrapping" the root module or component. When you run the **ng serve** command, it compiles the application and looks for the "main.ts" file first to initiate the "bootstrapping process".

If the **main.ts** file is missing or has issues, your application will fail to start, and you will face errors.

The **main.ts** file data will look like this:

```
import { bootstrapApplication } from '@angular/platform-browser';  
  
import { appConfig } from './app/app.config';  
  
import { AppComponent } from './app/app.component';
```

```
bootstrapApplication(AppComponent, appConfig)
```

```
.catch((err) => console.error(err));
```

Here,

- **AppComponent:** It is the main component that acts as the **root component** of your application.

The tsconfig.json Configuration File

A given Angular workspace contains several [TypeScript](#) configuration files. At the root **tsconfig.json** file specifies the base TypeScript and Angular compiler options that all projects in the workspace inherit.

Initially, the **tsconfig.json** file data will look like this:

```
{  
  "extends": "./tsconfig.json",  
  "compilerOptions": {  
    "outDir": "./out-tsc/app",  
    "types": [  
      "node"  
    ]  
  },  
  "files": [  
    "src/main.ts",  
    "src/main.server.ts",  
    "server.ts"  
  ],  
}
```

```
"include": [  
  "src/**/*.d.ts"  
]  
}
```

Conclusion

In conclusion, Angular configuration plays an important role in ensuring that your Angular application runs smoothly without any lag or errors. It involves setting up various aspects of the application, such as "environment settings", "build configurations", "module imports", etc.

Angular - Decorators & Metadata

Decorators are special functions used to add metadata, modify behavior, or add some additional functionalities to classes, methods, properties, or parameters in programming languages like [TypeScript](#) or [JavaScript](#).

In [Angular](#), each decorator has a base configuration with some default values and you can change or update it according to the project's need. It is defined using the **@ symbol** followed by a function name. This symbol helps Angular to recognize decorators.

Decorators in Angular

Angular provides the following decorators –

- [@Component decorator](#)
- [@Injectable decorator](#)
- [@NgModule decorator](#)
- [@Directive decorator](#)
- [@Input decorator](#)
- [@Output decorator](#)
- [@Pipe Decorator](#)

The @Component Decorator

The **@Component** decorator is used to define a [component](#) in Angular. A component generates a section of web page called View and this decorator helps Angular understand the metadata related to the component.

Example

The following example shows @component decorator in an angular component.

```
import { Component } from '@angular/core';  
import { CommonModule } from '@angular/common';
```

```
@Component({
```



```

    selector: 'app-my-component',
    standalone: true,
    imports: [CommonModule],
    templateUrl: './my-component.component.html',
    styleUrls: ['./my-component.component.css']
  })
  export class MyComponent {
    // your code
  }

```

The metadata included in the above example is `selector`, `imports`, `templateUrl`, and `styleUrl`. They define how the component should be displayed in the DOM.

The @Injectable Decorator

If a TypeScript class in Angular is decorated by **@Injectable decorator**, Angular will consider it as a service that can be injected into other classes.

Example

Let's see an example of @Injectable decorator.

```
import { Injectable } from '@angular/core';
```

```

@Injectable({
  providedIn: 'root'
})
export class MyService {
  // your service code
}

```

By marking a service with @Injectable, Angular knows to create and inject the service wherever it's needed in the application.

The @NgModule Decorator

The **@NgModule decorator** is used to define an Angular module. A module is a collection of related components, services, pipes, and directives that are bundled together.

Example

Here is an example of @NgModule decorator –

```
@NgModule({
  declarations: [MyComponent],
  imports: [CommonModule],
  providers: [MyService]
})
export class MyModule {}
```

The **@NgModule** decorator tells Angular about which components, directives, and pipes are part of the module and which other modules are imported.

The @Directive Decorator

The **@Directive decorator** is used to define a custom directive, which can modify the behavior or appearance of elements in the DOM.

Example

The example given below shows a custom directive that modifies the background color of any element it's applied to.

```
import { Directive, ElementRef } from '@angular/core';
import { CommonModule } from '@angular/common';

@Directive({
  selector: '[appHighlight]',
  standalone: true,
  imports: [CommonModule]
})
export class HighlightDirective {
  constructor(private el: ElementRef) {
    this.el.nativeElement.style.backgroundColor = 'yellow';
  }
}
```

The @Input Decorator

The **@Input decorator** is used in components to define inputs for property binding. It helps data to be passed into the component.

Example

In the following, we will see how to use @Input decorator within a component.

```
@Component({
```

```

    selector: 'app-child',
    template: `<div>{{childData}}</div>`
  })
export class ChildComponent {
  @Input() childData: string;
}

```

The @Output Decorator

Child component can send the data to parent component through the @Output decorator. This decorator is actually an event emitter that passes the data (output) along with event.

Example

The following example shows the use of @Output decorator.

```

import { Component, Input, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-child',
  template: `
    <div>{{childData}}</div>
    <button (click)="sendData()">Send Data</button>
  `
})
export class ChildComponent {
  @Input() childData: string;
  @Output() dataEmitter = new EventEmitter<string>();

  sendData() {
    this.dataEmitter.emit(this.childData);
  }
}

```

The @Pipe Decorator

The **@Pipe decorator** in Angular is used to define custom pipes, which transform data in templates.

Example

Here is an example of @Pipe decorator.

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({ name: 'capitalize' })

export class CapitalizePipe implements PipeTransform {

  transform(value: string): string {

    return value.charAt(0).toUpperCase() + value.slice(1).toLowerCase();

  }

}
```

This CapitalizePipe transforms the first letter of a string to uppercase and the rest to lowercase.

What Is Metadata in Angular?

In Angular, **Metadata** is the information that is attached to classes and other elements to define their behavior. It is defined via decorators, as we saw earlier. When Angular processes a component or service, it reads the metadata to configure the element's behavior.

Example

Let's see the metadata mentioned inside an Angular @Component decorator –

```
@Component({

  selector: 'app-my-component',

  standalone: true,

  imports: [CommonModule],

  templateUrl: './my-component.component.html',

  styleUrls: ['./my-component.component.css'],

})
```

Here,

- **selector** specifies the HTML tag to use for this component.
- **templateUrl** points to the location of the HTML template file.
- **styleUrls** specifies the location of the CSS files that define the component's styling.

Angular - Error Handling

Errors are the unexpected issue or problem that occurs during the execution of a program. Incorrect syntax, invalid input, system failures or bug in API logic could be the possible reasons for these errors. These errors may affect the user experience in negative way, hence, it is necessary to handle them at compile time.

Error handling is the process of detecting and resolving these errors or exceptions before execution. Angular provides a number of ways to handle the errors that we are going to learn in this tutorial.

Handling Errors in Angular

There are multiple ways to handle errors in [Angular](#), which are given below –

- [Using ErrorHandler class](#)
- [Using HttpInterceptor interface](#)
- [Using try-catch blocks](#)
- [Using validation](#)

Using ErrorHandler Class

The **ErrorHandler** is a built-in class of the Angular **@angular/core** package. It provides a hook to catch errors globally. When something goes wrong, like any unhandled exception occurs anywhere in the application, this class catches that exception and prints the error messages on console.

Remember! the ErrorHandler class simply prints the error message so that developers can see what went wrong and fix it. However, you can replace this default behavior by creating a custom ErrorHandler.

Extend the **ErrorHandler** class and override its **handleError()** method as shown below to create a custom behavior of this class –

```
import { Injectable, ErrorHandler } from '@angular/core';
```

```
@Injectable({
  providedIn: 'root'
})
export class GlobalErrorHandler implements ErrorHandler {
  handleError(error: any): void {
    // Log the error to an external service or display a message
    console.error("An error occurred:", error);

    // Create any custom behavior
  }
}
```

Using HttpInterceptor Interface

The **HttpInterceptor** is an interface that is used for handling HTTP-specific errors, such as network issues, server errors, etc. It can intercept and modify HTTP requests or responses. Think of it as a service that can check and change the data going to and coming from a server.

To use an **HttpInterceptor**, create a class that implements the HttpInterceptor interface and define the **intercept()** method. This method takes an HTTP request and a handler, and it returns an observable of the HTTP event. Inside this method, you can modify the request or response as needed.

```
import { Injectable } from '@angular/core';

import { HttpEvent, HttpInterceptor, HttpHandler, HttpRequest } from '@angular/common/http';

import { Observable } from 'rxjs';

import { catchError } from 'rxjs/operators';

import { throwError } from 'rxjs';
```

```
@Injectable({
  providedIn: 'root'
})

export class ErrorInterceptor implements HttpInterceptor {

  intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {

    return next.handle(req).pipe(
      catchError((error) => {
        // Handle error
        console.error('HTTP Error:', error);

        return throwError(error);
      })
    );
  }
}
```

Using Try-Catch Blocks

In Angular, errors can occur during component lifecycle. To handle these errors, you can use the a try-catch block within the **ngOnInit()**, which is a [component lifecycle hook](#) defined by **OnInit** interface. It is important to implement this interface as it helps to check if component is properly initialized.

The **try block** is used to wrap the code that might throw an error during its execution. And, the **catch block** is used to handle any errors or exceptions that occur within the try block.

Here, you can see a component with try-catch block:

```
import { Component, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-example',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './example.component.html',
  styleUrls: ['./example.component.css']
})
export class ExampleComponent implements OnInit {
  ngOnInit() {
    try {
      // code that might throw an error
      throw new Error("Error message");
    } catch (error) {
      console.error("error caught", error);
    }
  }
}
```

Using Validation

Validation is used in [Angular Forms](#) to validate whether the user input is in the correct format or not before submission. The [reactive forms](#) and [template-driven forms](#) has built-in validators to handle validation errors.

With reactive forms, you can easily validate user input and display custom error messages as shown in the following code block –

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';
import { CommonModule } from '@angular/common';

@Component({
```

```
selector: 'app-registration',
standalone: true,
imports: [CommonModule],
templateUrl: './registration.component.html',
styleUrls: ['./registration.component.css']
})

export class RegistrationComponent {
  registrationForm: FormGroup;
  constructor(private fb: FormBuilder) {
    this.registrationForm = this.fb.group({
      username: ['', [Validators.required, Validators.minLength(4)]],
      email: ['', [Validators.required, Validators.email]],
    });
  }
  onSubmit() {
    if (this.registrationForm.invalid) {
      this.showValidationErrors();
      return;
    }
  }
  private showValidationErrors() {
    Object.keys(this.registrationForm.controls).forEach(control => {
      const formControl = this.registrationForm.get(control);
      if (formControl?.invalid) { console.error(`${control} is invalid`);
    }
  });
}
```


Performance Optimization

Performance in Angular can be improved at many levels: change detection, templates, data flows, network, and build configuration.

Change detection: OnPush strategy

By default, Angular checks all components in a tree on many events (clicks, HTTP responses, timers). This can be slow in large apps. Using **ChangeDetectionStrategy.OnPush** tells Angular to check a component only when:

- Its @Input() reference changes,
- An event inside it fires,
- You trigger it manually.

Example: invoice list component with OnPush

```
import { Component, Input, ChangeDetectionStrategy } from '@angular/core';
```

```
import { Invoice } from '../models/invoice.model';
```

```
@Component({  
  selector: 'app-invoice-list',  
  templateUrl: './invoice-list.component.html',  
  changeDetection: ChangeDetectionStrategy.OnPush  
})
```

```
export class InvoiceListComponent {  
  @Input() invoices: Invoice[] = [];  
}
```

```
<!-- invoice-list.component.html -->
```

```
<tr *ngFor="let inv of invoices; trackBy: trackById">
```

```
  <td>{{ inv.invoiceNumber }}</td>
```

```
  <td>{{ inv.customerName }}</td>
```

```
  <td>{{ inv.totalAmount | currency }}</td>
```

```
</tr>
```

```
trackById(index: number, item: Invoice) {
```

```
  return item.id;
```

```
}
```

```
trackById(index: number, item: Invoice) {  
  return item.id;  
}
```

- OnPush reduces unnecessary checks.
 - trackBy (explained more below) avoids recreating DOM nodes for unchanged rows.
-

***ngFor with trackBy**

Without trackBy, Angular destroys and recreates all DOM elements when the array changes, even if only one row changed. trackBy tells Angular how to identify items (by id) so only changed rows are updated.

Example: customer list

```
<tr *ngFor="let cust of customers; trackBy: trackByCustomerId">  
  <td>{{ cust.name }}</td>  
  <td>{{ cust.email }}</td>  
</tr>
```

```
trackByCustomerId(index: number, item: Customer) {  
  return item.id;  
}
```

Use trackBy on any large list: customers, products, invoices, stock ledger.

Pure pipes instead of heavy template logic

Complex calculations or formatting directly in templates are re-run on every change detection. Encapsulate them in pure pipes, which only recalculate when inputs change.

Example: tax calculation pipe

```
import { Pipe, PipeTransform } from '@angular/core';  
  
@Pipe({ name: 'taxTotal', pure: true })  
export class TaxTotalPipe implements PipeTransform {  
  transform(amount: number, taxRate: number): number {  
    return amount + (amount * taxRate) / 100;  
  }  
}
```

```
<td>{{ invoice.totalAmount | taxTotal:invoice.taxRate | currency }}</td>
```

Pure pipes are cached by Angular until input values change, reducing repeated work.

Avoid unnecessary async subscriptions in templates

Using many `| async` pipes directly in deep templates can recreate observables and trigger extra checks if misused. In many cases, subscribe once in the component and expose a simple property.

Better pattern:

```
invoices$: Observable<Invoice[]> = this.invoiceService.getInvoices();
```

```
<tr *ngFor="let inv of (invoices$ | async); trackBy: trackById">...</tr>
```

One `async` at the top-level list is fine; avoid nested `async` calls in every cell.

Lazy loading modules and routes

Load only the code needed for the current route instead of the whole app bundle at once. This reduces initial load time.

Example: lazy-load billing feature modules

```
const routes: Routes = [  
  {  
    path: 'customer',  
    loadChildren: () =>  
      import('./customer/customer.module').then(m => m.CustomerModule)  
  },  
  {  
    path: 'invoice',  
    loadChildren: () =>  
      import('./invoice/invoice.module').then(m => m.InvoiceModule)  
  }  
];
```

Users opening only the login or dashboard do not pay the cost of loading customer/product/invoice modules until needed.

Optimize images, CSS, and bundles

Angular's production build already does tree-shaking, minification, and bundling.

- Run `ng build --configuration production` (or `--prod` for older CLI) for optimized bundles.
- Use lazy-loaded feature modules to reduce main bundle size.

- Use shared modules only for genuinely shared components to avoid pulling big dependencies into all bundles.

For static resources (logos, product images), compress them and use correct formats; large images significantly hurt performance.

Security

Angular provides several built-in features to improve security, especially against XSS and unauthorized route access. Below are the key concepts you asked for, with clear examples you can adapt to your billing system.

DOM Sanitizer and XSS basics

Angular treats all template values as untrusted and automatically sanitizes them when writing to the DOM (e.g. in [href], [src], innerHTML). This blocks most cross-site scripting (XSS) attempts, like injected <script> tags or javascript: URLs.

Safe use of innerHTML with DomSanitizer

If you must render HTML from a trusted backend (e.g. invoice terms stored as HTML), you can use DomSanitizer. Angular will already sanitize normal bindings; only bypass in rare, audited cases.

ts

```
import { Component } from '@angular/core';
```

```
import { DomSanitizer, SafeHtml } from '@angular/platform-browser';
```

```
@Component({  
  selector: 'app-invoice-terms',  
  template: `<div [innerHTML]="safeHtml"></div>`  
})
```

```
export class InvoiceTermsComponent {
```

```
  safeHtml: SafeHtml;
```

```
  constructor(private sanitizer: DomSanitizer) {
```

```
    const htmlFromServer = '<p><strong>Payment due in 30 days.</strong></p>';
```

```
    this.safeHtml = this.sanitizer.bypassSecurityTrustHtml(htmlFromServer);
```

```
  }
```

```
}
```

Key rules:

- Prefer letting Angular sanitize automatically (just bind a string).
 - Only use `bypassSecurityTrustHtml/Url/ResourceUrl` for content you have fully validated or generated yourself.
 - Never pass raw user input into `bypassSecurityTrust...` functions.
-

XSS protection best practices

Even with Angular's automatic sanitization, you should:

- Avoid innerHTML whenever possible; prefer standard bindings and components.
- Avoid concatenating HTML strings from user input.
- Use property binding [href], [src], [style] instead of manually building HTML.
- Never disable Angular's default sanitization globally or in lint rules.

Example of safe URL binding:

xml

```
<a [href]="safeInvoiceUrl">Download Invoice</a>
```

ts

```
safeInvoiceUrl = 'https://your-api/invoices/123/pdf';
```

Angular sanitizes this URL and rejects dangerous protocols like javascript: automatically.

Route Guard (authentication)

Route guards control whether a user can navigate to a given route (e.g. prevent unauthenticated users from opening /invoice). The most common is CanActivate.

Example: basic AuthGuard

ts

```
// auth.guard.ts
```

```
import { Injectable } from '@angular/core';
```

```
import { CanActivate, Router, ActivatedRouteSnapshot, RouterStateSnapshot } from '@angular/router';
```

```
import { AuthService } from './auth.service';
```

```
@Injectable({ providedIn: 'root' })
```

```
export class AuthGuard implements CanActivate {
```

```
  constructor(private auth: AuthService, private router: Router) {}
```

```
  canActivate(route: ActivatedRouteSnapshot, state: RouterStateSnapshot): boolean {
```

```
    if (this.auth.isLoggedIn()) {
```

```
      return true;
```

```
    }
```

```

    this.router.navigate(['/login'], { queryParams: { returnUrl: state.url } });

    return false;
  }
}

```

Route configuration:

ts

```

const routes: Routes = [
  { path: 'invoice', canActivate: [AuthGuard], loadChildren: () =>
import('./invoice/invoice.module').then(m => m.InvoiceModule) },
];

```

- AuthService.isLoggedIn() can check a JWT in localStorage or an in-memory state.
- If not logged in, redirect to /login.

Role-based routes with guards

You can extend guards to check user roles (e.g. Admin, Clerk, Viewer) so only certain users can access routes like Tax Rate management or Stock Ledger.

1. AuthService with roles

ts

// auth.service.ts (simplified)

```
@Injectable({ providedIn: 'root' })
```

```
export class AuthService {
```

```
  // Example: store roles in JWT or localStorage
```

```
  hasRole(requiredRoles: string[]): boolean {
```

```
    const rolesString = localStorage.getItem('ROLES') || '[]';
```

```
    const userRoles: string[] = JSON.parse(rolesString);
```

```
    return requiredRoles.some(r => userRoles.includes(r));
```

```
  }
```

```
  isLoggedIn(): boolean {
```

```
    return !!localStorage.getItem('TOKEN');
```

```
  }
```

```
}
```

2. Role-based guard

ts

// role.guard.ts

```
import { Injectable } from '@angular/core';
```

```
import { CanActivate, ActivatedRouteSnapshot, Router } from '@angular/router';
```

```
import { AuthService } from './auth.service';
```

```
@Injectable({ providedIn: 'root' })
```

```
export class RoleGuard implements CanActivate {
```

```
  constructor(private auth: AuthService, private router: Router) {}
```

```
  canActivate(route: ActivatedRouteSnapshot): boolean {
```

```
    const requiredRoles = route.data['roles'] as string[];
```

```
    if (!this.auth.isLoggedIn() || !this.auth.hasRole(requiredRoles)) {
```

```
      this.router.navigate(['/unauthorized']);
```

```
      return false;
```

```
    }
```

```
    return true;
```

```
  }
```

```
}
```

3. Routes with role metadata

ts

```
const routes: Routes = [
```

```
{
```

```
  path: 'tax-rate',
```

```
  canActivate: [RoleGuard],
```

```
  data: { roles: ['Admin'] }, // only admins
```

```
  loadChildren: () => import('./tax-rate/tax-rate.module').then(m => m.TaxRateModule)
```

```
},
```

```
{
```



```
path: 'payment',
canActivate: [RoleGuard],
data: { roles: ['Admin', 'Cashier'] } // multiple allowed roles
}
];
```

Here:

- Route data holds allowed roles.
- RoleGuard checks login + roles and redirects unauthorized users.

How this fits your billing system

- DomSanitizer / XSS:
Use normal bindings for customer names, product descriptions, invoice details. Only use DomSanitizer if you must render trusted HTML (e.g. rich-text invoice notes) and never with raw user input.
- Route Guard:
Protect modules like /invoice, /stock-ledger, /payment so only authenticated users access them.
- Role-based routes:
 - Admin: full access, including Tax Rate and User Management.
 - Cashier: Payment, Invoice.
 - Viewer: read-only invoice/customer views.
Configure this by adding RoleGuard with data.roles on each route.

If you share one concrete route setup (e.g. your current app-routing.module.ts), a tailored guard and role configuration can be provided matching that structure.

RxJS

RxJS is a library for reactive programming using Observables—streams of data that emit over time (like events, HTTP responses, user input, timers). In Angular, RxJS powers HTTP requests, forms, routing, and all async operations. Mastering it unlocks powerful data flows for your billing system (real-time stock updates, filtered searches, concurrent API calls).

Core Concepts

Observable vs Promise

Observable	Promise
Lazy (runs only when subscribed)	Eager (runs immediately)
Can emit multiple values over time	Single value or error
Cancelable (unsubscribe)	Not cancelable
Supports operators (map, filter, etc.)	No operators

Example: HTTP vs timer

// HTTP (single value)

```
this.http.get('/customers').subscribe(customers => ...);
```

// Timer (multiple values)

```
interval(1000).subscribe(i => console.log(`Tick ${i}`)); // 0,1,2,3...
```

Creating Observables

1. HTTP (most common in Angular)

```
import { HttpClient } from '@angular/common/http';
```

```
import { Observable } from 'rxjs';
```

```
customers$: Observable<Customer[]> = this.http.get<Customer[]>('/api/customers');
```

2. fromEvent (DOM events)

```
import { fromEvent } from 'rxjs';
```

```
searchInput$ = fromEvent<HTMLInputElement>(searchBox, 'input');
```

```
searchInput$.pipe(
```

```
map(event => (event.target as HTMLInputElement).value),
debounceTime(300), // wait 300ms after typing stops
distinctUntilChanged() // only emit if value changed
).subscribe(value => this.searchCustomers(value));
```

3. Manual creation

```
import { Observable, Subject } from 'rxjs';
private searchSubject = new Subject<string>();
search$ = this.searchSubject.asObservable();
searchCustomers(term: string) {
  this.searchSubject.next(term); // trigger search
}
```

Essential Operators (Chaining)

Operators transform/filter/combine streams without side effects (pure functions).

Creation + Transformation

```
import { of, combineLatest, forkJoin } from 'rxjs';
import { map, switchMap, debounceTime, distinctUntilChanged } from 'rxjs/operators';
```

```
const customers$ = this.http.get<Customer[]>('/api/customers');
const selectedCustomerId$ = this.customerForm.get('id')!.valueChanges;
```

// Transform single value

```
highValueCustomers$ = customers$.pipe(
  map(customers => customers.filter(c => c.totalSpent > 1000))
);
```

// Switch to new HTTP when customer changes

```
customerDetails$ = selectedCustomerId$.pipe(
  switchMap(id => this.http.get(`/api/customers/${id}`))
);
```

Combining streams

// Wait for BOTH to complete

```
forkJoin({
  customers: this.http.get('/api/customers'),
  products: this.http.get('/api/products')
}).subscribe(({ customers, products }) => {
  // Both loaded
});
```

// Emit when EITHER changes

```
combineLatest([customer$, invoice$]).pipe(
  map(([customer, invoice]) => ({
    customerName: customer.name,
    totalDue: invoice.total
  }))
);
```

Time-based Operators (Critical for UX)

```
import { debounceTime, throttleTime, retry, catchError } from 'rxjs/operators';
```

```
import { timer, EMPTY } from 'rxjs';
```

// Search as user types (debounce)

```
searchTerms$ = fromEvent(searchInput, 'input').pipe(
  map((event: any) => event.target.value),
  debounceTime(300), // wait 300ms after last keystroke
  distinctUntilChanged(), // skip if same as previous
  switchMap(term => this.http.get(`/api/customers?search=${term}`))
);
```

// Rate-limit expensive operations

```
saveClicks$ = fromEvent(saveBtn, 'click').pipe(
```

```
throttleTime(2000), // max 1 save per 2 seconds
switchMap(() => this.saveCustomer())
);
```

Error Handling

```
customer$ = this.http.get<Customer>('/api/customers/123').pipe(
  retry(3), // retry 3 times
  catchError(error => {
    console.error('Customer load failed', error);
    return of({ name: 'Unknown', id: 0 }); // fallback value
  })
);
```

Template error handling:

```
<div *ngIf="customer$ | async as customer; else loading">
  {{ customer.name }}
</div>
<ng-template #loading>Loading...</ng-template>
```

Practical Billing System Examples

1. Real-time filtered invoice list

```
export class InvoiceListComponent {
  searchTerm$ = new BehaviorSubject("");
  invoices$ = this.invoiceService.getInvoices();

  filteredInvoices$ = combineLatest([
    this.invoices$,
    this.searchTerm$
  ]).pipe(
    map(([invoices, term]) =>
      invoices.filter(inv =>
        inv.customerName.toLowerCase().includes(term.toLowerCase())
      )
    )
  );
}
```

```
)  
)  
);
```

```
onSearch(term: string) {  
  this.searchTerm$.next(term);  
}  
}  
  
<input (input)="onSearch($event.target.value)" placeholder="Search invoices">  
<div *ngFor="let inv of (filteredInvoices$ | async); trackBy: trackById">  
  {{ inv.customerName }}  
</div>
```

2. Form + API coordination

```
customerForm = this.fb.group({ name: "", email: "" });  
saveCustomer() {  
  this.customerForm.valueChanges.pipe(  
    debounceTime(500),  
    switchMap(form => this.http.post('/api/customers', form))  
  ).subscribe();  
}
```

3. Concurrent API calls with progress

```
loadDashboard() {  
  const requests = {  
    customers: this.http.get('/api/customers'),  
    invoices: this.http.get('/api/invoices'),  
    payments: this.http.get('/api/payments')  
  };  
  
  forkJoin(requests).pipe(  
    map(data => ({  
      totalRevenue: data.invoices.reduce((sum, i) => sum + i.amount, 0),
```

```
        customerCount: data.customers.length
    )))
    ).subscribe(
      dashboard => this.dashboardData = dashboard
    );
  }
}
```

Memory Management (Avoid Leaks)

Use async pipe in templates (auto-unsubscribes):

```
<div *ngIf="data$ | async as data">{{ data.value }}</div>
```

Manual subscriptions (use takeUntil):

```
private destroy$ = new Subject<void>();
```

```
ngOnInit() {
  this.dataService.getData().pipe(
    takeUntil(this.destroy$)
  ).subscribe();
}
```

```
ngOnDestroy() {
  this.destroy$.next();
  this.destroy$.complete();
}
```

Common Patterns for Billing Apps

Pattern	Use Case	Operators
Search	Live filtering	debounceTime, distinctUntilChanged, switchMap
Form validation	Real-time validation	valueChanges, debounceTime
Pagination	Infinite scroll	switchMap, finalize

Pattern	Use Case	Operators
Caching	Repeated lookups	shareReplay(1)
Retry failed API	Network issues	retry(3), catchError

Quick Reference: Most Used Operators

// Transform

```
map(x => x * 2)
```

```
pluck('user', 'name')
```

// Filter

```
filter(x => x > 10)
```

```
distinctUntilChanged()
```

// Time

```
debounceTime(300)
```

```
throttleTime(1000)
```

// Combine

```
combineLatest([a$, b$])
```

```
forkJoin({a: a$, b: b$})
```

// Switch/cancel previous

```
switchMap(() => httpCall())
```

1. map - Transform data

Transforms each emitted value (like Array.map but for streams).

```
import { map } from 'rxjs/operators';
```

```
import { of } from 'rxjs';
```

```
const invoices$ = of([
```



```

    { id: 1, amount: 100, taxRate: 18 },
    { id: 2, amount: 200, taxRate: 12 }
  ]);

```

// Add tax to each invoice amount

```

invoicesWithTax$ = invoices$.pipe(
  map(invoices =>
    invoices.map(inv => ({
      ...inv,
      totalWithTax: inv.amount + (inv.amount * inv.taxRate / 100)
    }))
  )
);

```

Template usage:

```

<div *ngFor="let inv of (invoicesWithTax$ | async)">
  {{ inv.amount | currency }} + {{ inv.totalWithTax | currency }}
</div>

```

2. filter - Filter values

Only emits values that pass a condition (like Array.filter).

```
import { filter } from 'rxjs/operators';
```

```

overdueInvoices$ = this.invoices$.pipe(
  filter(invoices =>
    invoices.some(inv => new Date(inv.dueDate) < new Date())
  ),
  map(invoices => invoices.filter(inv => new Date(inv.dueDate) < new Date()))
);

```

Filter high-value customers:

```

vipCustomers$ = this.customers$.pipe(
  map(customers => customers.filter(c => c.totalSpent > 10000))
);

```

3. switchMap - Switch to new Observable

Cancels previous inner observable when a new value arrives. Perfect for search-as-you-type, form changes.

```
import { switchMap } from 'rxjs/operators';  
import { fromEvent } from 'rxjs';  
import { debounceTime, distinctUntilChanged } from 'rxjs/operators';
```

```
ngOnInit() {  
  // Search customers as user types (cancels previous requests)  
  fromEvent(this.searchInput.nativeElement, 'input').pipe(  
    map((event: any) => event.target.value),  
    debounceTime(300),  
    distinctUntilChanged(),  
    switchMap(searchTerm =>  
      this.http.get(`/api/customers?search=${searchTerm}`)  
    )  
  ).subscribe(customers => {  
    this.filteredCustomers = customers;  
  });  
}
```

Form value changes → API call:

```
this.customerForm.get('id')!.valueChanges.pipe(  
  switchMap(customerId =>  
    this.http.get(`/api/customers/${customerId}`)  
  )  
).subscribe(customer => this.selectedCustomer = customer);
```

4. combineLatest - Combine multiple streams

Emits when ANY source emits, combining latest values from ALL sources.

```
import { combineLatest } from 'rxjs';
```

```
loadDashboard() {
```

```

combineLatest({
  customers: this.http.get('/api/customers'),
  invoices: this.http.get('/api/invoices'),
  payments: this.http.get('/api/payments')
}).pipe(
  map(({ customers, invoices, payments }) => ({
    totalCustomers: customers.length,
    totalRevenue: invoices.reduce((sum, i) => sum + i.amount, 0),
    pendingPayments: payments.filter(p => !p.paid).length
  }))
).subscribe/dashboard => {
  this.dashboardData = dashboard;
});
}

```

Invoice filtering by customer + status:

```

<input [(ngModel)]="selectedCustomerId">
<select [(ngModel)]="statusFilter">

```

```

<div *ngFor="let inv of (filteredInvoices$ | async)">
  {{ inv.customerName }}
</div>

```

```

filteredInvoices$ = combineLatest({
  customerId: this.customerId$,
  status: this.statusFilter$
}).pipe(
  switchMap(({ customerId, status }) =>
    this.http.get(`/api/invoices?customer=${customerId}&status=${status}`)
  )
);

```

5. debounceTime - Wait for pause

Waits specified time after last emission before passing value downstream. Perfect for search, typing.

```
import { debounceTime } from 'rxjs/operators';

searchCustomers() {
  fromEvent(this.searchInput.nativeElement, 'input').pipe(
    map(event => (event.target as HTMLInputElement).value),
    debounceTime(400),    // Wait 400ms after user stops typing
    distinctUntilChanged(), // Only emit if value actually changed
    switchMap(term => term.length > 2
      ? this.http.get(`/api/customers/search?q=${term}`)
      : of([])
    )
  ).subscribe(customers => {
    this.searchResults = customers;
  });
}
```

6. async pipe - Auto-subscribe + unsubscribe

Subscribes to Observable in template and auto-unsubscribes (no memory leaks).

```
<!-- Perfect usage -->

<div *ngIf="customers$ | async as customers; else loading">
  <div *ngFor="let customer of customers; trackBy: trackById">
    {{ customer.name }}
  </div>
</div>

<ng-template #loading>Loading customers...</ng-template>

<!-- With combineLatest -->

<div *ngIf="dashboard$ | async as dashboard">
  Customers: {{ dashboard.totalCustomers }}
  Revenue: {{ dashboard.totalRevenue | currency }}
</div>
```

Multiple async pipes:

```
<div class="row">

  <div class="col-md-4">

    <!-- One async per section is ideal -->

    <div *ngFor="let inv of (overdueInvoices$ | async); trackBy: trackById">

      {{ inv.customerName }}

    </div>

  </div>

</div>
```

Complete Billing Dashboard Example

Puts all 6 operators together:

```
export class DashboardComponent {

  searchTerm$ = new BehaviorSubject("");
  customerFilter$ = new BehaviorSubject<any>(null);

  // Combine search + customer filter → API → transform
  filteredInvoices$ = combineLatest([

    this.searchTerm$,

    this.customerFilter$

  ]).pipe(

    debounceTime(300),          // 5. Wait for typing pause
    distinctUntilChanged(),
    switchMap(([search, customer]) => // 3. Cancel previous API call
      this.http.get(`/api/invoices?search=${search}&customer=${customer}`)
    ),
    map(invoices =>              // 1. Transform data
      invoices.filter(inv => inv.status === 'Pending') // 2. Filter
    )
  );

  constructor(private http: HttpClient) {}
}
```

```
<!-- Template with async pipe (6) -->
```

```
<input (input)="searchTerm$.next($event.target.value)">
```

```
<select (change)="customerFilter$.next($event.target.value)">
```

```
<div *ngFor="let inv of (filteredInvoices$ | async); trackBy: trackById">
```

```
  {{ inv.customerName }} - {{ inv.amount | currency }}
```

```
</div>
```

Operator	Purpose	Billing Example
map	Transform values	Add tax, format currency
filter	Filter values	Show only overdue invoices
switchMap	Switch to new HTTP	Search as you type
combineLatest	Combine streams	Dashboard: customers + invoices + payments
debounceTime	Wait for pause	Live search input
async pipe	Template subscription	Auto-unsubscribe in HTML

Data Binding

Data binding in Angular is how you connect your TypeScript component class (data/logic) with the HTML template (UI). Angular supports several binding types that together cover almost all ways data can flow between component and view.

Main types of data binding

Angular data binding is often grouped as:

1. One-way (component → template)
 - Interpolation
 - Property binding
 - Attribute / class / style binding
2. One-way (template → component)
 - Event binding
3. Two-way (component ↔ template)
 - Two-way binding with [(ngModel)]

Below each type is shown with a simple billing example (like customer or invoice forms).

Interpolation (one-way: component → template)

Used to display data as text or inside attribute-like places where Angular allows expressions, using {{ ... }}.

Example

```
// customer-summary.component.ts
export class CustomerSummaryComponent {
  customerName = 'John Doe';
  totalDue = 1250.5;
}
```

```
<!-- customer-summary.component.html -->
```

```
<h3>Customer: {{ customerName }}</h3>
```

```
<p>Total Due: {{ totalDue | currency }}</p>
```

- Data flows from component to view only.
 - Angular updates the text automatically when customerName or totalDue change.
-

Property binding (one-way: component → template)

Binds a DOM property (not just an HTML attribute) using [property]="expression".

Example

Disable “Pay Now” button if nothing is due:

```
export class InvoiceActionsComponent {  
  totalDue = 0;  
}
```

```
<button class="btn btn-success"  
  [disabled]="totalDue <= 0">
```

Pay Now

```
</button>
```

Other common property bindings:

```
<img [src]="productImageUrl">
```

```
<input [value]="customerName">
```

```
<div [hidden]="isLoading">Loaded!</div>
```

Attribute, class, and style binding

These are specialized forms of property binding for attributes Angular cannot reach as normal properties, and for dynamic CSS.

Attribute binding

```
<button [attr.aria-label]="'Pay ' + customerName">
```

Pay

```
</button>
```

Class binding

```
<tr [class.table-danger]="invoice.overdue"
```

```
  [class.table-success]="invoice.paid">
```

...

```
</tr>
```

Style binding

```
<div [style.color]="totalDue > 0 ? 'red' : 'green'">
```

```
  {{ totalDue | currency }}
```

```
</div>
```


These allow dynamic styling and accessibility based on component state.

Event binding (one-way: template → component)

Captures DOM events and calls component methods using (event)="handler(\$event)".

Example

Handle button click to save customer:

```
export class CustomerFormComponent {  
  saveCustomer() {  
    // call API, show message, etc.  
  }  
  
  onAmountChange(event: Event) {  
    const value = (event.target as HTMLInputElement).value;  
    console.log('New amount:', value);  
  }  
}
```

```
<button class="btn btn-primary"  
  (click)="saveCustomer()">  
  Save  
</button>
```

```
<input type="number"  
  (input)="onAmountChange($event)">
```

Common events: click, input, change, keyup, submit, etc.

Two-way binding (component ↔ template) with ngModel

Two-way binding keeps component property and form control value in sync, using [(ngModel)] which combines property + event binding under the hood.

You must import FormsModule in your module to use ngModel.

Example: customer name field

```
// customer-edit.component.ts  
  
export class CustomerEditComponent {  
  customerName = 'John Doe';  
}
```

```
<!-- customer-edit.component.html -->
```

```
<input type="text"
```

```
  [(ngModel)]="customerName"
```

```
  placeholder="Enter customer name">
```

```
<p>Preview: {{ customerName }}</p>
```

Here:

- When user types, customerName updates automatically.
- When customerName changes in code, the input shows the new value.

Conceptually, this is equivalent to:

```
<input [value]="customerName"
```

```
  (input)="customerName = $any($event.target).value">
```

Putting all bindings together (billing example)

A small invoice form using multiple binding types:

```
// invoice-form.component.ts
```

```
export class InvoiceFormComponent {
```

```
  customerName = 'John Doe';
```

```
  amount = 0;
```

```
  isSaving = false;
```

```
  save() {
```

```
    this.isSaving = true;
```

```
    // call API, then set isSaving = false
```

```
  }
```

```
}
```

```
<h3>New Invoice for {{ customerName }}</h3> <!-- Interpolation -->
```

```
<input type="text"
```

```
  [(ngModel)]="customerName">    <!-- Two-way -->
```

```
<input type="number"
```

```
  [(ngModel)]="amount">    <!-- Two-way -->
```

```

<button class="btn btn-primary"
  [disabled]="isSaving || amount <= 0" <!-- Property binding -->
  (click)="save()" <!-- Event binding -->
  {{ isSaving ? 'Saving...' : 'Save Invoice' }}
</button>

```

```

<p [class.text-danger]="amount <= 0"
  [class.text-success]="amount > 0" <!-- Class binding -->
  Amount: {{ amount | currency }}
</p>

```

This example shows:

- Interpolation for text.
- Two-way binding for form fields.
- Property binding for button disabled state.
- Event binding for click.
- Class binding for conditional styling.

Type	Syntax example	Direction
Interpolation	{{ totalDue }}	Component → Template
Property binding	[disabled]="totalDue <= 0"	Component → Template
Attribute/class/style	[class.error]="isError"	Component → Template
Event binding	(click)="save()"	Template → Component
Two-way binding (ngModel)	[(ngModel)]="customerName"	Component ↔ Template